Important Information

Thank you for choosing Freenove products!

Getting Started

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.
https://github.com/Freenove/Freenove_ESP32_S3_Display

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

sale@freenove.com

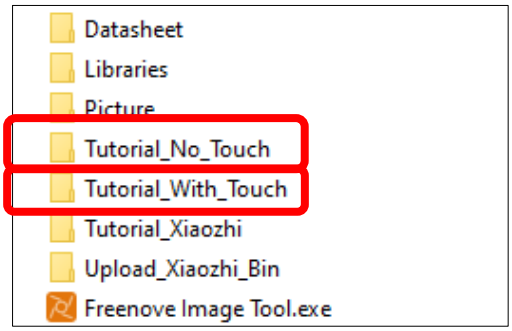
Copyright

FREENOVE

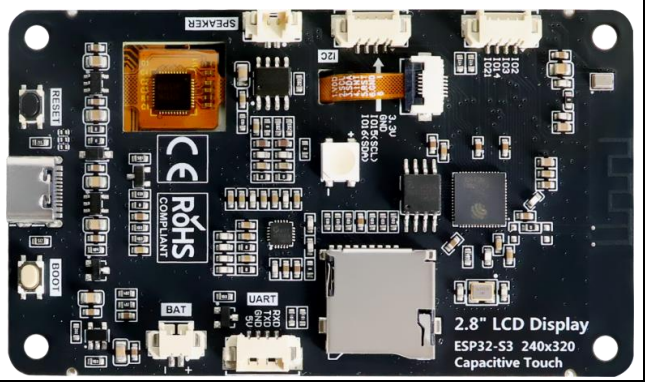
Contents

Freenove ESP32S3 Display

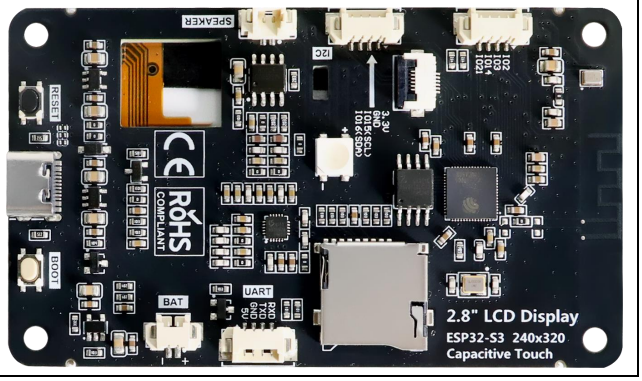
As of the time this tutorial was written, the Freenove ESP32-S3 Display series of development boards is available in two models: one equipped with a touchscreen and the other without a touch interface. Due to differences in hardware configuration, the tutorial content varies between the two models. To ensure you receive accurate guidance, please confirm your device model before use and select the tutorial documentation that matches it.



Touch

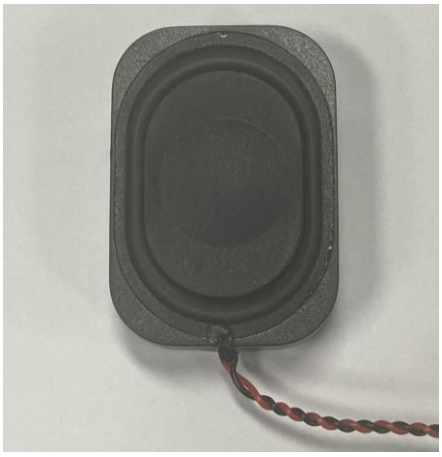
Top	Bottom
	

NonTouch

Top	Bottom
	

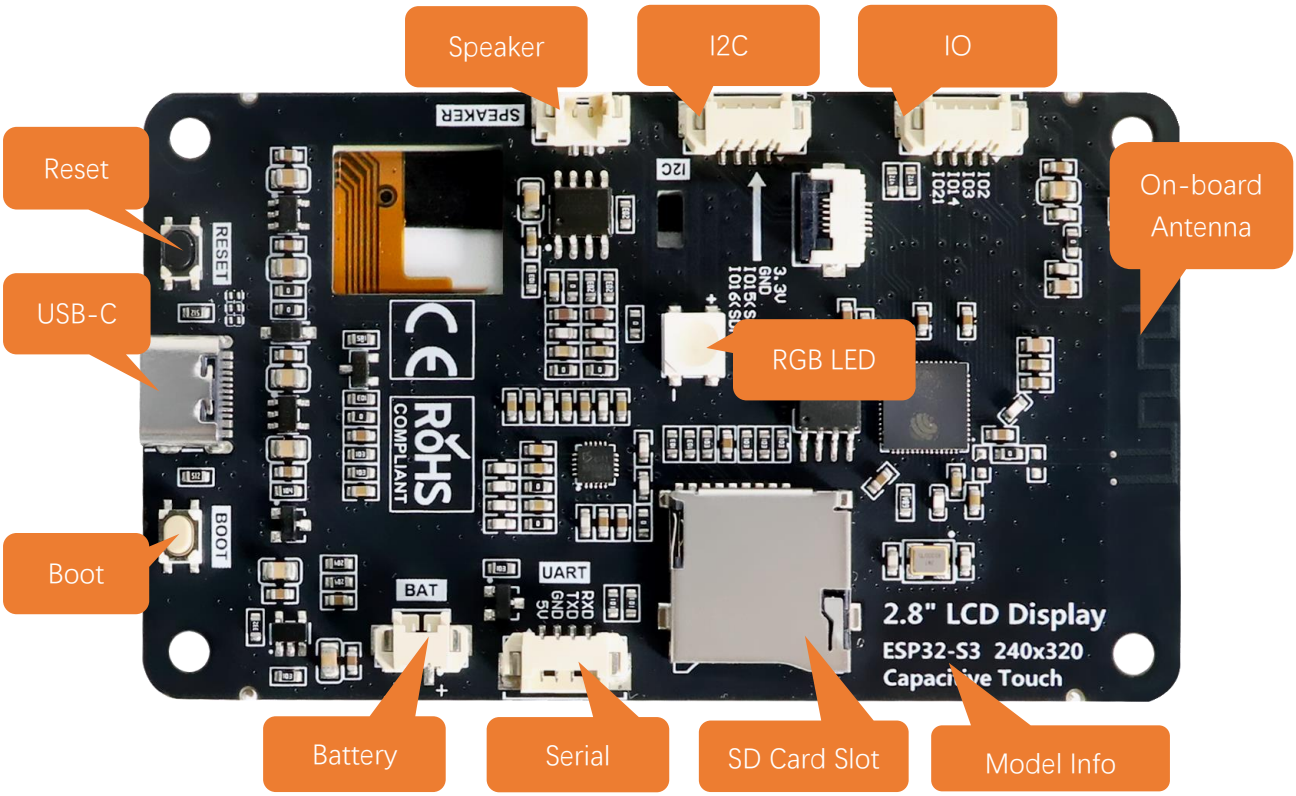
List

If you have any concerns, please feel free to contact us via support@freenove.com

Freenove ESP32 Display x 1 	USB-C Data Cable x 1 
Battery adapter cable x1 	Speaker x1 
Jumper Wire x1 	

Preface

Hardware Interfaces



Battery (Optional)

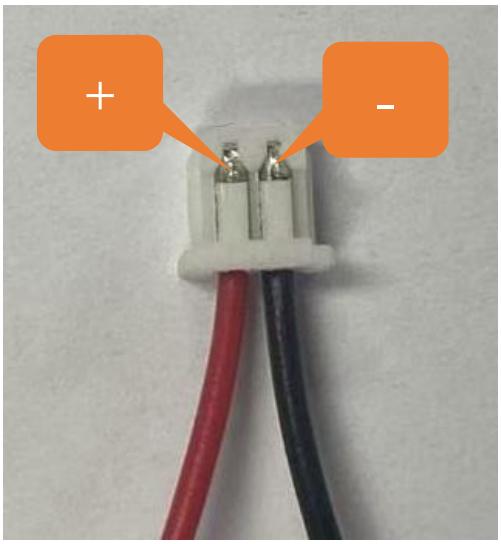
Please note that this product does not come with lithium batteries; please purchase them yourself.

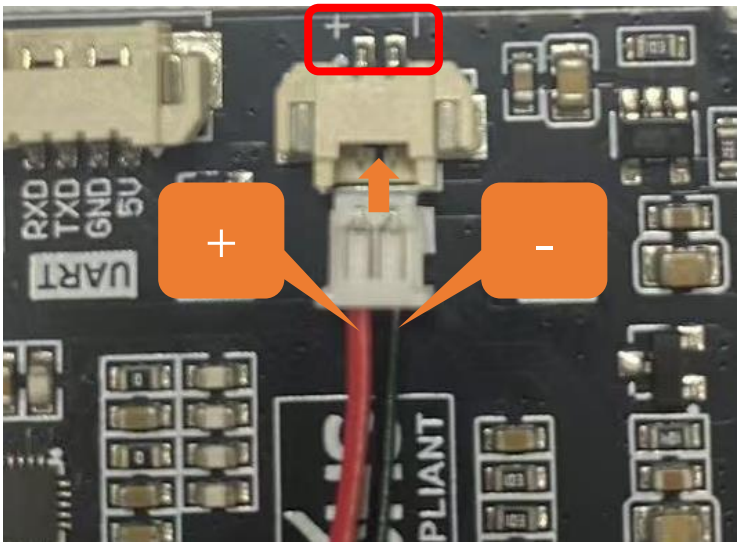
This device supports both **USB-powered and lithium battery-powered operation**. For optimal safety, USB power is recommended. Due to the **hazardous nature of lithium batteries**, we advise against their use unless absolutely necessary.

This device features an **MX1.25mm** connector and supports lithium batteries of various capacities. Note: The input voltage must be maintained within **3.7-4.2V** range.

Market-available batteries may feature **two distinct wiring configurations where the positive (+) and negative (–) terminals are reversed between models**. Please verify the battery's wiring matches the product requirements (refer to the diagram below) to prevent equipment failure or safety risks due to improper connection.

The red cable is the positive terminal while the black one is negative.

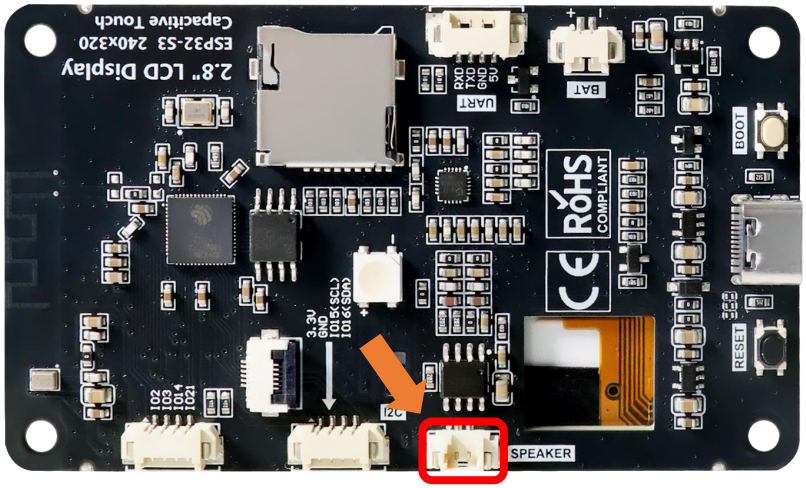




We recommend using a charger specially designed for lithium batteries. Due to various specifications and quality of lithium batteries, using a proper charger helps ensure peak performance, safety, and battery longevity.
While our product also supports USB charging as a backup option, please note that this method does not support fast charging and is limited to standard slow charging.

Speaker

There is a speaker connector (PH1.25mm) on the Freenove ESP32-S3 Display.



SD Card

This connector circuit employs the 4 - bit communication mode of SD MMC to establish data communication with the SD card and provides support for high - speed Micro SD card storage.

Item	Pins	Definition
SD Card	GPIO38	SD_CLK
	GPIO40	SD_CMD
	GPIO39	SD_D0
	GPIO41	SD_D1
	GPIO48	SD_D2
	GPIO47	SD_D3

Note: This product does not include SD cards or SD card readers. Please buy them yourself

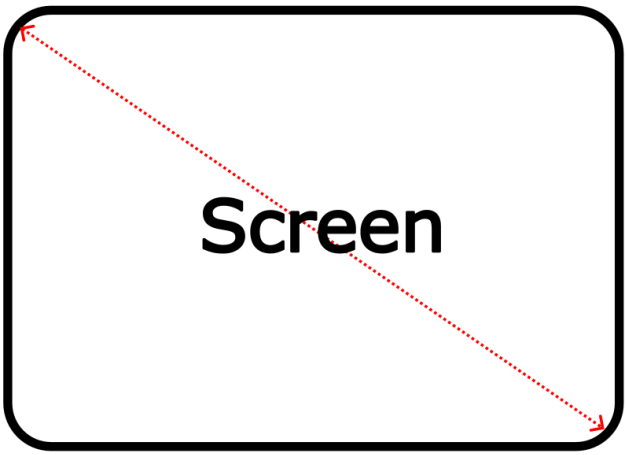
TFT Screen

Item	Pins	Definition
TFT Screen	GPIO10	TFT_CS
	GPIO11	TFT_MOSI
	GPIO12	TFT_SCK
	GPIO13	TFT_MISO
	GPIO45	TFT_BL
	GPIO46	TFT_DC

Screen Size

An internationally recognized unit of length, the **inch** (equivalent to 2.54 centimeters) has been the standard measurement for screen sizes in the display industry with a long history. This convention originated in the early television era when British manufacturers first adopted inches to measure cathode-ray tube (CRT) dimensions. The practice quickly became the global norm and remains the industry standard today.

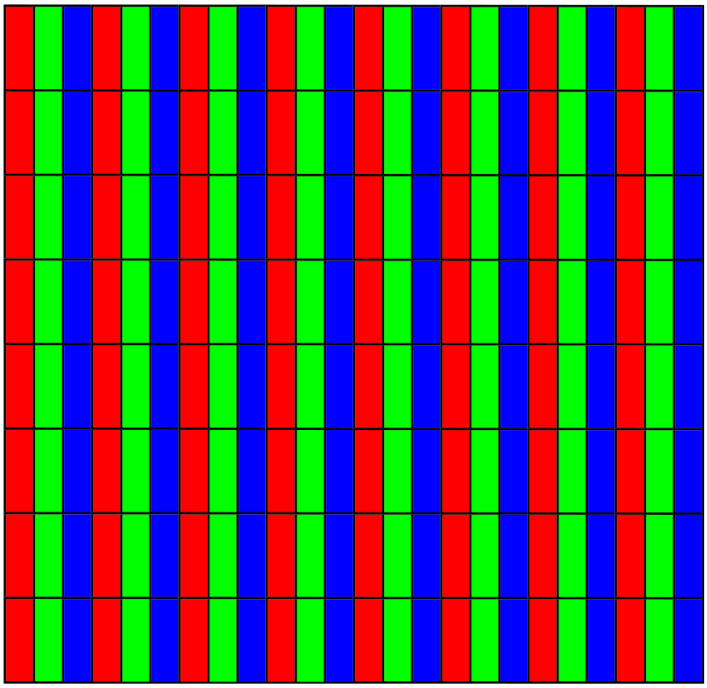
A critical clarification: screen size always refers to **the diagonal measurement of the display** panel. For instance, a 3.5-inch display features a 3.5-inch (8.89 cm) diagonal, ensuring standardized and comparable sizing across all devices.



Resolution

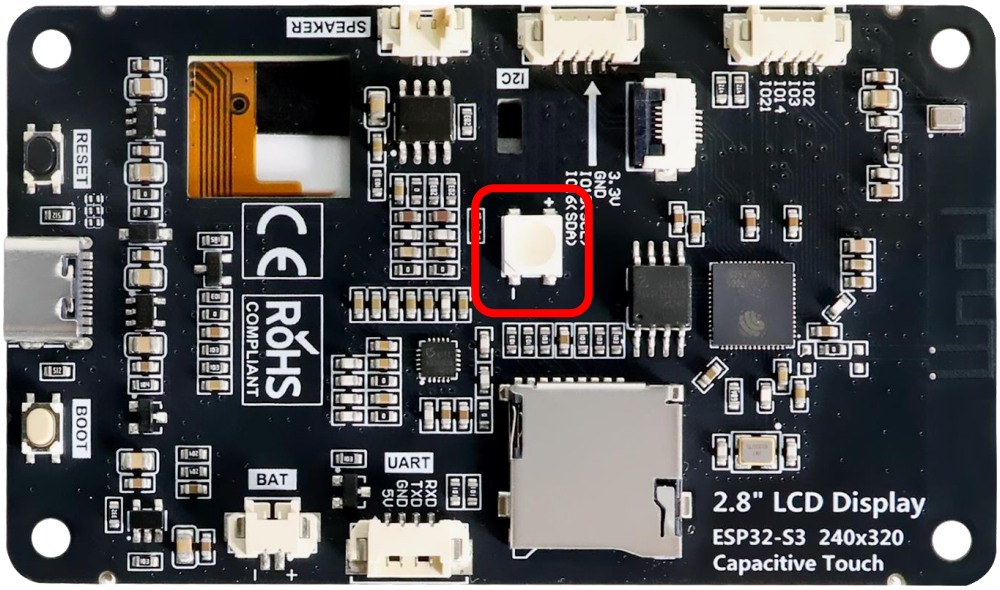
Screen resolution quantifies a display's pixel density along its horizontal and vertical axes, expressed as **width × height** (e.g., 320 × 480). This specification applies universally across displays, such as smartphones, monitors, and television screens.

- **Pixel**: The smallest unit of a display, composed of red (R), green (G), and blue (B) subpixels. Through precise brightness variation, these subpixels generate the complete color spectrum.
- **Resolution vs. Clarity**: Higher resolution means more pixels per unit area, resulting in sharper and more detailed imagery.

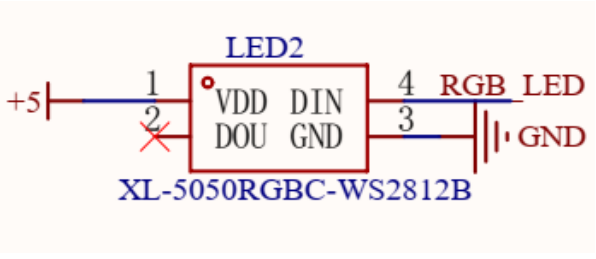


RGB LED

The Freenove ESP32-S3 Display includes an RGB LED (red, green, blue) that can blend colors to create various lighting effects.



Item	Pins
DIN	GPIO42



GPIO Pinout Table

To learn what each GPIO corresponds to, please refer to the following table.
The functions of the pins are allocated as follows:

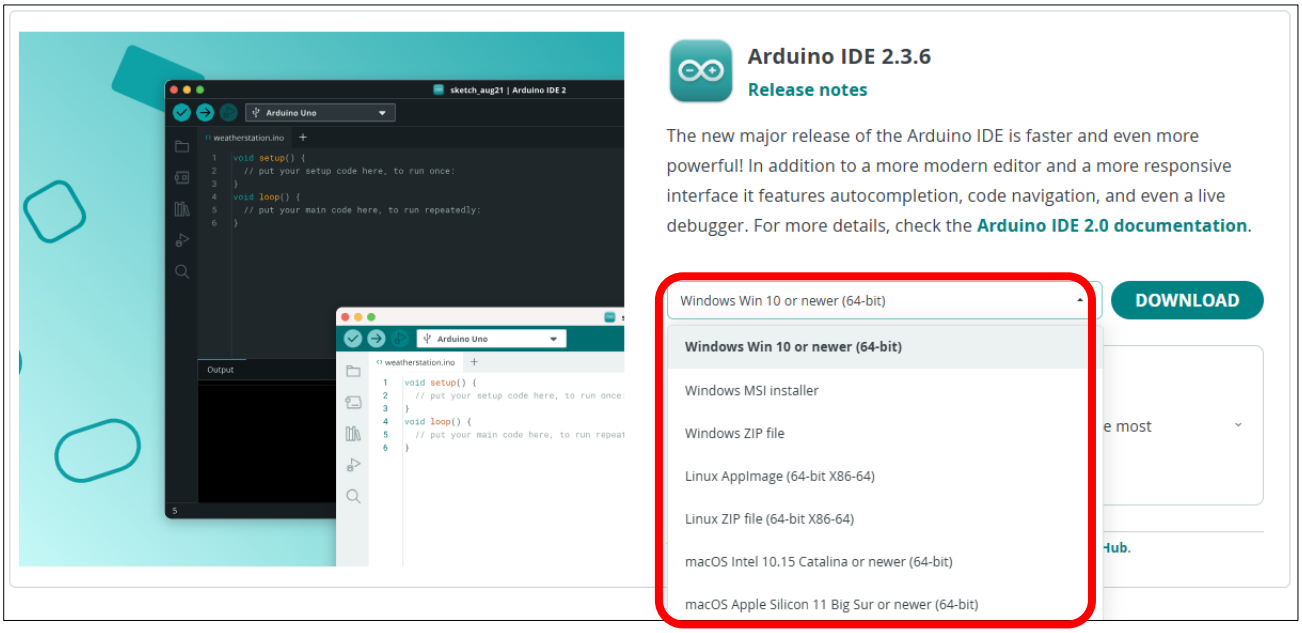
ESP32-S3	Functions	Description
GPIO42	DIN	RGB
GPIO38	SD_CLK	SD Card
GPIO40	SD_CMD	
GPIO39	SD_D0	
GPIO41	SD_D1	
GPIO48	SD_D2	
GPIO47	SD_D3	
GPIO10	TFT_CS	TFT_LCD
GPIO11	TFT_MOSI	
GPIO12	TFT_SCK	
GPIO13	TFT_MISO	
GPIO45	TFT_BL	
GPIO46	TFT_DC	
GPIO19	USB_D-	USB
GPIO20	USB_D+	
GPIO43	TXD0	Serial
GPIO44	RXD0	

For more information, refer to the schematic.

If you have any concerns, please feel free to contact us via support@freenove.com

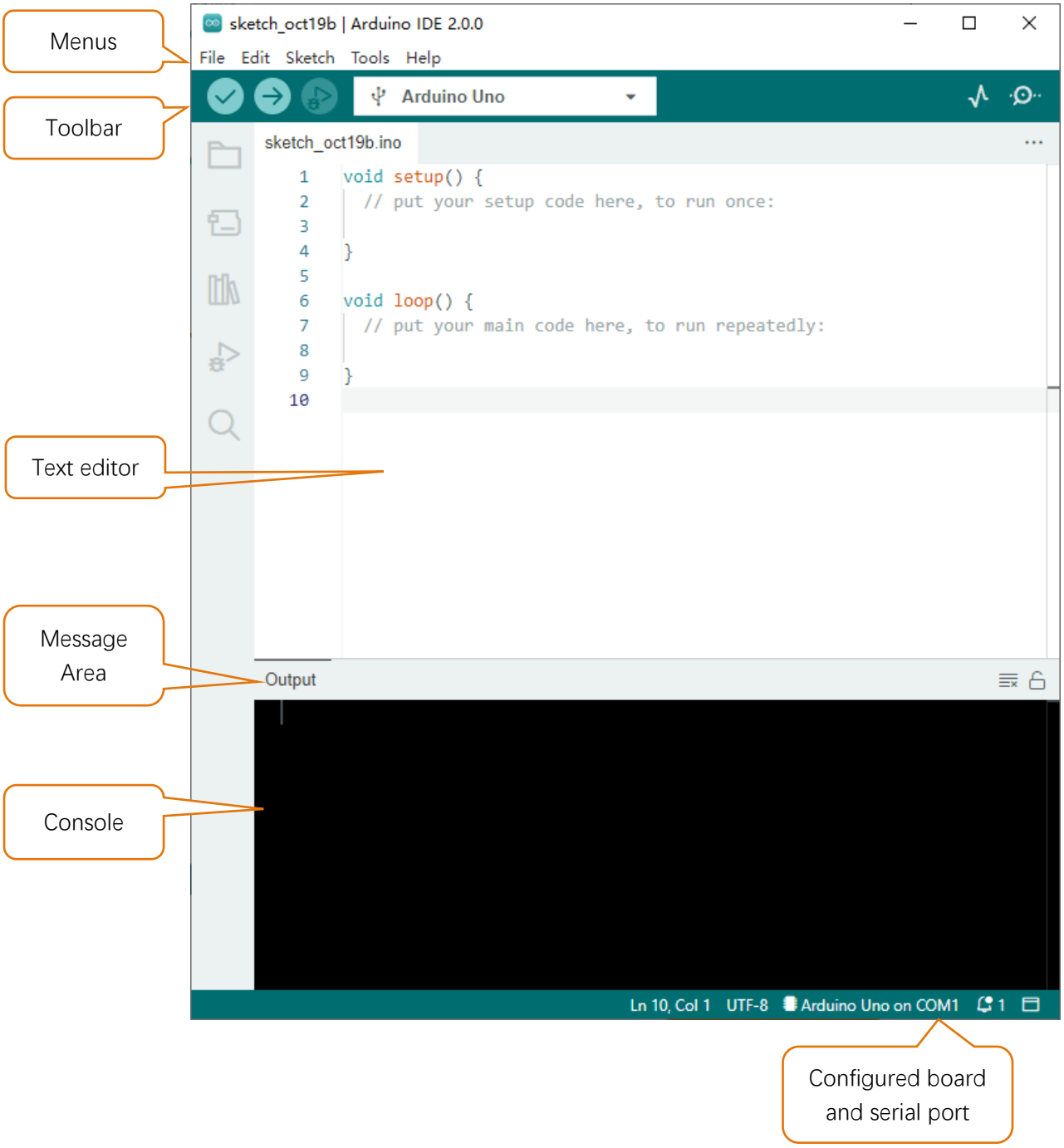
Programming Software

We use the Arduino Software (IDE) to write and upload the code for this product.
First, install Arduino Software (IDE): visit https://www.arduino.cc/en/software/, Select and download corresponding installer according to your operating system. If you are a Windows user, please select the "Windows" to download and install it correctly.

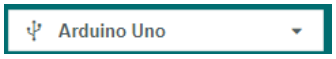


After the download completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it popes up, please allow the installation.
After installation completes, an Arduino Software shortcut will be generated in the desktop. Run the Arduino Software.

The interface of Arduino Software is as follows:



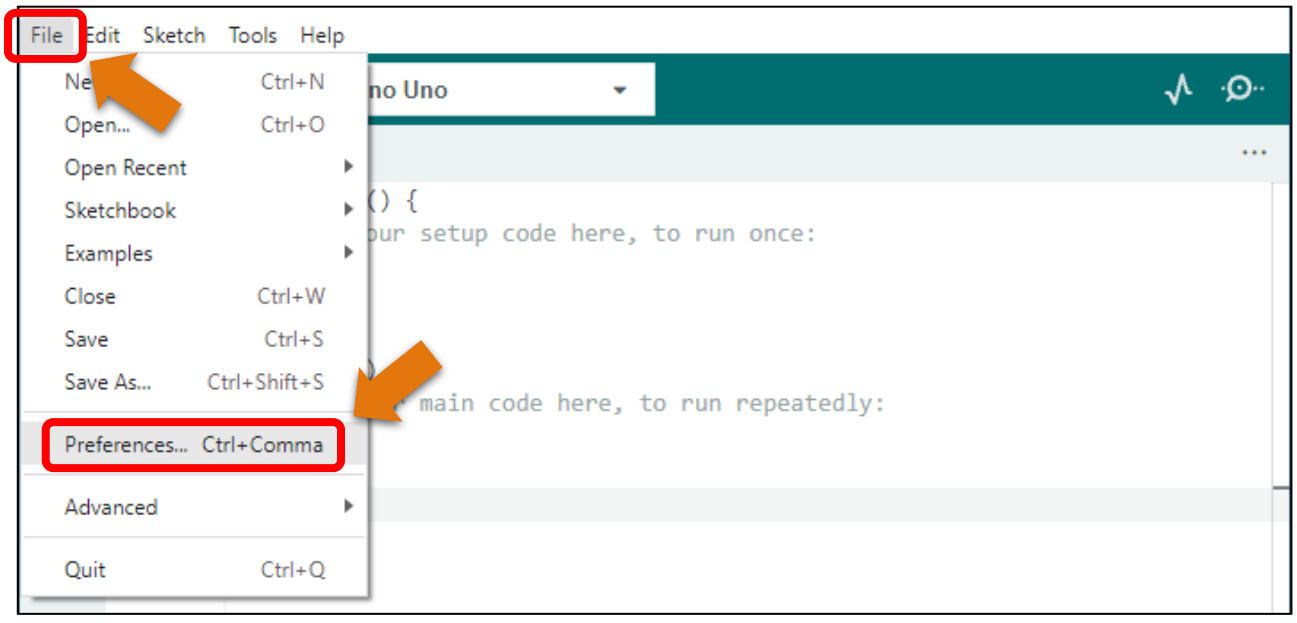
Programs written with Arduino Software (IDE) are called **sketches**. These sketches are written in the text editor and saved with the file extension**.ino**. The editor features text cutting/pasting and searching/replacing. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

	Verify Check your code for compile errors.
	Upload Compile your code and upload them to the configured board.
	Debug Debug code running on the board. (Some development boards do not support this function)
	Development board selection Configure the support package and upload port of the development board.
	Serial Plotter Receive serial port data and plot it in a discounted graph.
	Serial Monitor Open the serial monitor.

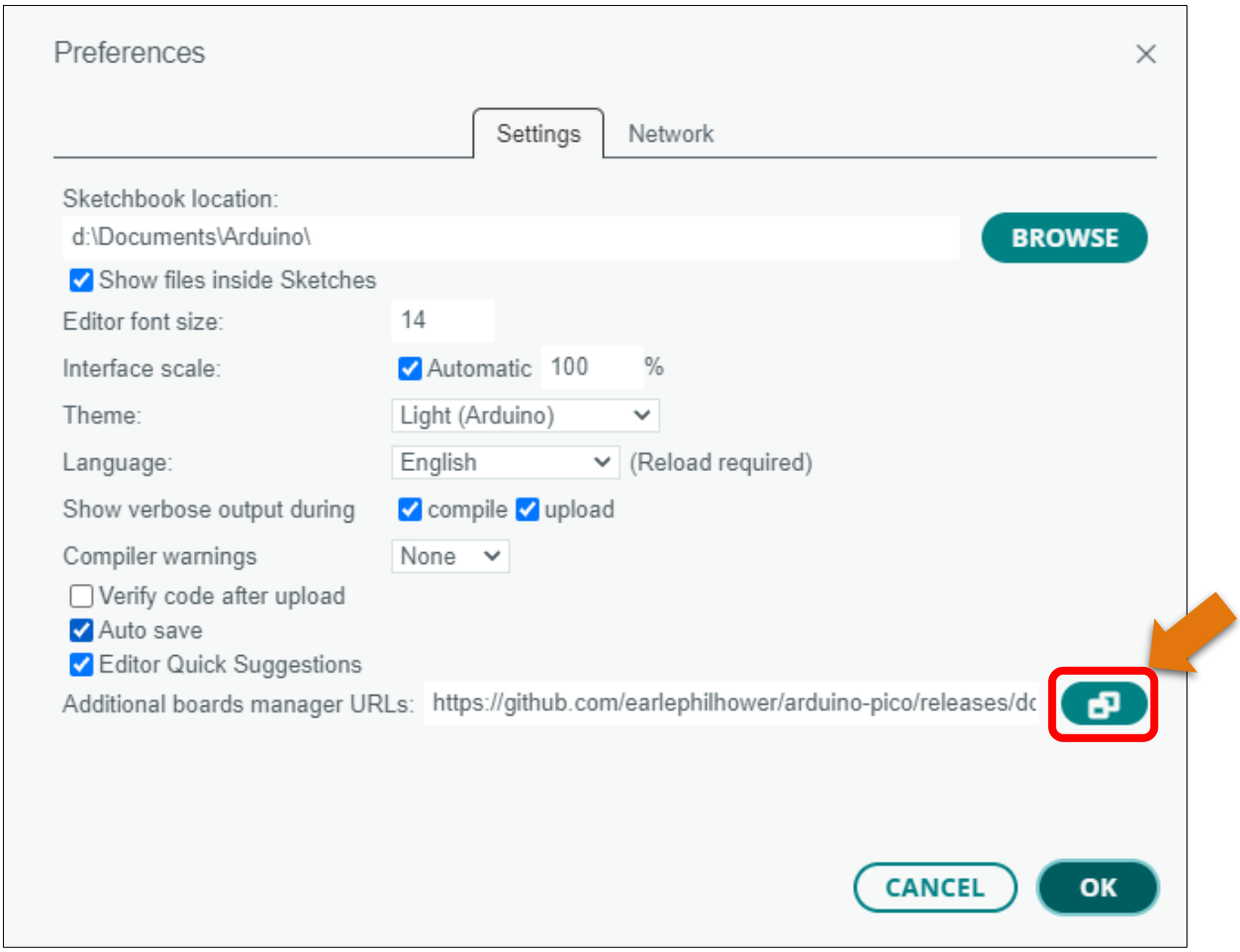
Additional commands are found within the five menus: File, Edit, Sketch, Tools, Help. The menus are context sensitive, which means only those items relevant to the work currently being carried out are available.

Environment Configuration

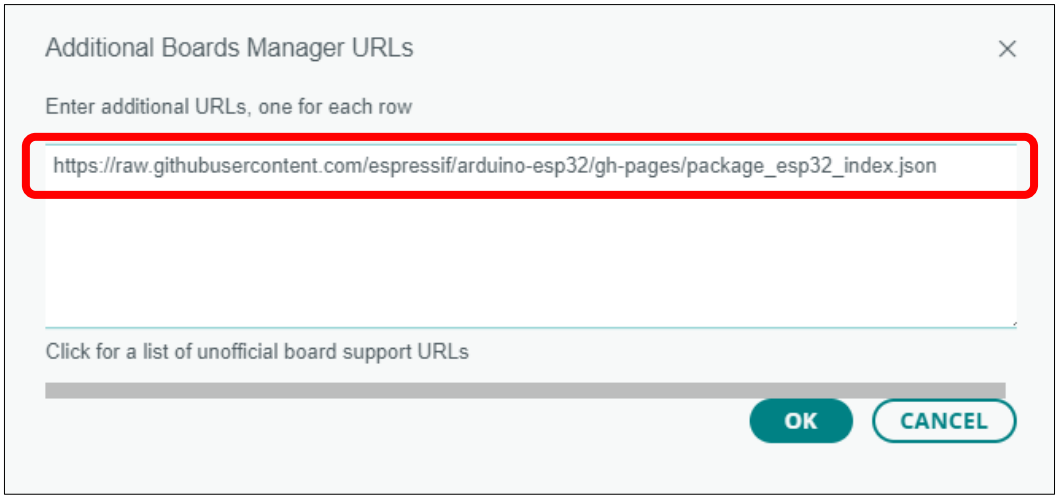
First, open the software platform Arduino, and then click File in Menus and select Preferences.



Second, click on the symbol behind "Additional Boards Manager URLs"

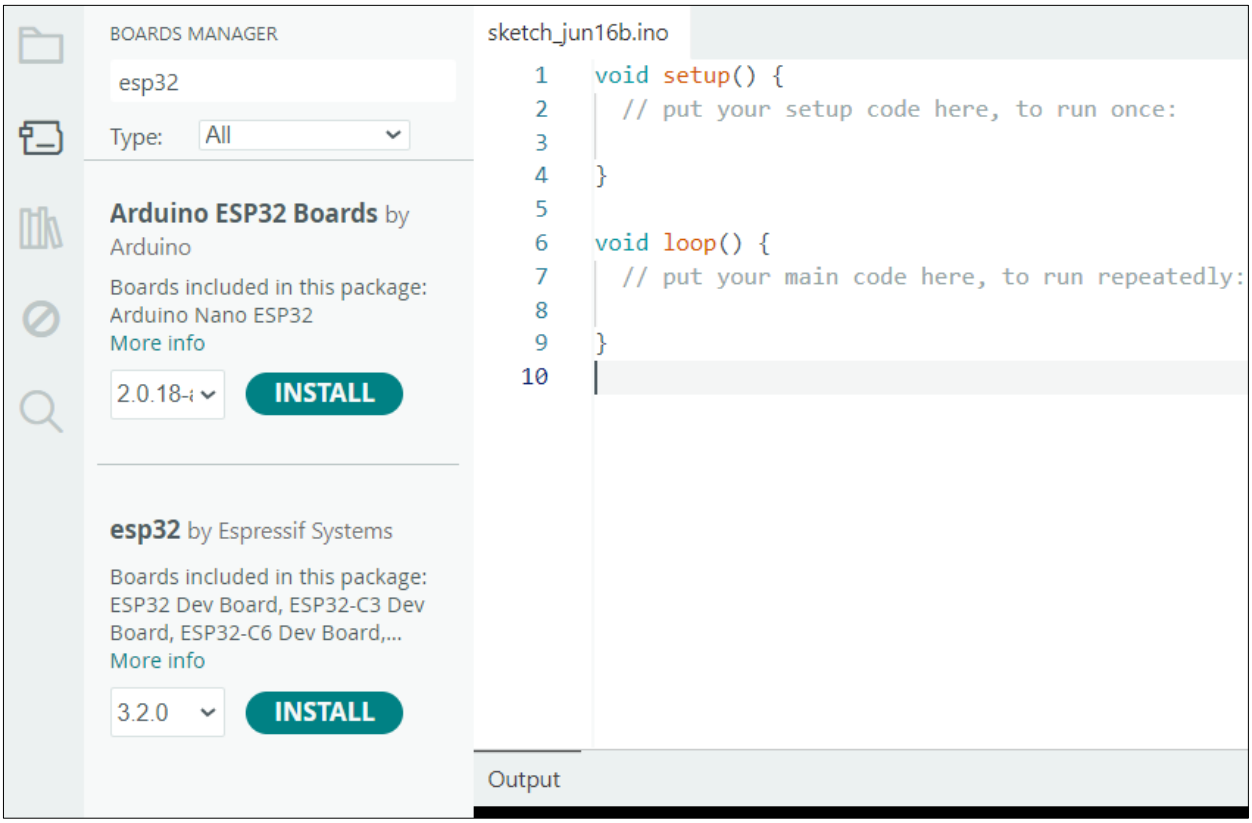


Third, fill in https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json in the new window, click OK, and click OK on the Preferences window again.

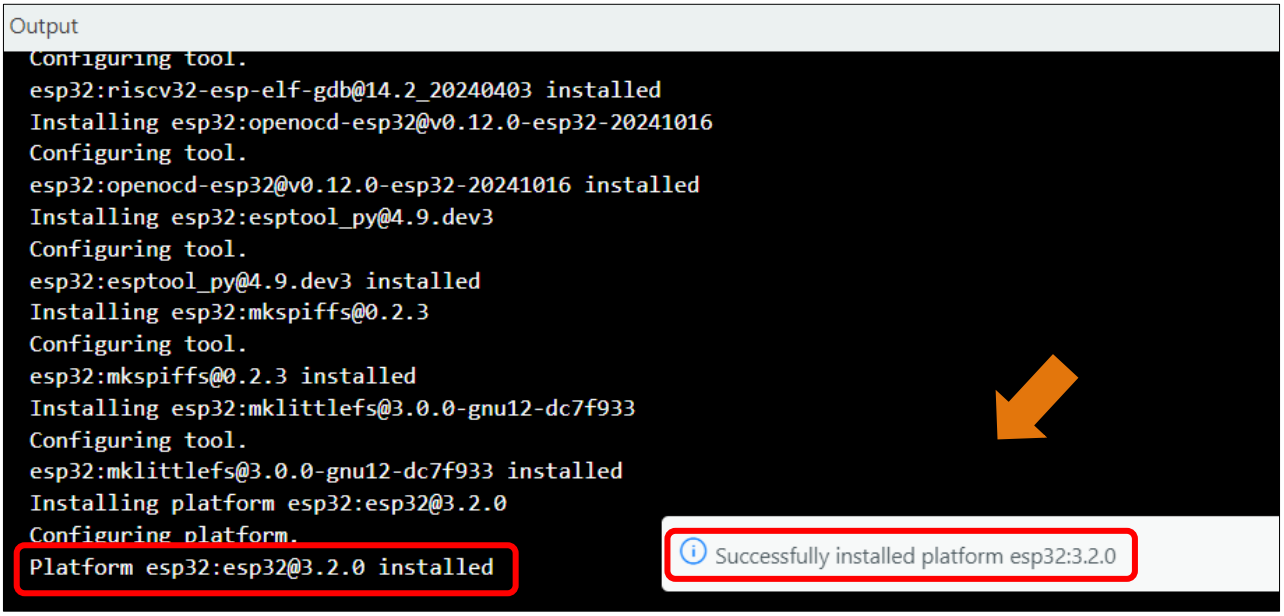


Note: if you copy and paste the URL directly, you may lose the "-". Please check carefully to make sure the link is correct.

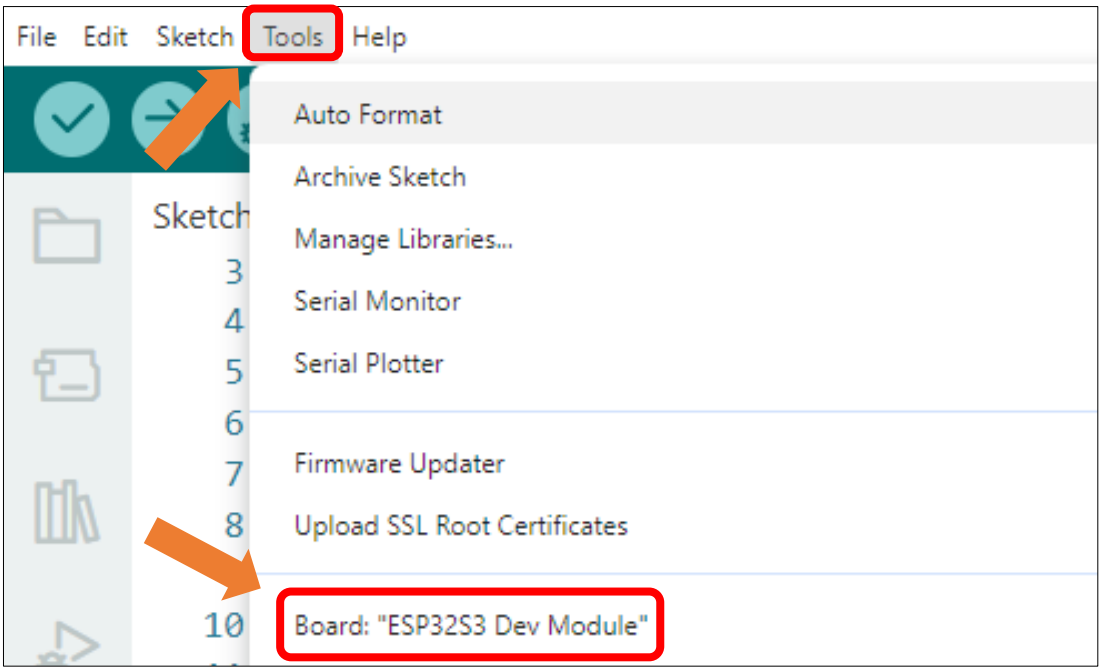
Fourth, click "Boards Manager". Enter **"esp32"** in Boards manager, select **3.2.0**, and click "INSTALL".



Arduino will download these files automatically. Wait for the installation to complete.



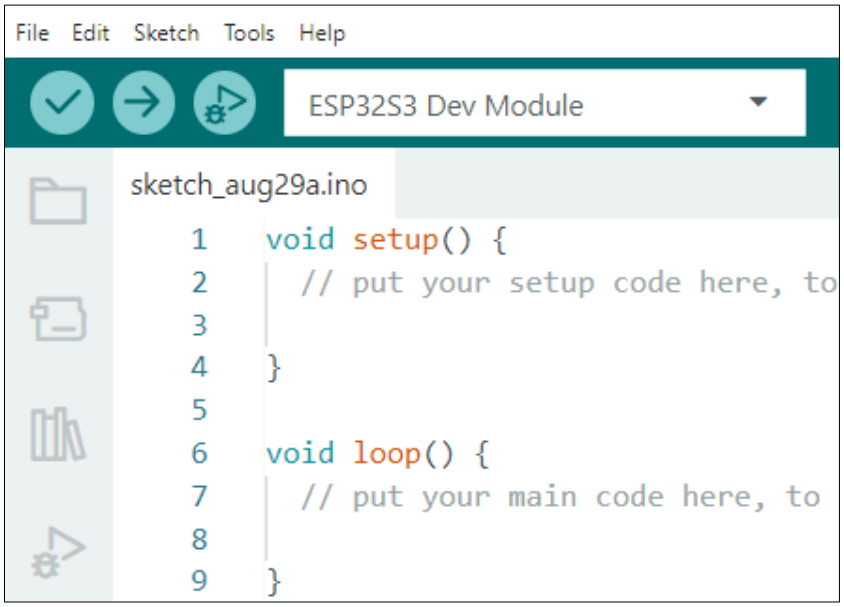
When finishing installation, click Tools in the Menus again and select Board: "ESP32S3 Dev Module", and then you can see information of ESP32S3.



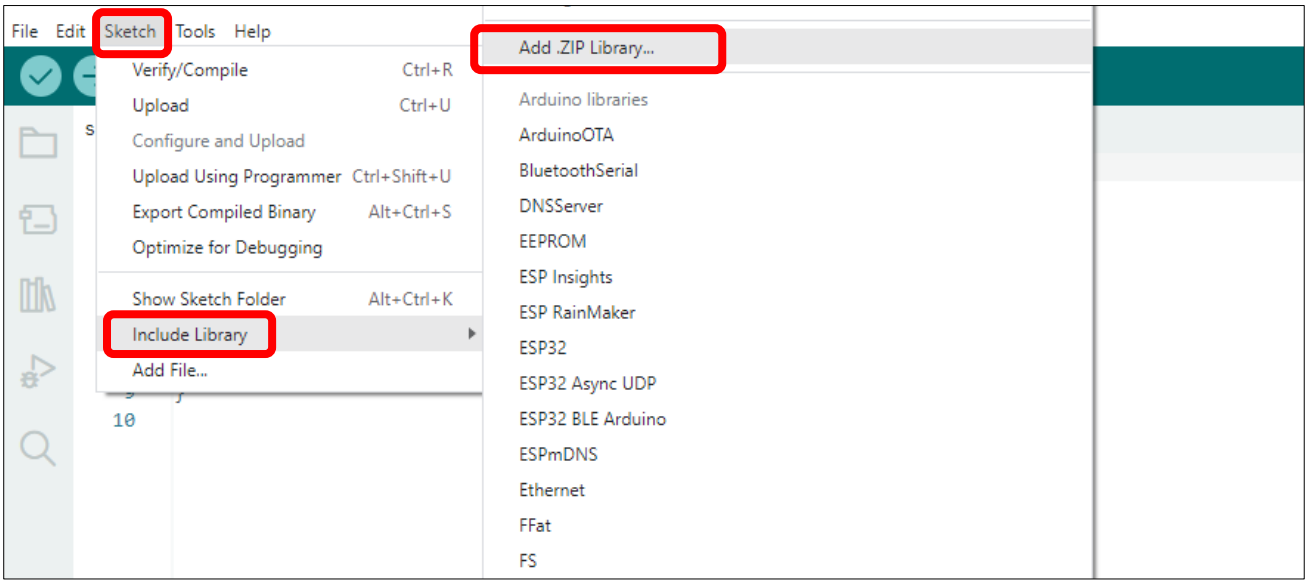
Library Installation

Before starting the learning process, it is necessary to install some libraries in advance to enable the code to be compiled properly. For convenience, we have already packaged these libraries and placed them in the **Freenove_ESP32_S3_Display/Libraries** folder. Please refer to the following steps to install these libraries into the Arduino IDE.

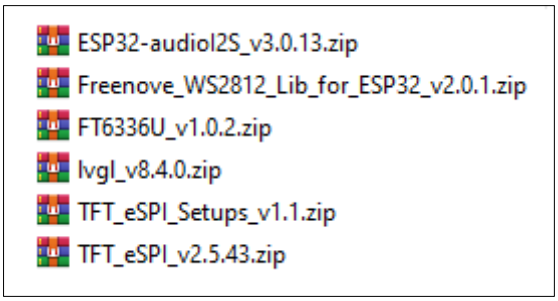
1. Open Arduino IDE.



2. Select Sketch->Include Library->Add .ZIP library….

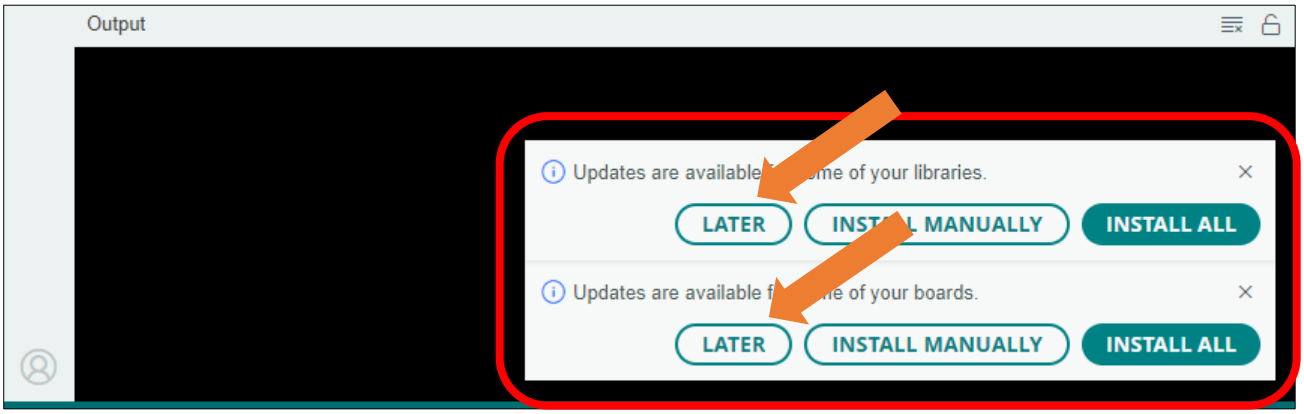


3. On the newly pop-up window, select the files from the **Freenove_ESP32_S3_Display/Libraries**. Click Open to install the library.



4. **Repeat the above steps until all the six libraries are installed to Arduino.** So far, all libraries have been installed.

Note: Some libraries are not the latest version. Please do not update them even if it prompts every time you open the IDE. Just click LATER. Otherwise, it may lead to compilation failure.



Chapter 1 Serial

Project 1.1 USB_Serial

The Freenove ESP32-S3 Display is equipped with an onboard USB2.0 port, which can be used to upload code to the ESP32-S3.

Meanwhile, it can also abstract the physical USB channel into a virtual COM port based on the USB CDC protocol, enabling the computer host to communicate with the ESP32-S3 through standard serial port tools without requiring additional hardware.

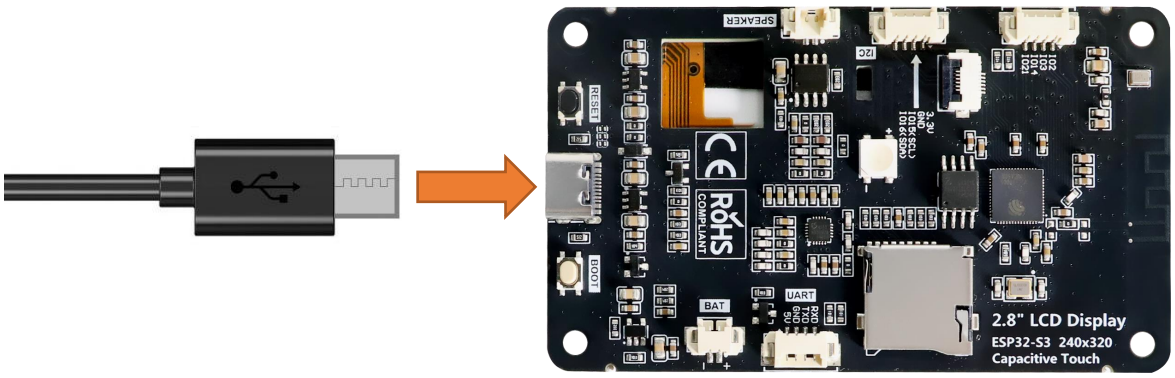
In this section, we will upload code via the USB interface and virtualize it as a serial port (COM) to enable data communication with the computer.

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

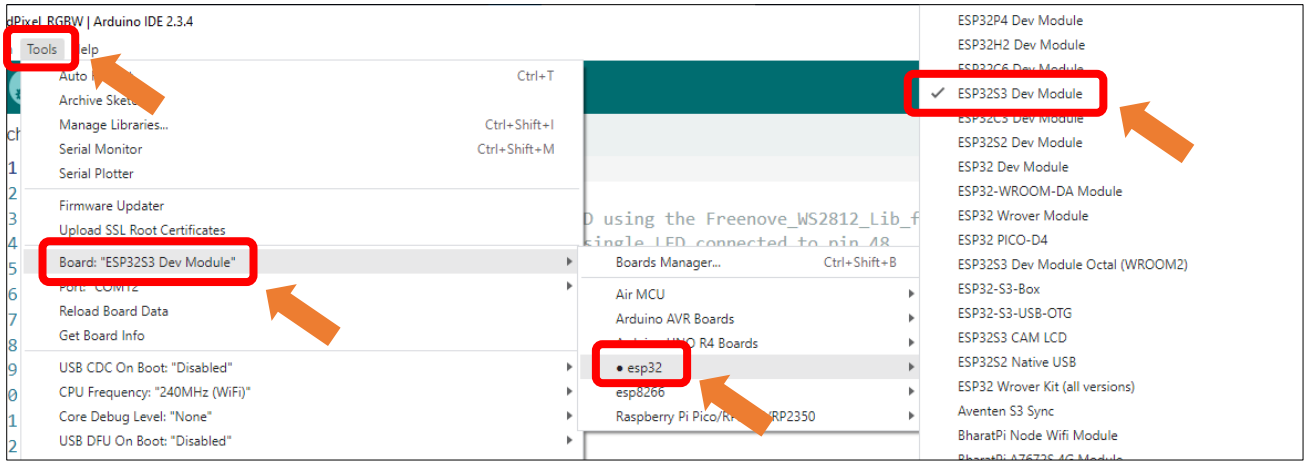
Connect Freenove ESP32-S3 Display to the computer with USB cable.



Configuration

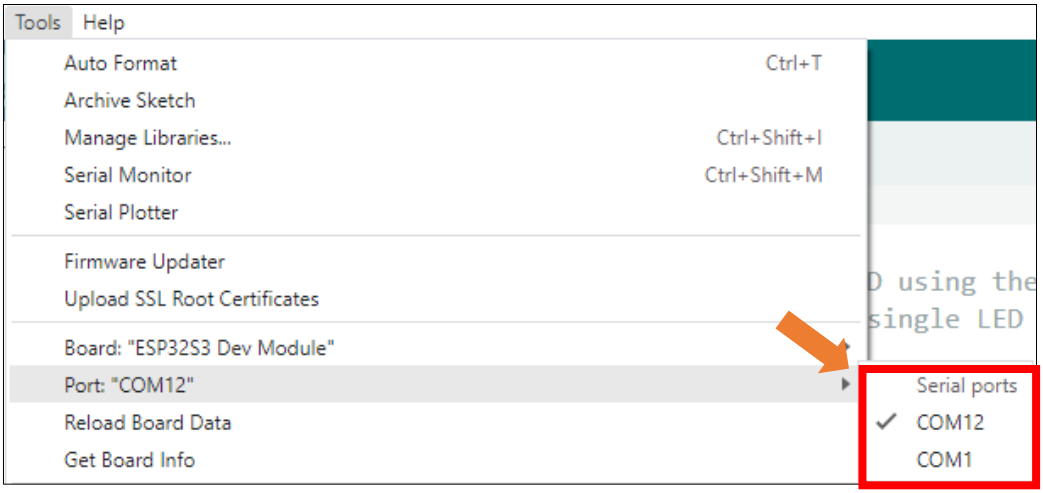
If you have not installed ESP32 SDK in Arduino IDE, please refer to Environment Configuration.

If you have installed it, please continue to proceed.
Select **Tools** –> **Board** -> **esp32** -> **ESP32S3 Dev Module**.

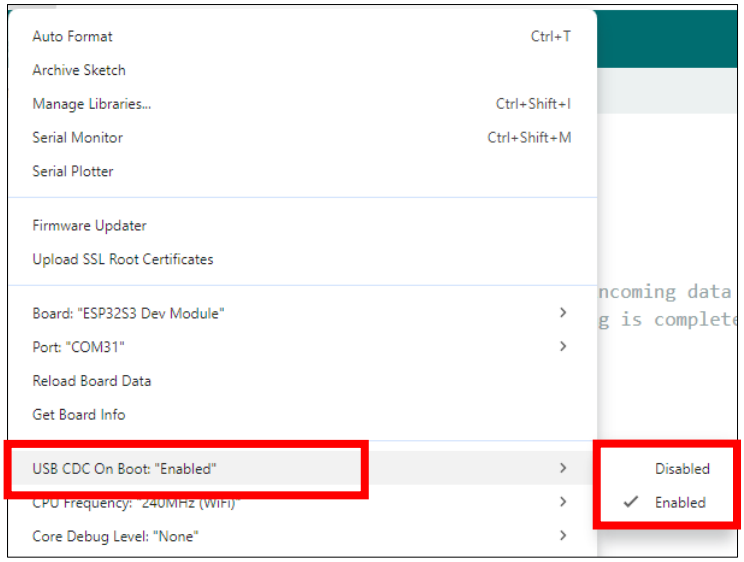


After connecting the Freenove ESP32-S3 Display, the system will assign a serial communication port named in the format 'COMx' (**where 'x' is a numeric ID that may vary across computers**). You must select the correct port under Tools → Port.

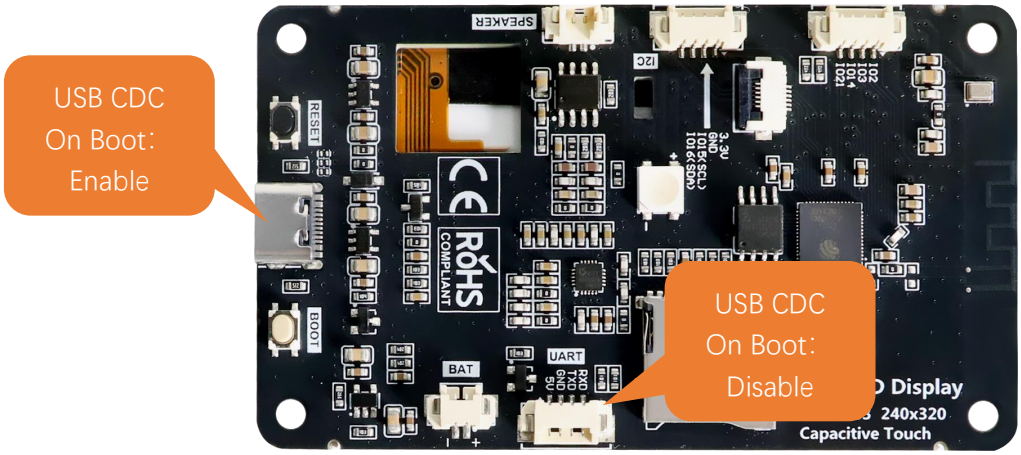
Note: COM1 is typically NOT the port of the Freenove ESP32-S3 Display.



Enable the "USB CDC On Boot" feature.



Please note that when "USB CDC On Boot" is set to "Enable", the ESP32-S3 will virtualize the onboard USB as a serial port after code upload, enabling serial communication with external devices via the data cable.



If “USB CDC On Boot” is set to “Disable”, the onboard USB interface can only be used to upload code.

Sketch

Open “**Sketch_01.1_SerialRW**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_01.1_SerialRW.ino**”.

Sketch_01.1_SerialRW

```
String inputString = "";      //a String to hold incoming data
bool stringComplete = false;  // whether the string is complete

void setup() {
   Serial.begin(115200);
   Serial.println(String("\nESP32-S3 initialization completed!\n")
                + String("Please input some characters,\n")
```

```
                + String("select \"Newline\" below and click send button. \n"));
}

void loop() {
  if (Serial.available()) {         // judge whether data has been received
    char inChar = Serial.read();          // read one character
    inputString += inChar;
    if (inChar == '\n') {
      stringComplete = true;
    }
  }
  if (stringComplete) {
    Serial.printf("InputString: %s", inputString);
    inputString = "";
    stringComplete = false;
  }
}
```

Code Explanation

Set the baud rate to 115200.

```
Serial.begin(115200);
```

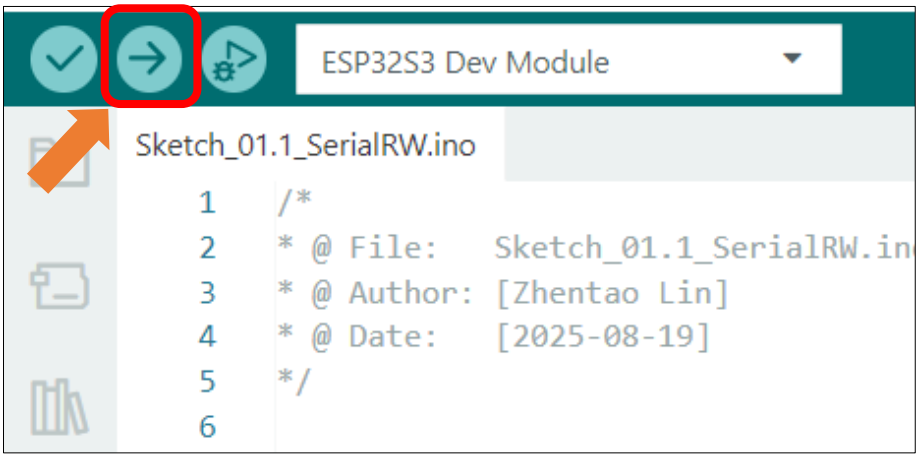
Determine whether there is data in the serial port buffer.

```
if (Serial.available())
```

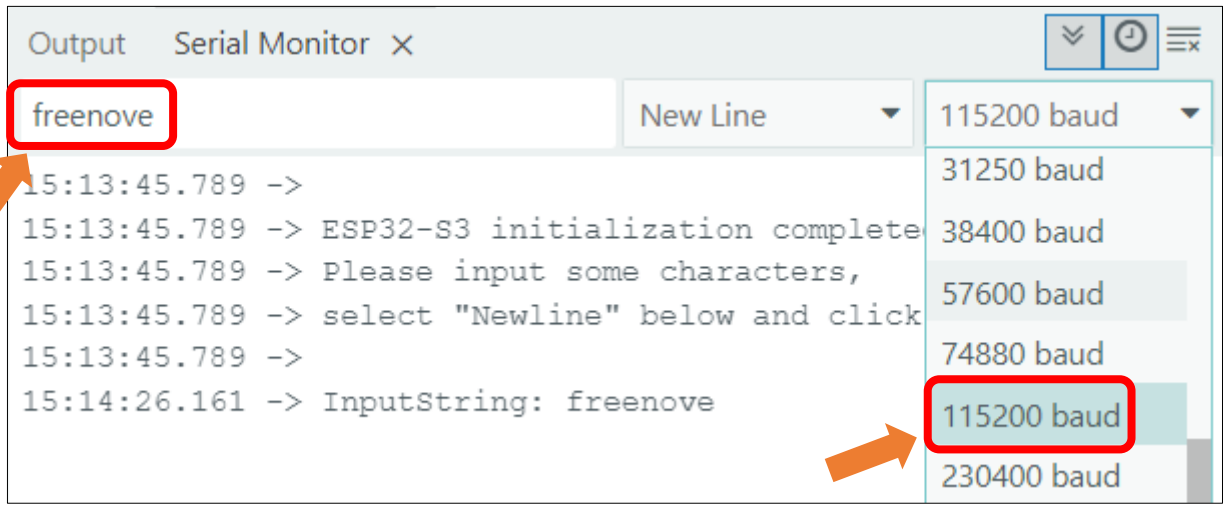
Receive serial port data and save it in the inputString string.

```
char inChar = Serial.read();          // read one character
inputString += inChar;
```

The purpose of this code is to display data on the serial monitor. Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.



After downloading the code, open the serial port monitor, and set the baud rate to **115200**, input any data in the messages bard and press Enter key, Freenove ESP32-S3 Display will print the received data.



If your serial monitor remains unresponsive, please verify that "USB CDC On Boot" is set to "Enable", then recompile and upload the code.

Important notes:

1. When "USB CDC On Boot" is enabled, the USB port serves both for code uploading and as a Serial interface, while the hardware UART port operates as Serial1.

2. When "USB CDC On Boot" is disabled, the USB port is used exclusively for code uploading and cannot function as a Serial interface. In this case, the UART port is Serial.

3. All examples in this tutorial require "USB CDC On Boot" to be enabled by default. To communicate with external devices via the onboard UART, use Serial1 instead, which shares identical usage methods with Serial.

Reference

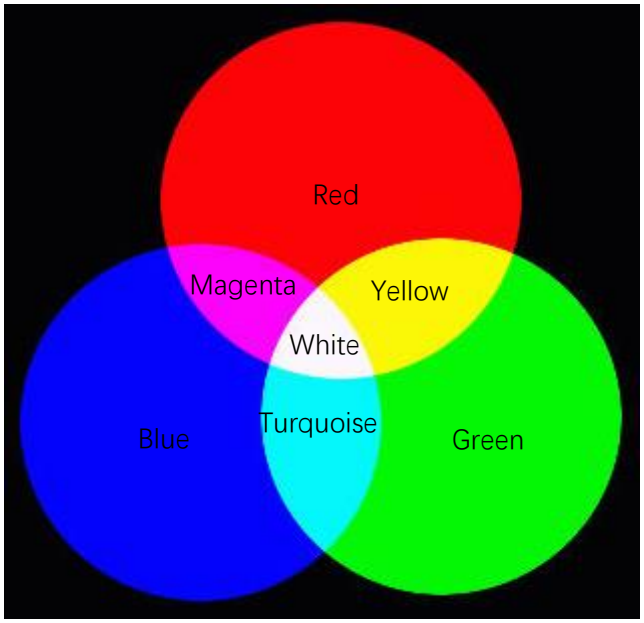
`int available()`
Serial.available() checks the number of bytes currently available to read in the Serial receive buffer. It returns the number of bytes available (int type), or 0 if the buffer is empty.
`int read ()`
Serial.read() reads one byte of data from the Serial receive buffer and returns it as an int. If no data is available to read, it returns -1.

Chapter 2 RGB

Project 2.1 RGB

Related Knowledge

Red, green, and blue are called the three primary colors. When you combine these three primary colors of different brightness, it can produce almost all kinds of visible light.



RGB

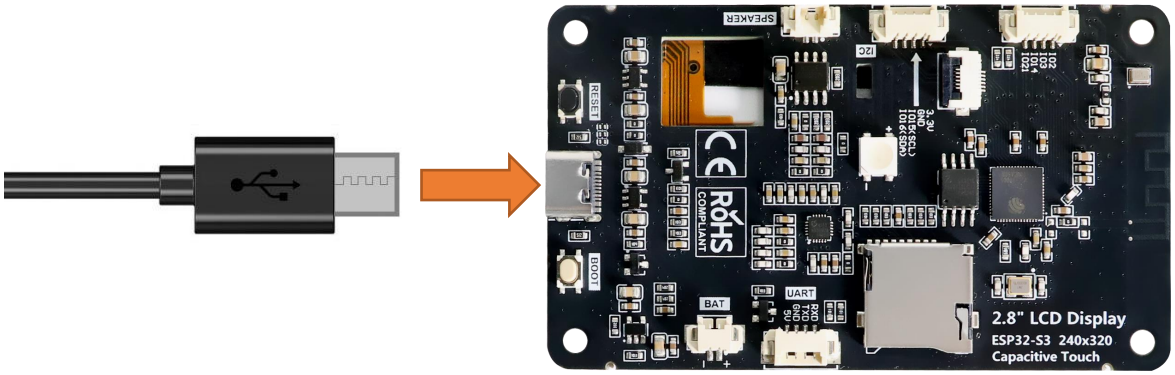
The onboard RGB LED can emit three basic colors of red, green and blue, and supports 256-level brightness adjustment, which means that it can emit 2^24=16,777,216 different colors.

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



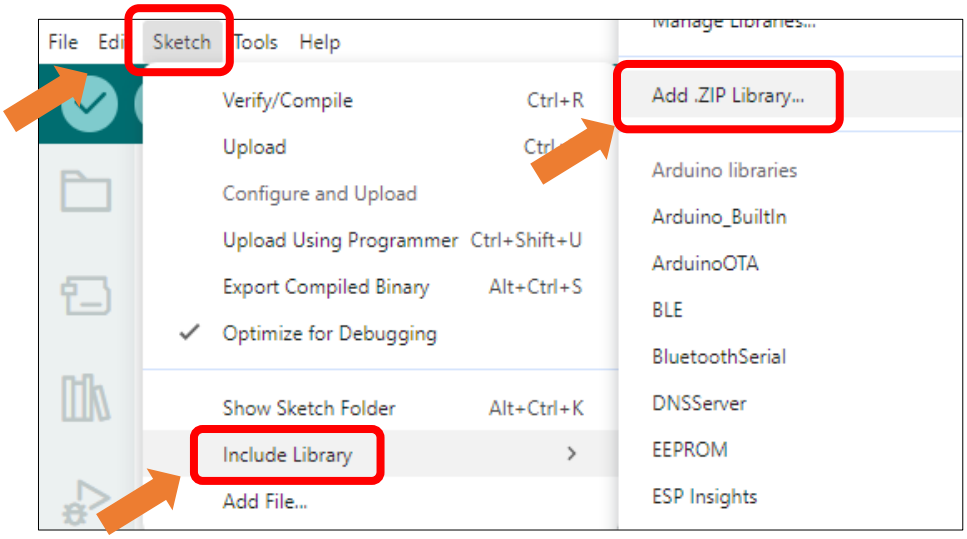
Sketch

Open "**Sketch_02.1_LedPixel**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_02.1_LedPixel.ino**".

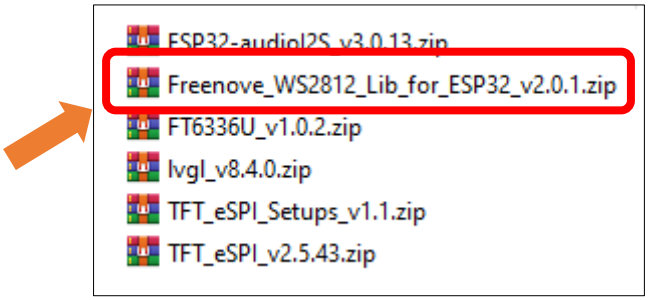
Install the needed libraries.

If you have not installed all libaries as per the previous section, please follow the following steps to install the ws2812 library.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **Freenove_WS2812_Lib_for_ESP32_v2.0.1.zip**



Sketch_02.1_LedPixel

The following is the program code:

```
#include "Freenove_WS2812_Lib_for_ESP32.h"

#define LEDS_COUNT 1
#define LEDS_PIN   42
#define CHANNEL    0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

uint8_t m_color[5][3] = { { 255, 0, 0 }, { 0, 255, 0 }, { 0, 0, 255 }, { 255, 255, 255 }, { 0,
0, 0 } };
int delayval = 100;

void setup() {
  strip.begin();
  strip.setBrightness(10);
}
void loop() {
  for (int j = 0; j < 5; j++) {
    for (int i = 0; i < LEDS_COUNT; i++) {
      strip.setLedColorData(i, m_color[j][0], m_color[j][1], m_color[j][2]);
      strip.show();
      delay(delayval);
    }
    delay(500);
  }
}
```

Code Explanation

Define the pins for the RGB LED.

```
#define LEDS_COUNT 1
#define LEDS_PIN   42
#define CHANNEL    0
```

Initialize the LED, set the brightness to 10

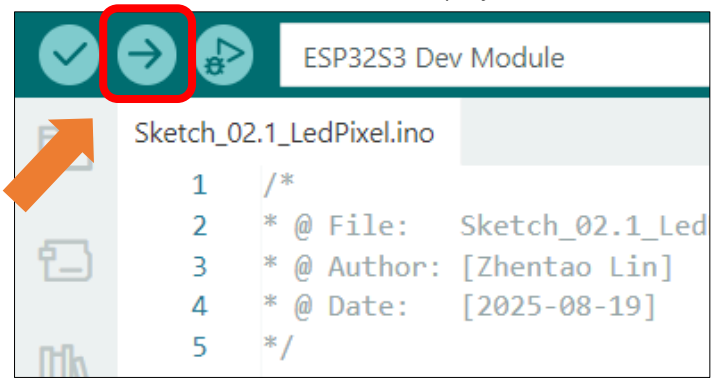
```
strip.begin();
strip.setBrightness(10);
```

Cycle through five different colors.

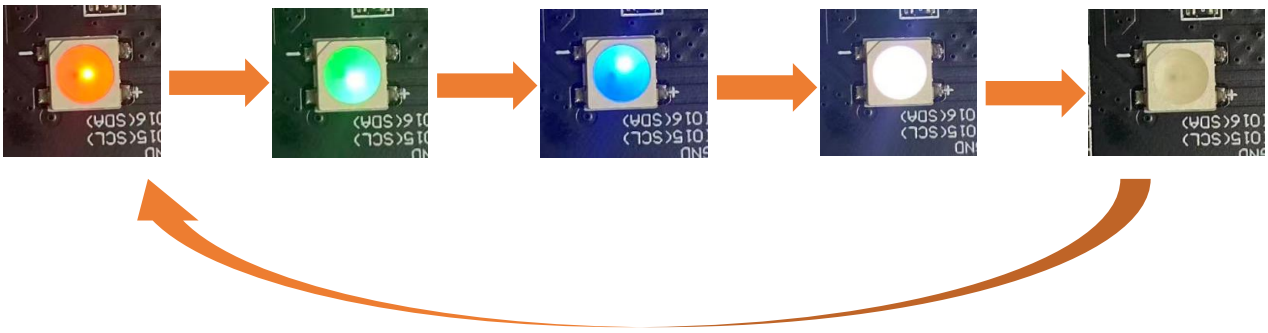
```
for (int j = 0; j < 5; j++) {
  for (int i = 0; i < LEDS_COUNT; i++) {
    strip.setLedColorData(i, m_color[j][0], m_color[j][1], m_color[j][2]);
    strip.show();
    delay(delayval);
  }
  delay(500);
```

```
31 }
```

Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.



The RGB LEDs will cycle through Red -> Green -> Blue –> White -> Off every 500ms after code upload.



Reference

Freenove_ESP32_WS2812(u16 **n** = 8, u8 **pin_gpio** = 2, u8 **chn** = 0, LED_TYPE **t** = TYPE_GRB)

A constructor to create a ws2812 object.

Before each use of the constructor, please add "**#include "Freenove_WS2812_Lib_for_ESP32.h**"

Parameters

n: The number of led.

pin_gpio: The pin connected to the LED.

Chn: RMT channel, which has eight channels, 0-7, and uses channel 0 by default. This means that you can use eight ws2812 modules for the display at the same time, and these modules do not interfere with each other.

t: Types of LED.

TYPE_RGB: The sequence of ws2812 module loading color is red, green and blue.

TYPE_RBG: The sequence of ws2812 module loading color is red, blue and green.

TYPE_GRB: The sequence of ws2812 module loading color is green, red and blue.

TYPE_GBR: The sequence of ws2812 module loading color is green, blue and red.

TYPE_BRG: The sequence of ws2812 module loading color is blue, red and green.

TYPE_BGR: The sequence of ws2812 module loading color is blue, green and red.

Project 2.2 Rainbow

In the previous project, we have mastered the use of LEDPixel. This project will realize a slightly complicated rainbow light. The component list and the circuit are exactly the same as the project fashionable light.

Sketch

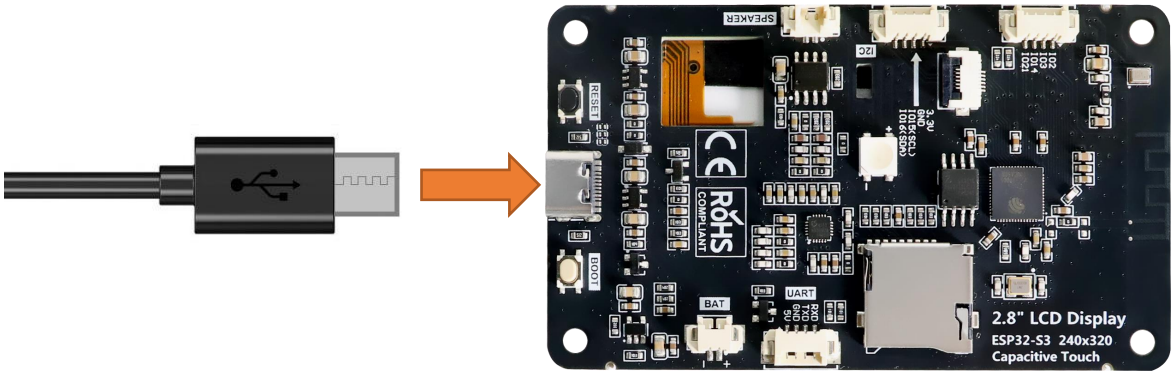
Continue to use the following color model to equalize the color distribution of the LED and gradually change.

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Open "**Sketch_02.2_Rainbow**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_02.2_Rainbow**".

Sketch_02.2_Rainbow

The following is the program code:

```
#include "Freenove_WS2812_Lib_for_ESP32.h"

#define LEDS_COUNT  1
#define LEDS_PIN    42
#define CHANNEL     0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

void setup() {
  strip.begin();
  strip.setBrightness(20);
}

void loop() {
  for (int j = 0; j < 255; j += 2) {
    for (int i = 0; i < LEDS_COUNT; i++) {
      strip.setLedColorData(i, strip.Wheel((i * 256 / LEDS_COUNT + j) & 255));
    }
    strip.show();
    delay(10);
  }
}
```

Code Explanation

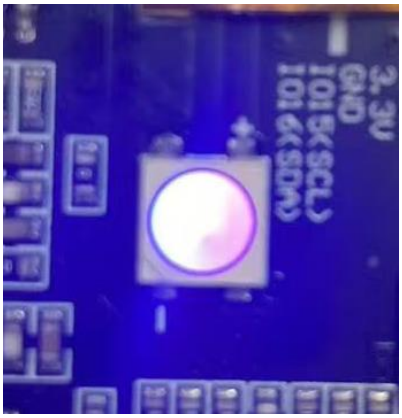
In the loop(), two "for" loops are used, the internal "for" loop(for-j) is used to set the color of each LED, and the external "for" loop(for-i) is used to change the color, in which the self-increment value in i+=2 can be changed to change the color step distance. Changing the delay parameter changes the speed of the color change. `strip.Wheel((i * 256 / LEDS_COUNT + j) & 255)` will take color from the color model at equal intervals starting from i.

```
for (int j = 0; j < 255; j += 2) {
    for (int i = 0; i < LEDS_COUNT; i++) {
      strip.setLedColorData(i, strip.Wheel((i * 256 / LEDS_COUNT + j) & 255));
    }
    strip.show();
    delay(10);
}
```

This code achieves the function of printing data on serial monitor. Click "**Upload**" to upload the code to

Freenove_ESP32_S3_Display.

After uploading the code, the LED lights will display a smooth rainbow gradient effect.



Reference

```
bool ledcAttachChannel(uint8_t pin, uint32_t freq, uint8_t resolution, uint8_t channel);
```

This function binds the specified PWM channel to a GPIO pin and configures the frequency and resolution of the PWM signal.

Parameters:

pin: GPIO pin number to bind

freq: PWM signal frequency in Hz

resolution: Bit depth for PWM duty cycle resolution (range: 1-16 bits).
For example, 8 represents 2^8 (0~255) levels.

channel: PWM channel number to assign (integer)

```
void ledcWrite(uint8_t channel, uint32_t duty);
```

This function sets the duty cycle for a specified PWM channel to control output signal intensity.

Parameters:

channel: PWM channel number

duty: Duty cycle value (range determined by resolution bit depth)

Chapter 3 Button

The Boot button on the Freenove ESP32-S3 Display can be configured as a regular input button after the program starts.

Project 3.1 Button RGB

In the previous chapter, we learned about RGB LEDs. In this section, we will further explore how to integrate RGB LEDs with buttons.

Related Knowledge

Button

In embedded systems, buttons are one of the most common human-machine input devices. When a button is not pressed, its opposing contacts are connected while adjacent contacts remain disconnected. Conversely, when the button is pressed, adjacent contacts connect while opposing contacts disconnect.

Usage of the Boot Button on the Freenove ESP32-S3 Display:

Manual Entry into Download Mode:

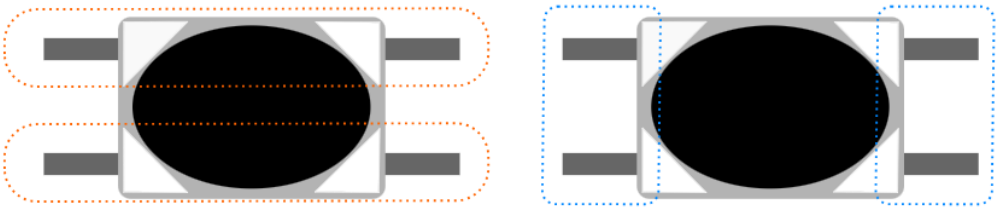
When the board is powered on or reset, pressing and holding the Boot button forces the board into download mode.

In this mode, users can upload programs to the board via the serial port.

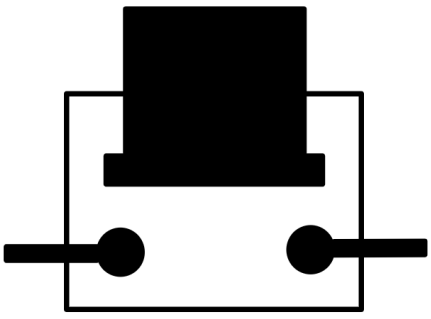
Use as a Regular Input Button:

If the Boot button is not pressed during power-on or reset, the board enters normal operation mode.

Once in operation mode, the Boot button can function as GPIO0, typically configured as a standard input button for user interaction.

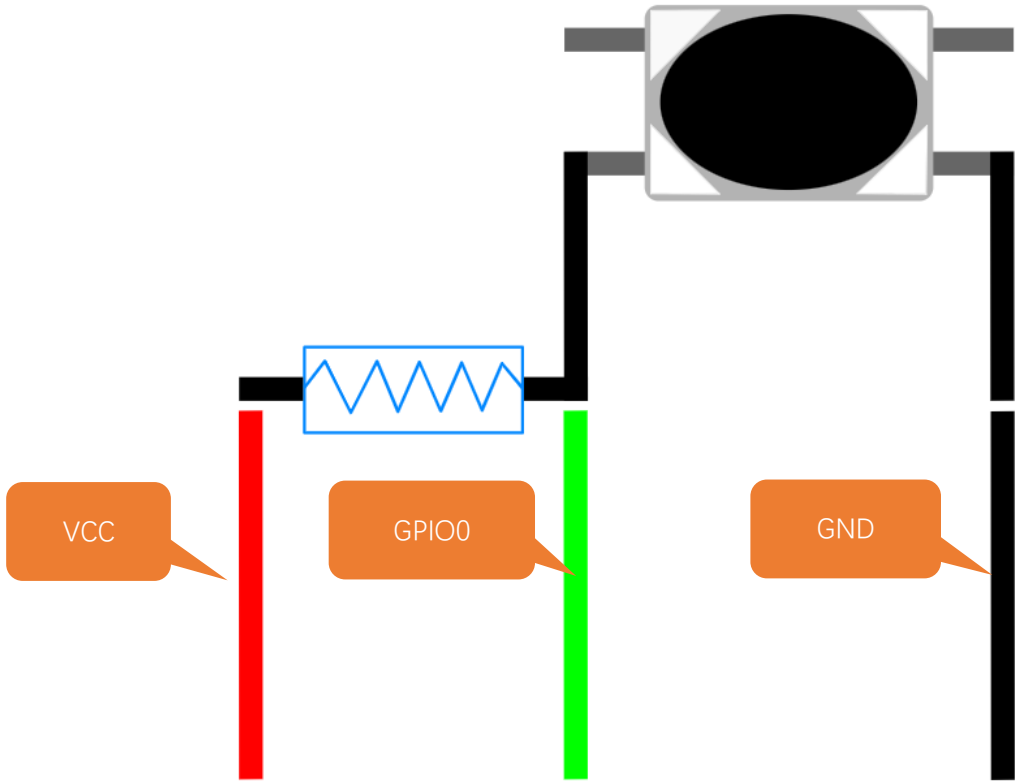


Before pressing After pressing



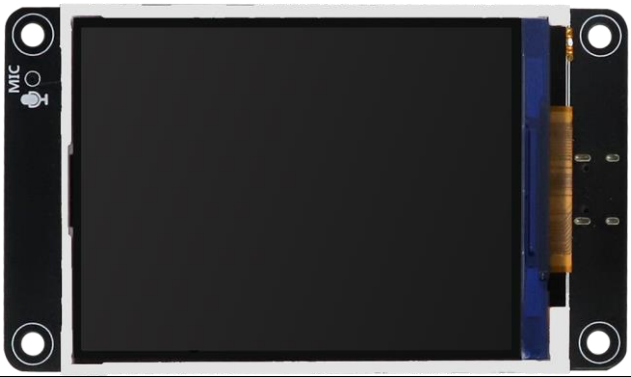
Pull-up Resistor

The primary function of a pull-up resistor is to ensure that the button maintains a **stable high-level state** when not pressed, preventing false triggering caused by floating pins or signal noise. The pull-up resistor is connected between the GPIO pin and VCC.



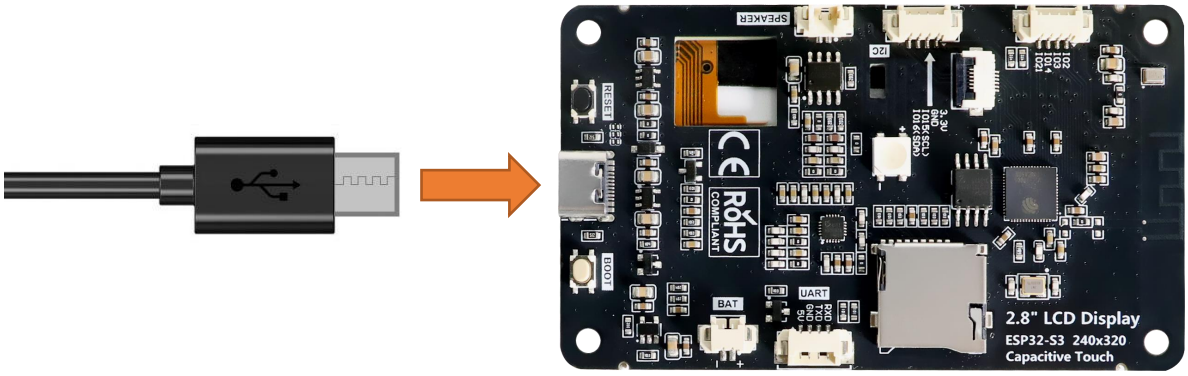
When the button is not pressed, Vcc is connected to GPIO0 through the resistor, resulting in a high-level signal at this pin.
When the button is pressed, GPIO0 is connected to GND, resulting in a low-level signal.
Therefore, by reading the GPIO0 pin state, we can determine whether the button is pressed or not.

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Open "**Sketch_03.1_Button_RGB**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_03.1_Button_RGB.ino**".

Sketch_03.1_Button_RGB

The following is the program code:

```
#include <Arduino.h>
#include "Freenove_WS2812_Lib_for_ESP32.h"

#define KEY_PIN 0
#define LEDS_COUNT 1
#define LEDS_PIN   42
#define CHANNEL    0

Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);

uint8_t m_color[5][3] = { { 255, 0, 0 }, { 0, 255, 0 }, { 0, 0, 255 }, { 255, 255, 255 }, { 0,
0, 0 } };

static unsigned int keyCount = 0;
static unsigned int lastKeyCount = 1;

void buttonInit(void) {
    pinMode(KEY_PIN, INPUT_PULLUP);
}

bool readButton(void) {
    return digitalRead(KEY_PIN);
}
```

```
24
25  void switchRGB(int value) {
26    int colorIndex = value % 4;
27    switch (colorIndex) {
28      case 1:
29        strip.setLedColorData(0, m_color[0][0], m_color[0][1], m_color[0][2]);
30        strip.show();
31        break;
32      case 2:
33        strip.setLedColorData(0, m_color[1][0], m_color[1][1], m_color[1][2]);
34        strip.show();
35        break;
36      case 3:
37        strip.setLedColorData(0, m_color[2][0], m_color[2][1], m_color[2][2]);
38        strip.show();
39        break;
40      default:
41        strip.setLedColorData(0, m_color[4][0], m_color[4][1], m_color[4][2]);
42        strip.show();
43        break;
44    }
45  }
46
47  void setup() {
48    strip.begin();
49    strip.setBrightness(10);
50    buttonInit();
51    Serial.println("ESP32-S3 initialization completed!");
52  }
53
54  void loop() {
55    if (readButton() == 0) {
56      delay(20);
57      if (readButton() == 0) {
58        keyCount++;
59        while (!readButton());
60      }
61    }
62    if (keyCount != lastKeyCount) {
63      switchRGB(keyCount);
64      lastKeyCount = keyCount;
65    }
66  }
```

Code Explanation

Define the pins for the button and the RGB LED.

```
10  #define KEY_PIN 0
11  #define LEDS_COUNT 1
12  #define LEDS_PIN   42
13  #define CHANNEL    0
```

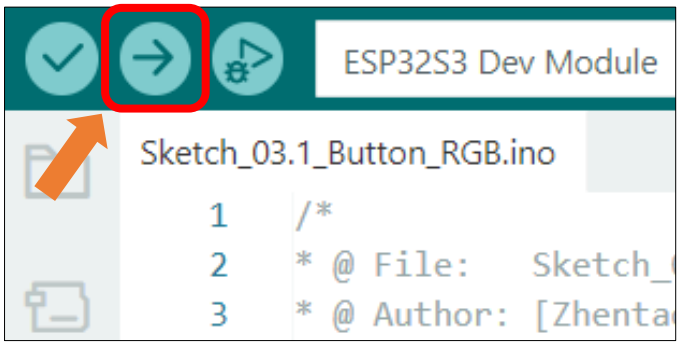
Initialize the RGB LED and the button.

```
54  strip.begin();
55  strip.setBrightness(10);
56  buttonInit()
```

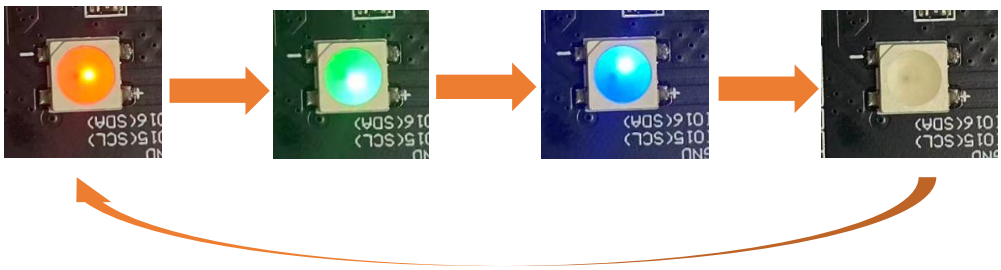
Control the color of the RGB LED through the button.

```
61   if (readButton() == 0) {
62     delay(20);
63     if (readButton() == 0) {
64       keyCount++;
65       while (!readButton());
66     }
67   }
68   if (keyCount != lastKeyCount) {
69     switchRGB(keyCount);
70     lastKeyCount = keyCount;
71   }
```

Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.



The RGB LEDs will cycle through Red -> Green -> Blue -> OFF every 500ms after code upload.



Chapter 4 Button Interrupt

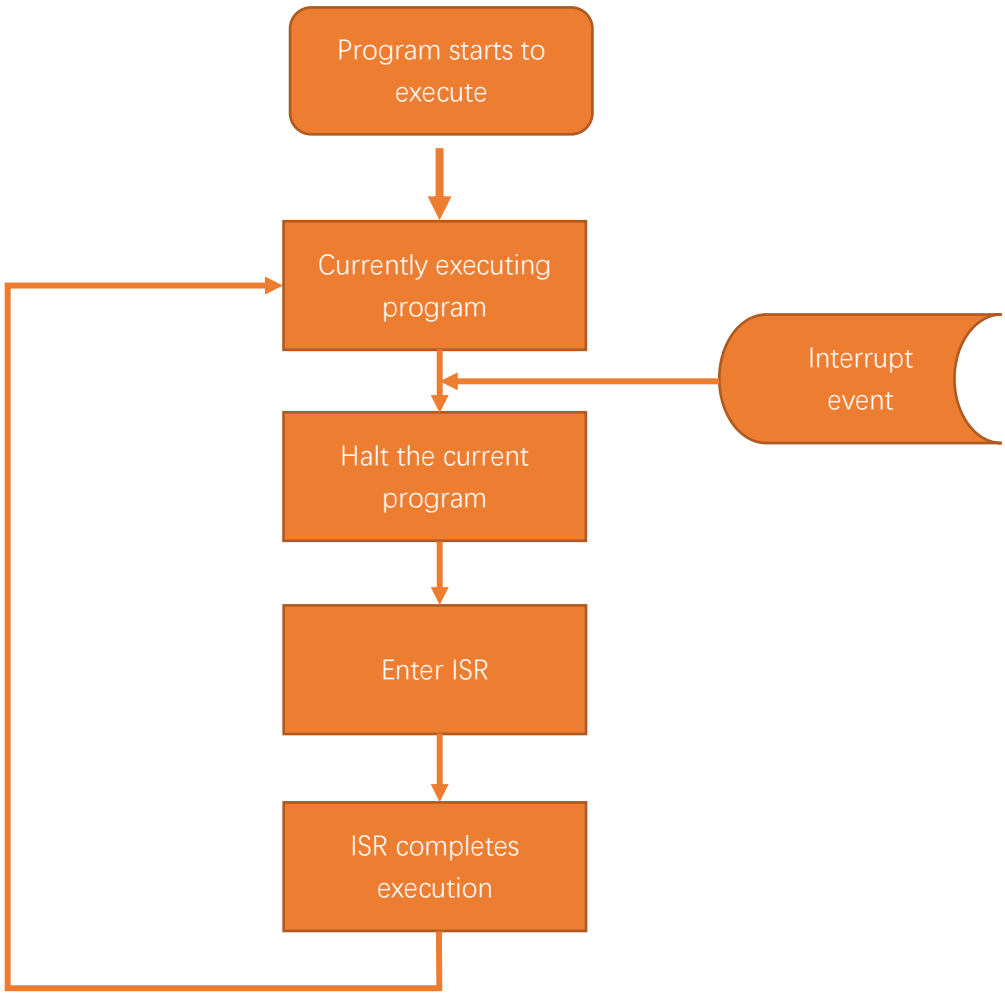
In this chapter will learn to use interrupt to detect the button state.

Project 4.1 Button Interrupt UART

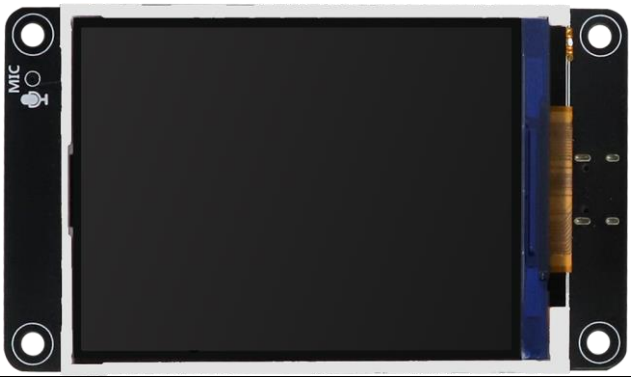
Related Knowledge

How Interrupts Work

When an interrupt event is triggered, the Freenove ESP32-S3 Display receives an interrupt signal. At this moment, the Freenove ESP32-S3 Display pauses the currently executing program and instead executes the Interrupt Service Routine (ISR). The ISR contains the code that needs to be processed after the interrupt event occurs. Once the ISR execution is complete, the Freenove ESP32-S3 Display will return to the state it was in before the interrupt occurred and continue executing the paused program, as shown in the diagram below.

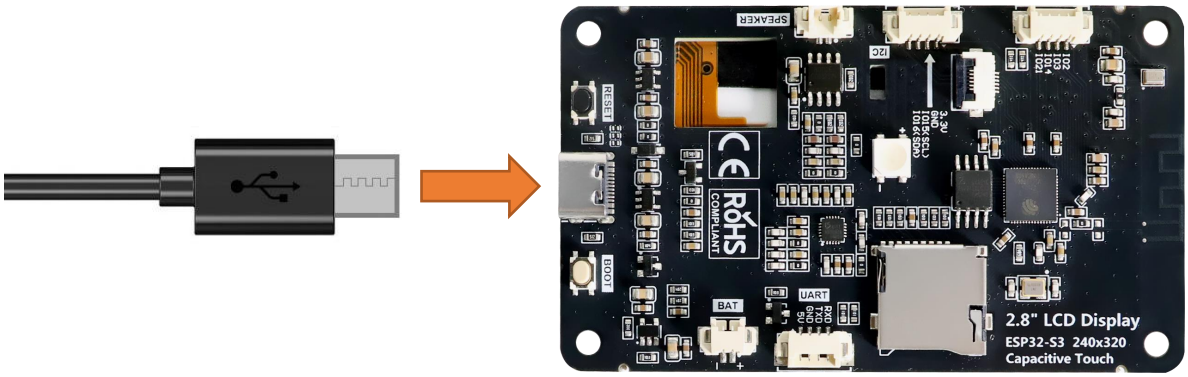


Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Open "**Sketch_04.1_Button_Interrupt_UART**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_04.1_Button_Interrupt_UART.ino**".

Sketch_04.1_Button_Interrupt_UART

The following is the program code:

```
#include <Arduino.h>

#define KEY_PIN 0
volatile int interruptCounter = 0;
int lastValue = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void buttonInit(void) {
    pinMode(KEY_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(KEY_PIN), handleInterrupt, FALLING);
}
```

```
12
13  void delayDebounce(unsigned int delayTime) {
14    unsigned int i = delayTime;
15    while (--i > 0) {
16      if (digitalRead(KEY_PIN) == HIGH) {
17        return;
18      }
19    }
20  }
21
22  void handleInterrupt(void) {
23    portENTER_CRITICAL_ISR(&mux);
24    interruptCounter++;
25    delayDebounce(20);
26    portEXIT_CRITICAL_ISR(&mux);
27  }
28
29  void setup() {
30    Serial.begin(115200);
31    buttonInit();
32    Serial.println("ESP32-S3 initialization completed!");
33  }
34
35  void loop() {
36    if (interruptCounter != lastValue) {
37      Serial.printf("interruptCounter: %d\r\n", interruptCounter);
38      lastValue = interruptCounter;
39    }
40  }
```

Code Explanation

Define the pin of the button.

```
9   #define KEY_PIN 0
```

Configure the button pin hardware interrupt to trigger on the falling edge signal.

```
16  attachInterrupt(digitalPinToInterrupt(KEY_PIN), handleInterrupt, FALLING);
```

The interrupt processing function, counting how many times the button is pressed.

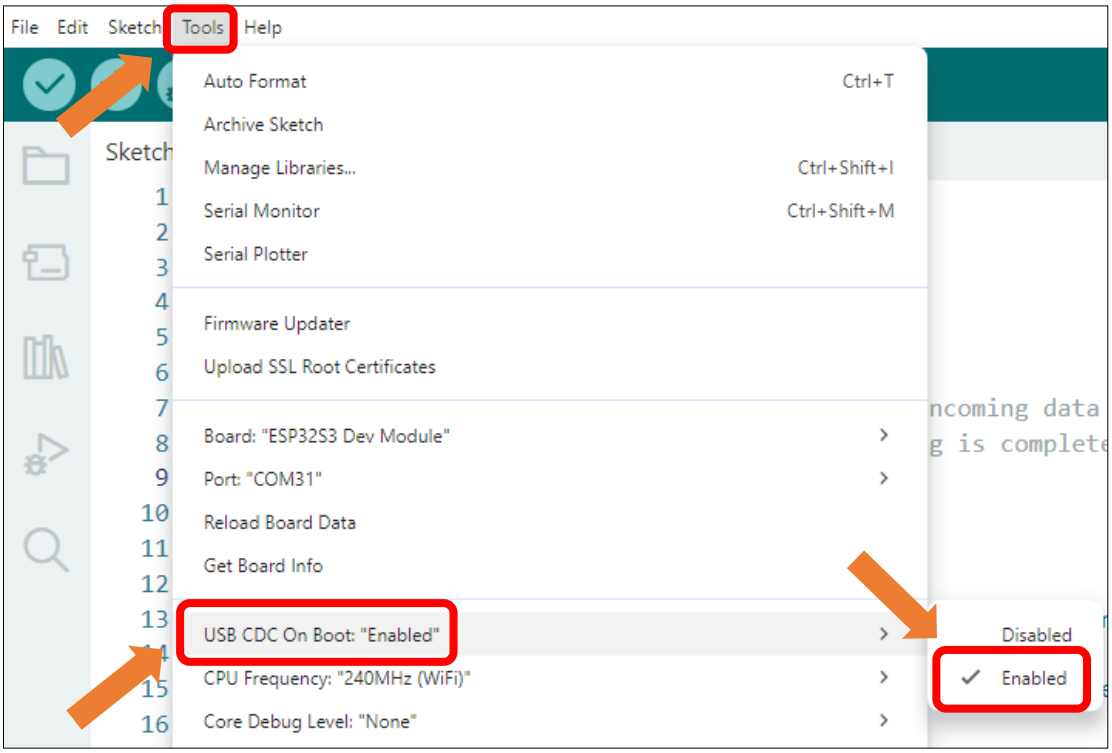
```
28  void handleInterrupt(void) {
29    portENTER_CRITICAL_ISR(&mux);
30    interruptCounter++;
31    delayDebounce(20);
32    portEXIT_CRITICAL_ISR(&mux);
33  }
```

When the number of button presses changes, print new data in the serial monitor.

```
41  void loop() {
42    if (interruptCounter != lastValue) {
```

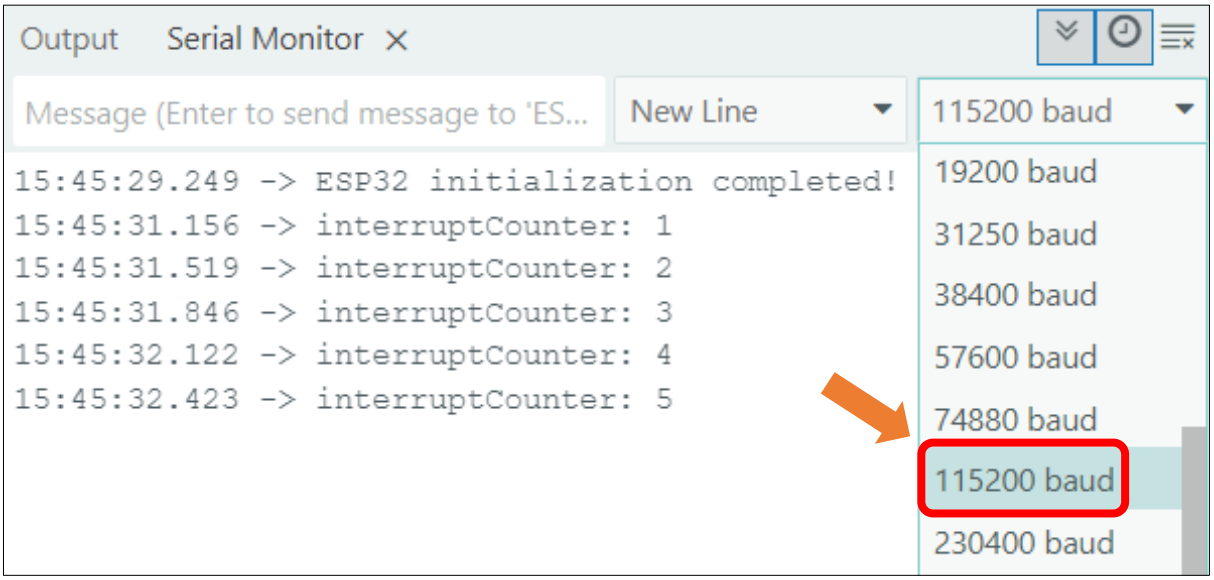
```
43        Serial.printf("interruptCounter: %d\r\n", interruptCounter);
44        lastValue = interruptCounter;
45      }
46  }
```

Enable the "USB CDC On Boot" feature.



Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.

When the code finishes uploading, open the serial monitor and set the baud rate to 115200. The serial monitor will print the number of button presses.



Reference

```
void attachInterrupt(uint8_t pin, void (*)(void), int mode);
```

external interrupts

Parameters:

pin: The digital pin number for the interrupt

void: The name of the interrupt function

mode:

- LOW (trigger on low level)
- CHANGE (trigger on change)
- RISING (trigger on rising edge)
- FALLING (trigger on falling edge)

Chapter 5 Battery Voltage

Project 5.1 Battery Voltage

Related Knowledge

ADC

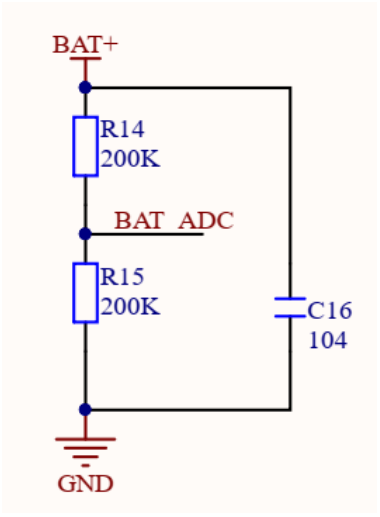
ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC on Freenove ESP32-S3 Display is 12 bits, which means the resolution is 2^12=4095, and it represents a range (at 3.3V) will be divided equally to 4095 parts. The rage of analog values corresponds to ADC values. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.

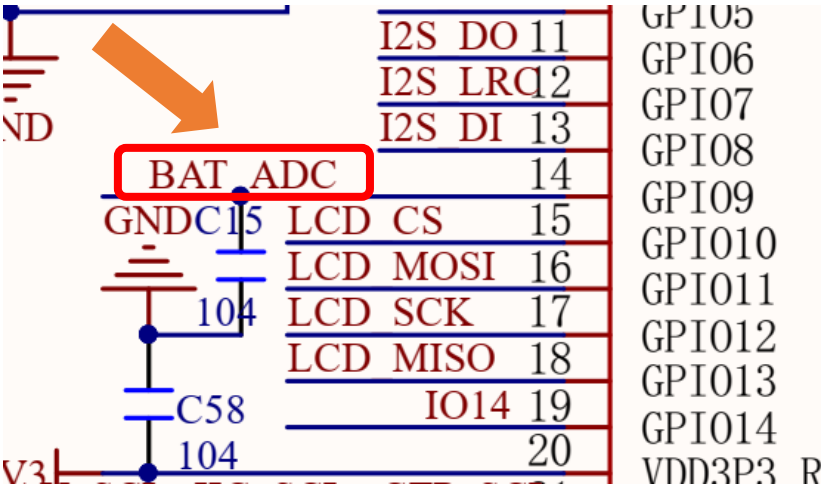
Subsection 1: the analog in rang of 0V---3.3/4095 V corresponds to digital 0;
Subsection 2: the analog in rang of 3.3/4095 V---2*3.3 /4095V corresponds to digital 1;
The following analog will be divided accordingly.
The conversion formula is as follows:

$$ADC\ Value = \frac{\text{Analog Voltage}}{3.3} * 4095$$

Battery Voltage

From the schematic diagram, we can know that Freenove ESP32-S3 Display reads the battery voltage through the ADC pin (GPIO9).





The ADC pin on this board has a voltage sampling range of 0~3.3V. However, when powered by a 3.7V lithium battery, the output voltage in a fully charged state can reach 4.2V, exceeding the ADC's rated input range.

To ensure accurate voltage measurement and ADC pin protection, the circuit employs a resistive voltage divider that scales the battery voltage down by a factor of 0.5. At a full charge (4.2V), the divided voltage at the ADC input is 2.1V, well within the safe operating range. This design enables reliable battery monitoring while preventing overvoltage damage to the ADC.

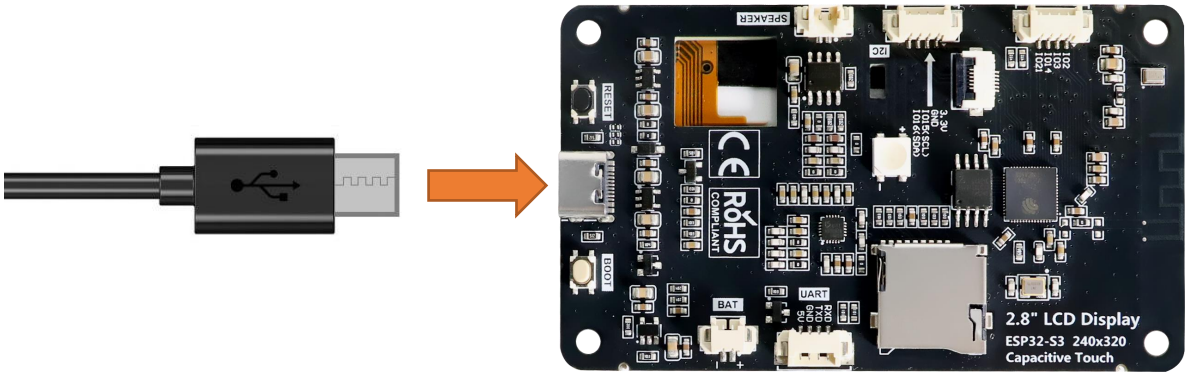
Please note that this kit does not include a lithium battery, please buy one yourself. For more information about battery, please refer to Battery.

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32-S3 Display to the computer with USB cable.



Sketch

Next, we download the code to Freenove ESP32-S3 Display to test Serial. Open "**Sketch_05.1_Battery_Voltage.ino**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_05.1_Battery_Voltage.ino**".

Sketch_05.1_Battery_Voltage

The following is the program code:

```
1  #define BAT_ADC_PIN  9
2
3  void setup(){
4     Serial.begin(115200);
5     pinMode(BAT_ADC_PIN, INPUT);
6  }
7
8  void loop() {
9     int adcValue= analogRead(BAT_ADC_PIN);
10    float batteryVoltage = analogReadMilliVolts(BAT_ADC_PIN) * 2.0 / 1000;
11    Serial.printf("Reading on Pin(%d), adcValue=%d, batteryVoltage2=%fV\n", BAT_ADC_PIN,
12 adcValue, batteryVoltage);
13    delay(300);
14 }
```

Code Explanation

Define the pins for the button and RGB LED.

```
7  #define BAT_ADC_PIN  9
```

Set the baud rate to 115200.

```
10 Serial.begin(115200);
```

Set the battery voltage detecting pin to input mode.

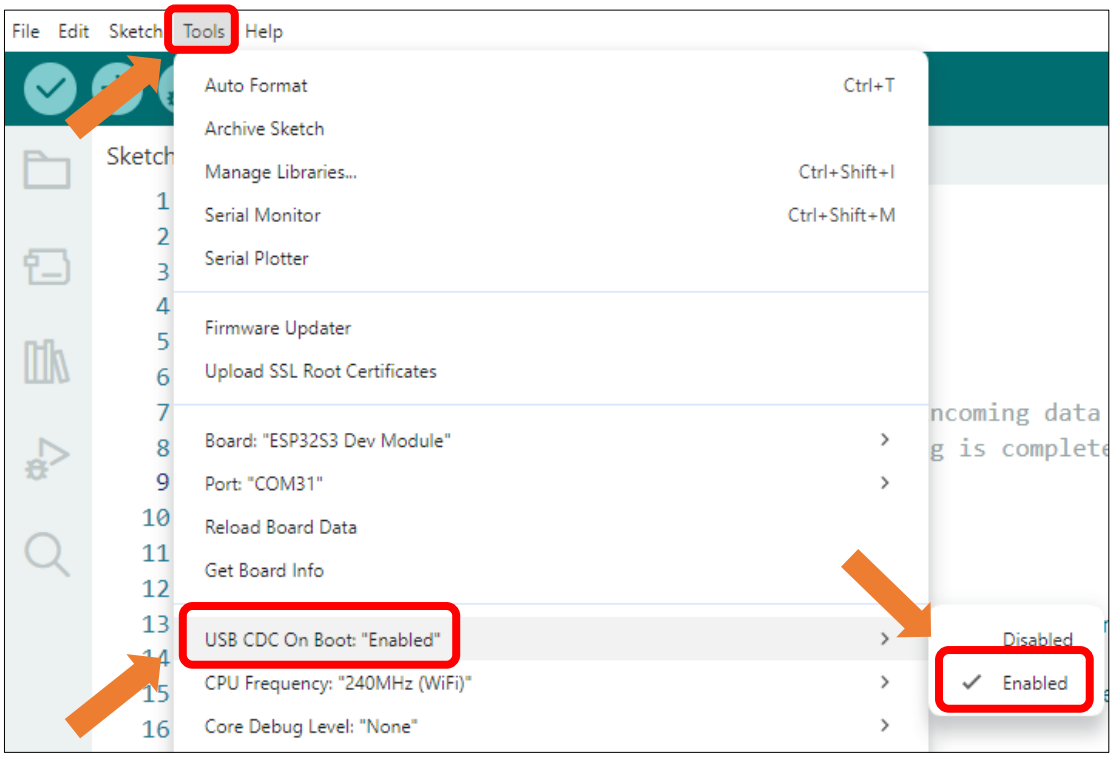
```
11 pinMode(BAT_ADC_PIN, INPUT);
```

Measure the battery voltage every 300ms and outputs the reading to the serial monitor.

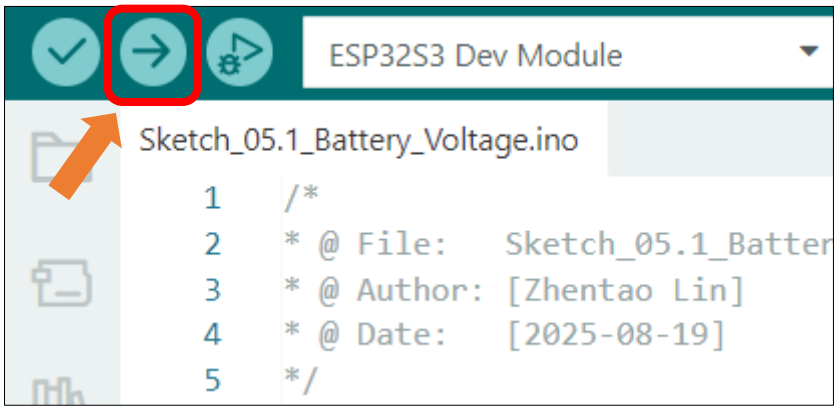
```
14 void loop() {
```

```
15      int adcValue= analogRead(BAT_ADC_PIN);
16      float batteryVoltage = analogReadMilliVolts(BAT_ADC_PIN) * 2.0 / 1000;
17      Serial.printf("Reading on Pin(%d), adcValue=%d, batteryVoltage2=%fV\n", BAT_ADC_PIN,
18  adcValue, batteryVoltage);
19      delay(300);
20  }
```

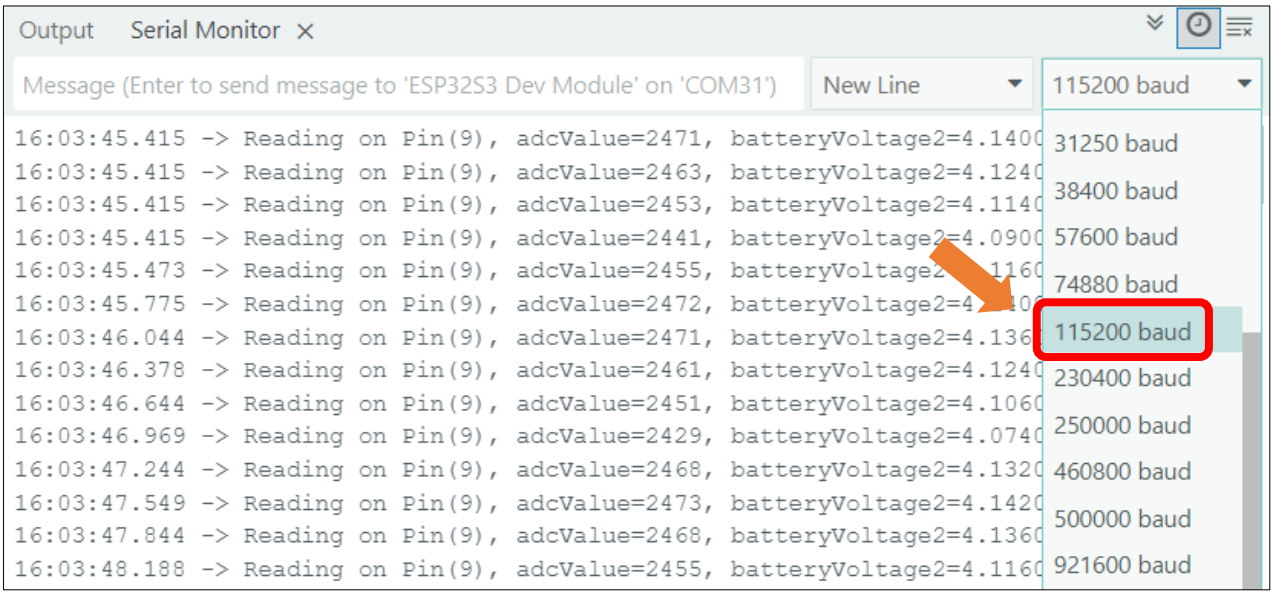
Enable the "USB CDC On Boot" feature.



Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.



Once the program starts, open the serial monitor to view the battery voltage readings, updated every 300ms.



Reference

uint32_t analogReadMilliVolts(uint8_t pin)

This function reads the voltage directly from the analog pin and returns the value in millivolts (mV), with a maximum return value of 3300mV (3.3V)

Chapter 6 SD Card

Project 6.1 SD Test

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Please note that this kit does not include SD card and card reader, please buy them by yourself.

Component Knowledge

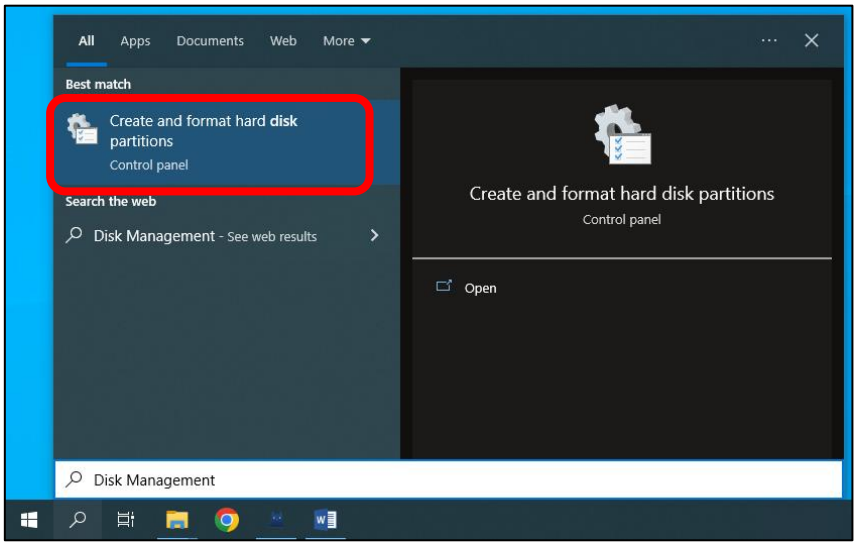
Format SD card

To initiate the project, you'll first need to prepare a blank SD card and a compatible card reader. The initial setup involves assigning a drive letter to the SD card and formatting it properly. Below you'll find system-specific instructions to complete these preparatory steps. Please follow the guide corresponding to your operating system.

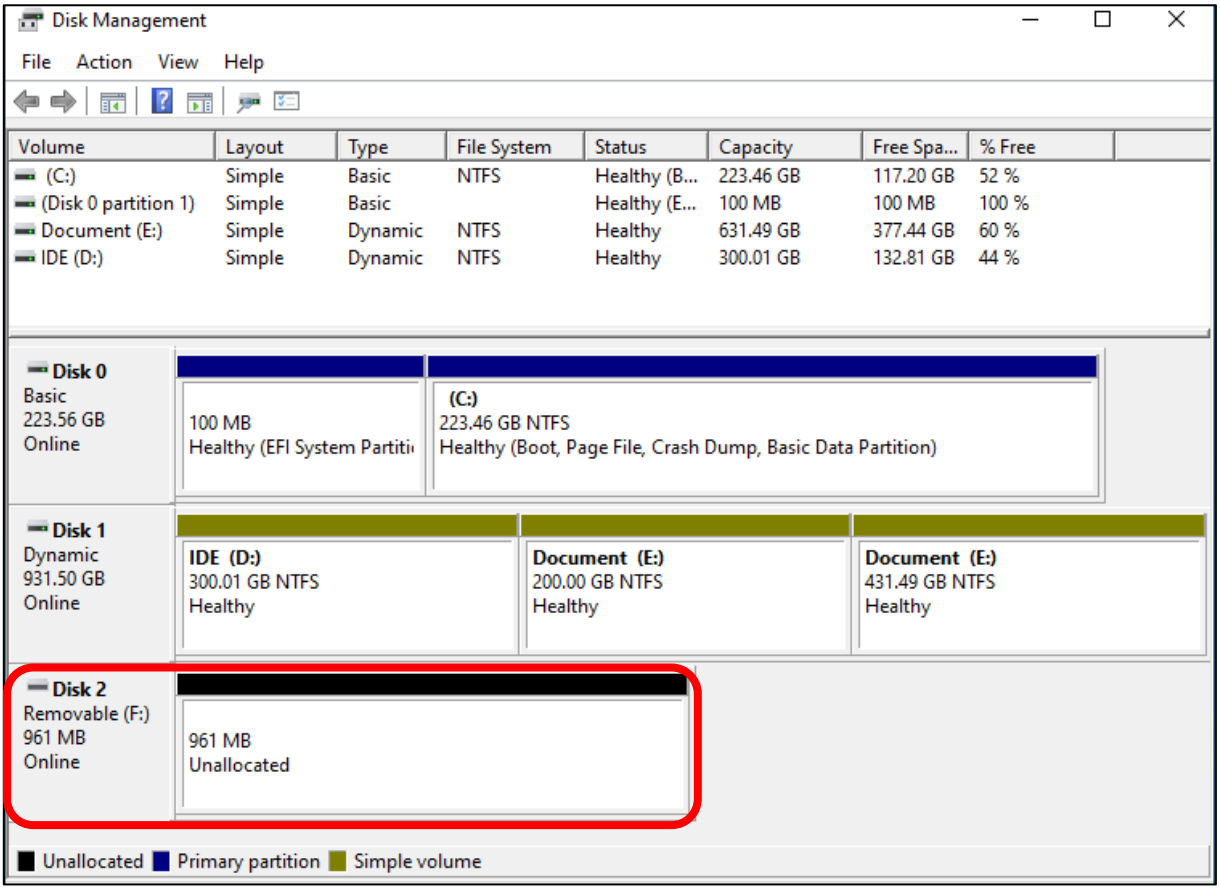
Windows

Insert the SD card into the card reader, and then insert the card reader into the computer.

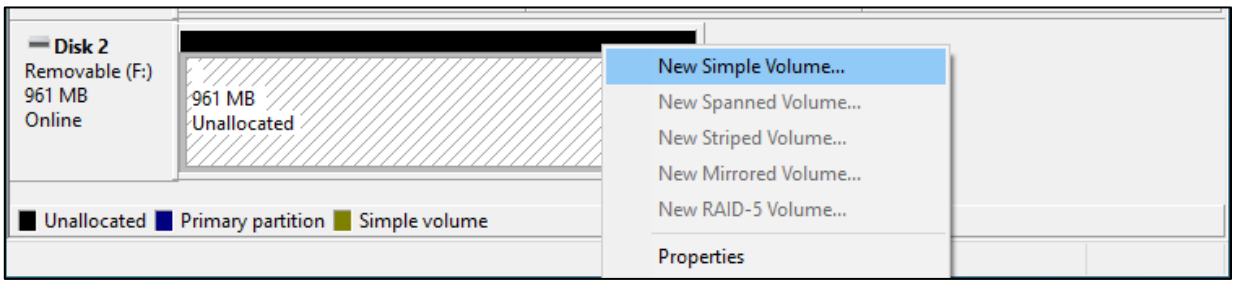
In the Windows search box, enter "Disk Management" and select "Create and format hard disk partitions".



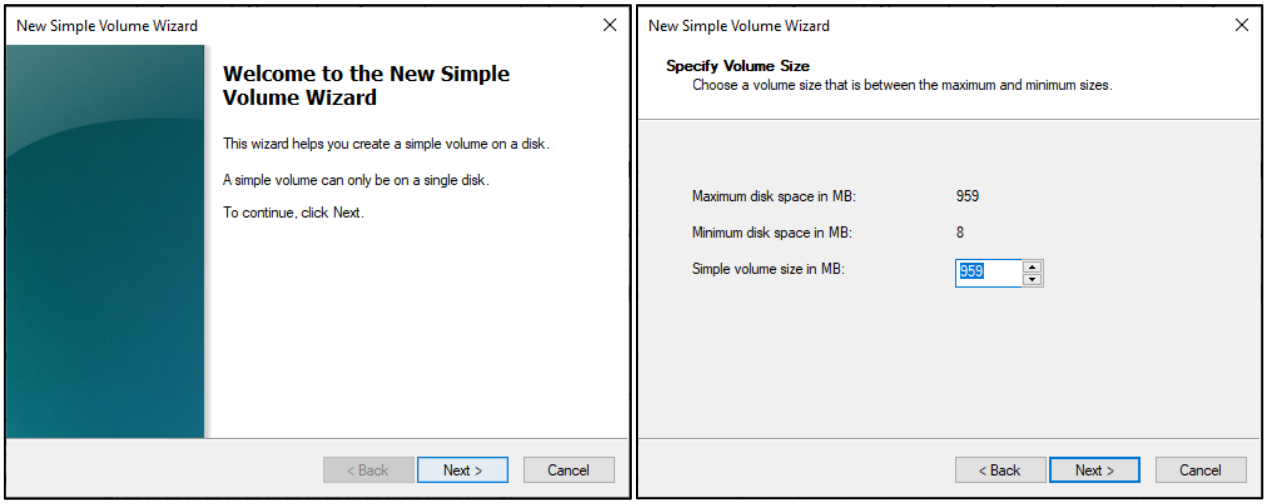
In the newly opened window, locate an unallocated volume with a size close to 1GB.



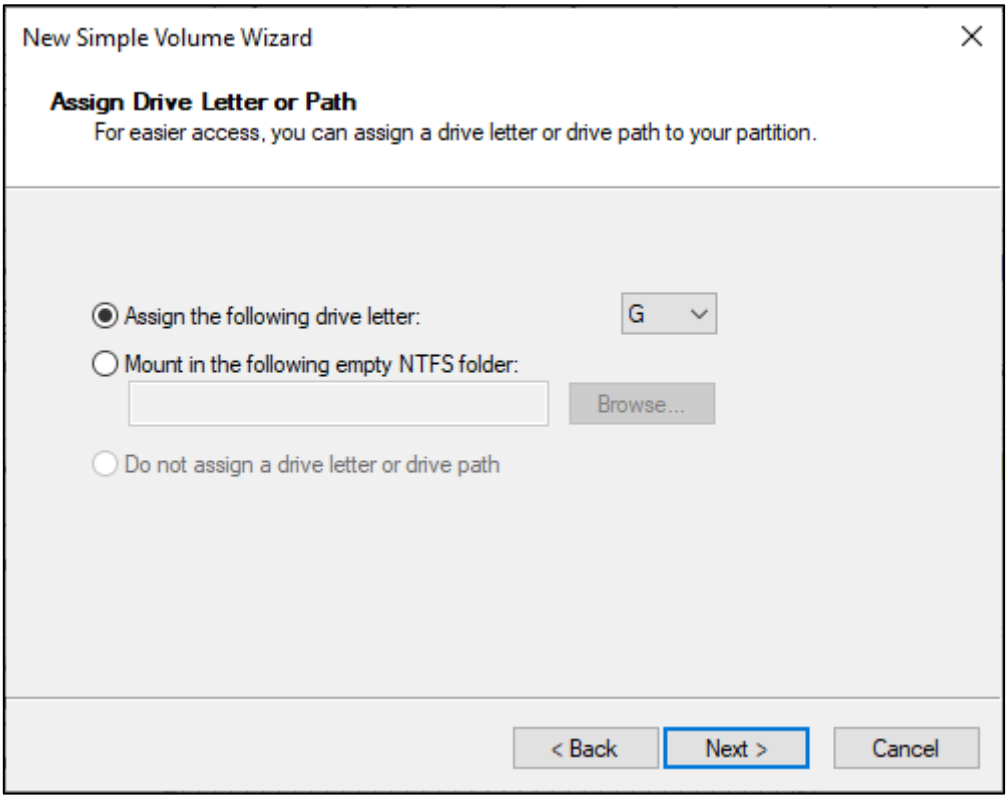
Click to select the volume, right-click and select "New Simple Volume".



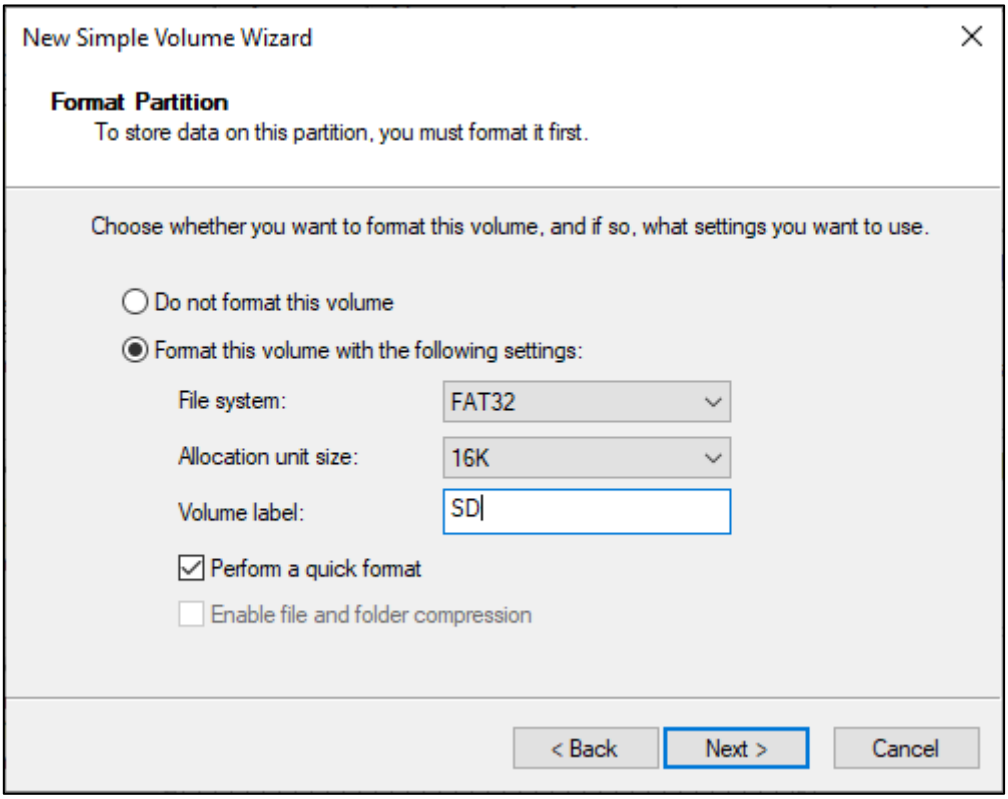
Click Next.



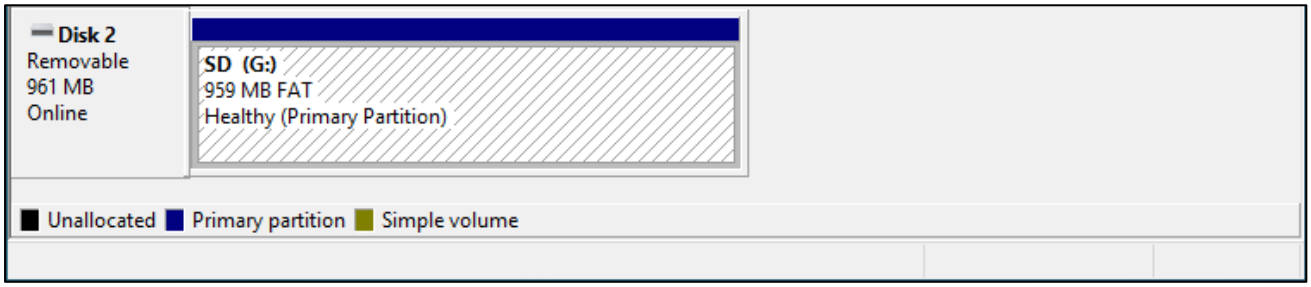
On the right-hand side, you can select a preferred drive letter for the SD card or simply proceed with the default setting by clicking on the "Next" button.



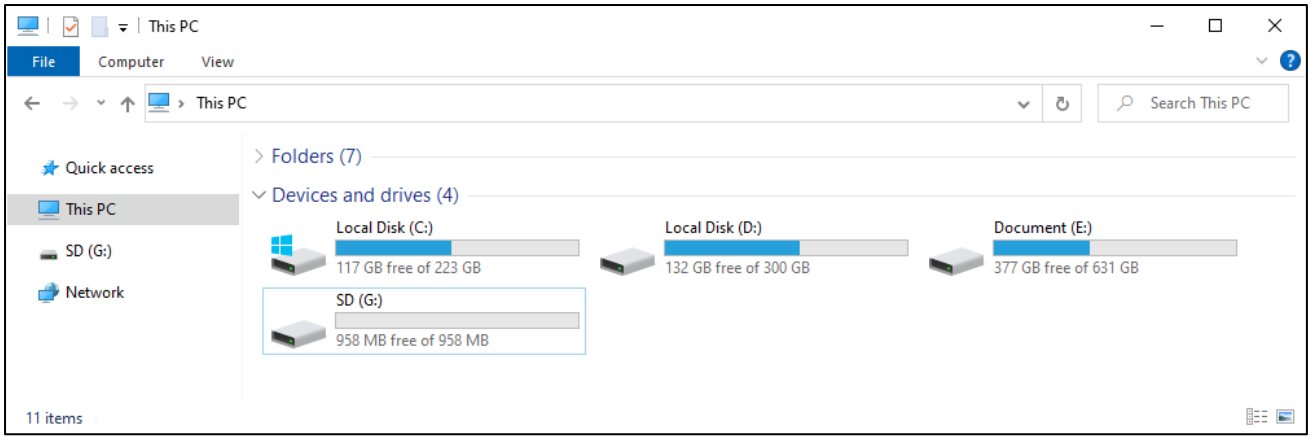
When formatting the SD card, select the file system as FAT (or FAT32) and set the allocation unit size to 16K. You can set the volume label to any name of your choice. Once you have made these selections, click on the "Next" button to proceed.



Click Finish. Wait for the SD card initialization to complete.

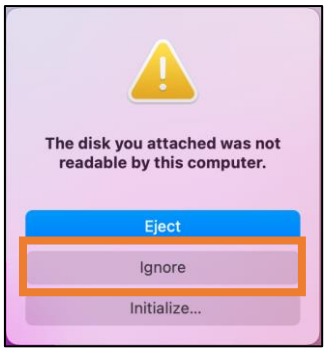


After completing the formatting process, you should be able to see the SD card in the "This PC" section of your computer.

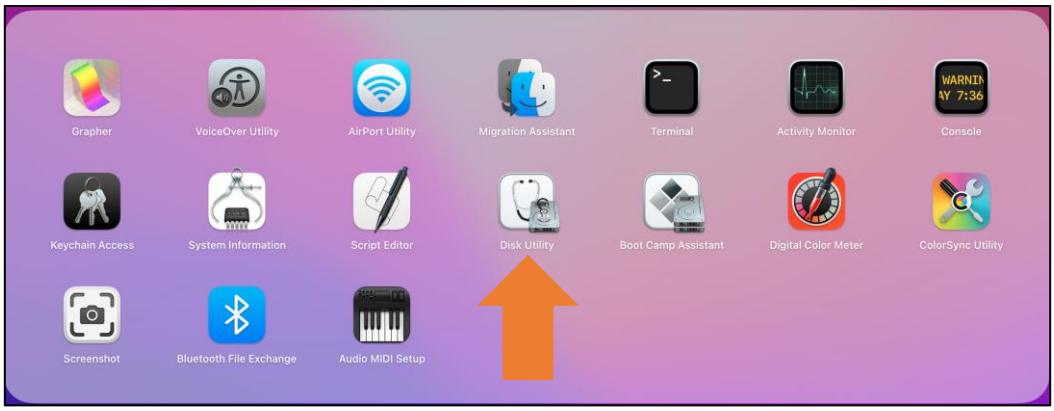


MAC

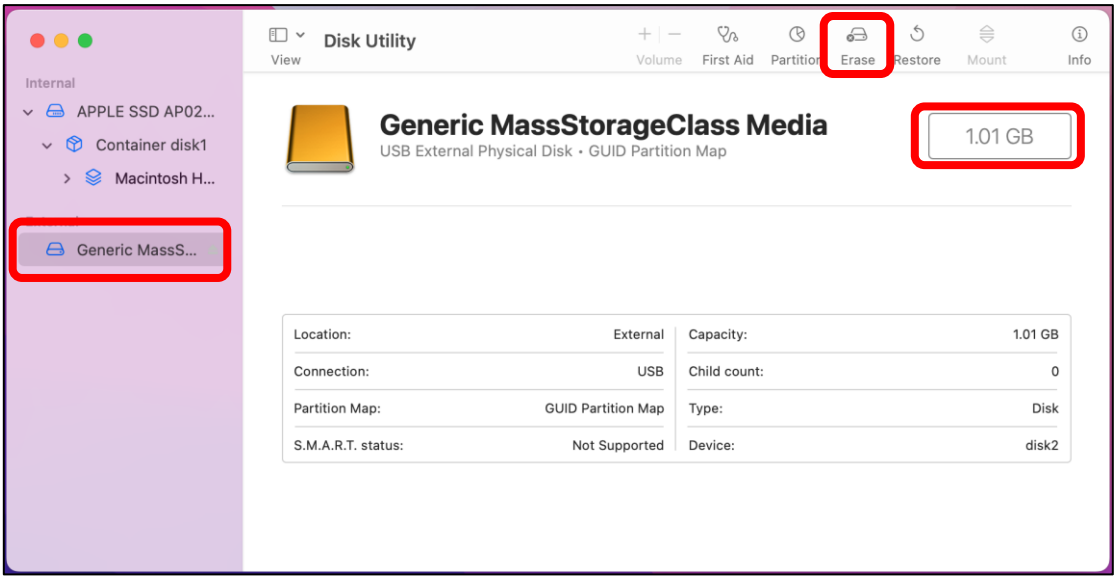
Insert the SD card into the card reader and then insert the card reader into your computer. Some computers may display a prompt with the following message. In this case, please click on the "Ignore" option to proceed.



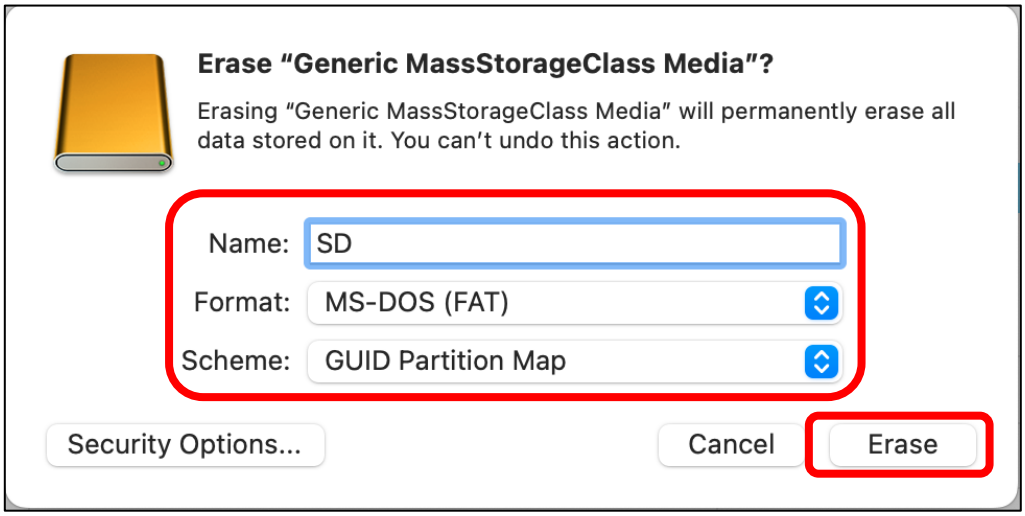
Find "Disk Utility" in the MAC system and click to open it.



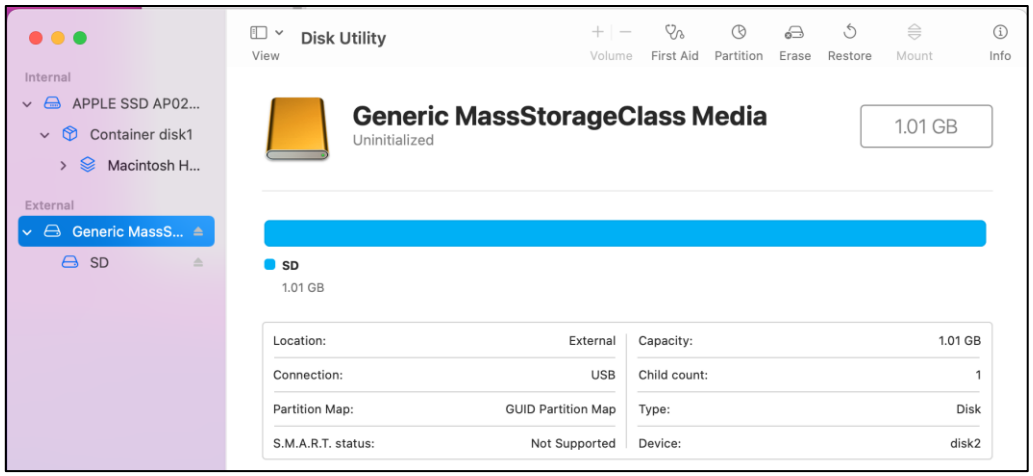
Select "Generic MassStorageClass Media", note that its size is about 1G. It is important to select the correct item to avoid accidentally erasing other device. Once you have verified that you have selected the correct device, click on the "Erase" button to proceed with erasing the SD card.



Select the configuration as shown in the figure below, and then click "Erase".



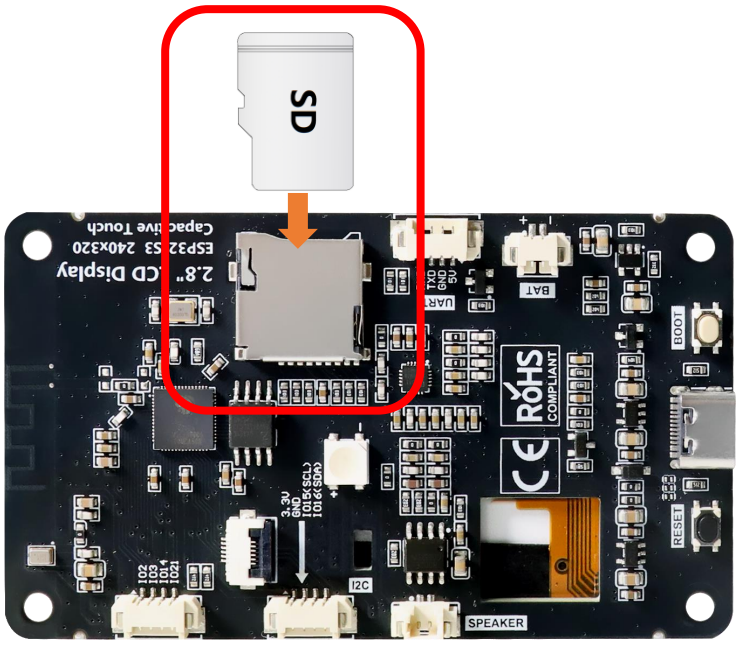
Please wait for the formatting process to complete. Once it is finished, the interface should resemble the image below. You should now be able to see a new disk named "SD" on your desktop.



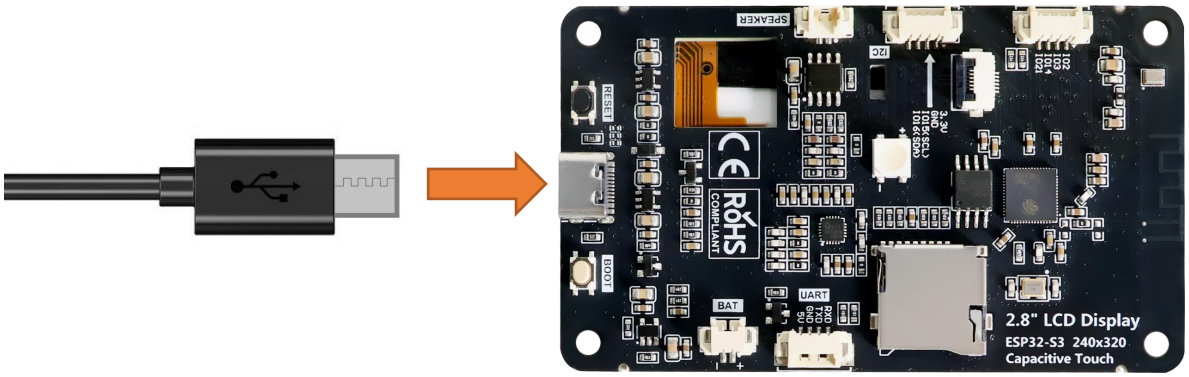
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect Freenove ESP32-S3 to the computer using the USB cable.



If you have any concerns, please feel free to contact us via support@freenove.com

Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test Serial. Open "**Sketch_06.1_SD_Test.ino**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_06.1_SD_Test.ino**".

Sketch_06.1_SD_Test

The following is the program code:

```
#include "driver_sdmmc.h"

#define SD_MMC_CMD 40  // Please do not modify it.
#define SD_MMC_CLK 38  // Please do not modify it.
#define SD_MMC_D0  39  // Please do not modify it.
#define SD_MMC_D1  41  // Please do not modify it.
#define SD_MMC_D2  48  // Please do not modify it.
#define SD_MMC_D3  47  // Please do not modify it.

void setup() {
    // Initialize serial communication at 115200 bits per second
    Serial.begin(115200);
    delay(3000);

    // Initialize the SDMMC interface with the specified pins
    sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);

    // List files in the root directory and print them
    std::list<File_Entry> fileList = list_dir("/", 0);
    print_file_list(fileList);

    // Create a new directory and list files again
    create_dir("/mydir");
    fileList = list_dir("/", 0);
    print_file_list(fileList);

    // Remove the directory and list files again
    remove_dir("/mydir");
    fileList = list_dir("/", 2);
    print_file_list(fileList);

    // Write text to a new file
    const char* text = "Hello ";
    write_file("/foo.txt", (const uint8_t*)text, strlen(text));

    // Append text to the existing file
    text = "World!\n";
```

```
38      append_file("/foo.txt", (const uint8_t*)text, strlen(text));
39
40      // Read the file content into a buffer and print it
41      uint8_t buffer[100];
42      read_file("/foo.txt", buffer, sizeof(buffer));
43      Serial.write(buffer, strlen((char*)buffer));
44
45      // Rename the file and read the new file content
46      rename_file("/foo.txt", "/hello.txt");
47      read_file("/hello.txt", buffer, sizeof(buffer));
48      Serial.write(buffer, strlen((char*)buffer));
49
50      // Print SD card information
51      Serial.println("\r\nSD card info:");
52      Serial.printf("Total space: %lluMB\r\n", SD_MMC.totalBytes() / (1024 * 1024));
53      Serial.printf("Used space: %lluMB\r\n", SD_MMC.usedBytes() / (1024 * 1024));
54  }
55
56  void loop() {
57      // Delay for 10 seconds before the next iteration
58      delay(10000);
59  }
```

Code Explanation

Include necessary header files.

```
7   #include "driver_sdmmc.h"
```

Define SD card pins. In this example, we read and write the SD card via SPI.

```
9   #define SD_MMC_CMD 40  // Please do not modify it.
10  #define SD_MMC_CLK 38  // Please do not modify it.
11  #define SD_MMC_D0  39  // Please do not modify it.
12  #define SD_MMC_D1  41  // Please do not modify it.
13  #define SD_MMC_D2  48  // Please do not modify it.
14  #define SD_MMC_D3  47  // Please do not modify it.
```

Set the baud rate to 115200.

```
18  Serial.begin(115200);
```

Initialize the SD card

```
19  sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);
```

List files in the root directory and print them

```
25  std::list<File_Entry> fileList = list_dir("/", 0);
26  print_file_list(fileList);
```

Create a directory named “mydir” and list all files under the root directory.

```
29  create_dir("/mydir");
30  fileList = list_dir("/", 0);
```

Delete the “mydir” directory and list all files under the root directory.

```
34  remove_dir("/mydir");
```

```
35  fileList = list_dir("/", 2);
```

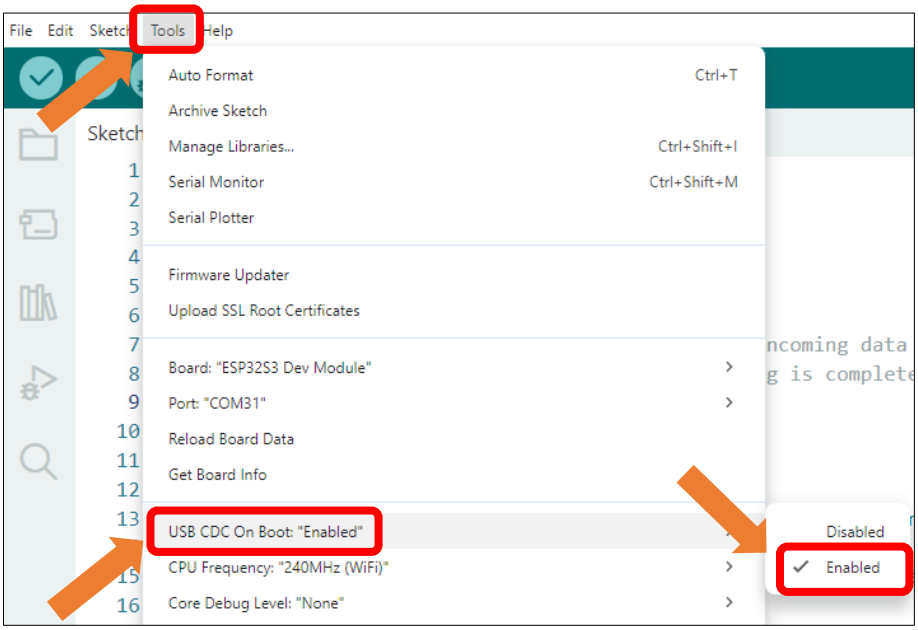
Write "Hello" in the hello.txt file, append "World!" to it, then read and print the file contents.

```
38  // Write text to a new file
39  const char* text = "Hello ";
40  write_file("/foo.txt", (const uint8_t*)text, strlen(text));
41
42  // Append text to the existing file
43  text = "World!\n";
44  append_file("/foo.txt", (const uint8_t*)text, strlen(text));
45
46  // Read the file content into a buffer and print it
47  uint8_t buffer[100];
48  read_file("/foo.txt", buffer, sizeof(buffer));
49  Serial.write(buffer, strlen((char*)buffer));
```

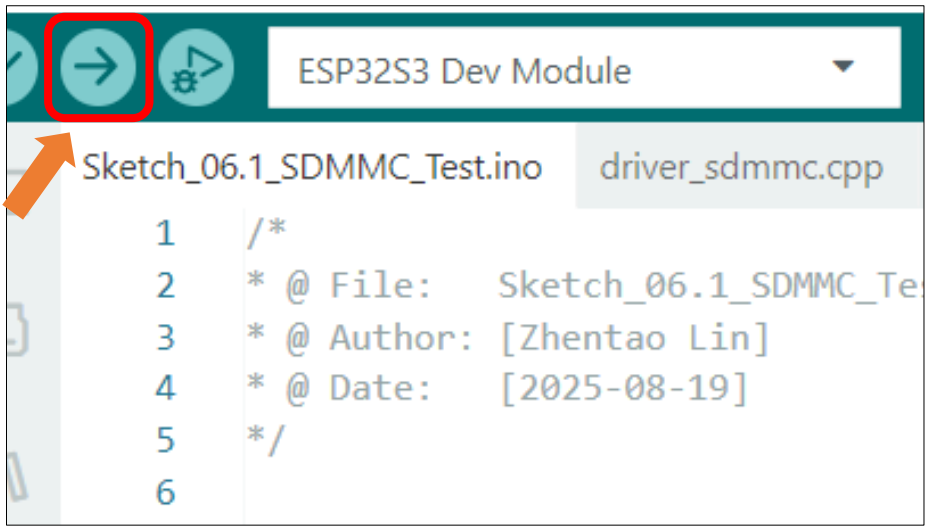
Rename foo.txt to hello.txt and read the file.

```
51  // Rename the file and read the new file content
52  rename_file("/foo.txt", "/hello.txt");
53  read_file("/hello.txt", buffer, sizeof(buffer));
54  Serial.write(buffer, strlen((char*)buffer));
```

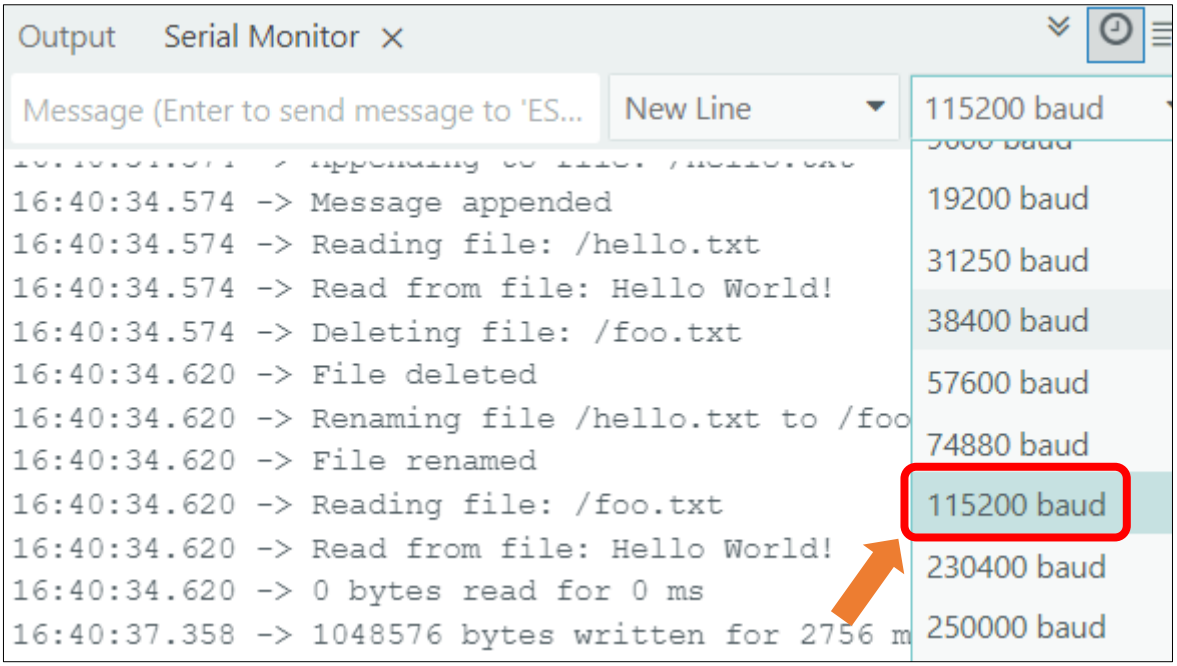
Enable the "USB CDC On Boot" feature.



Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.



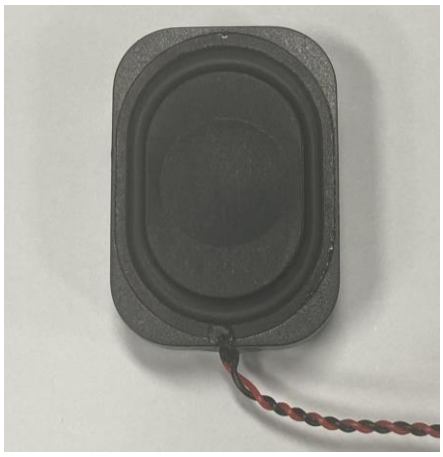
Upon the code runs, open the serial and set the baud rate to 115200.



Chapter 7 Music

Project 7.1 Music

Component List

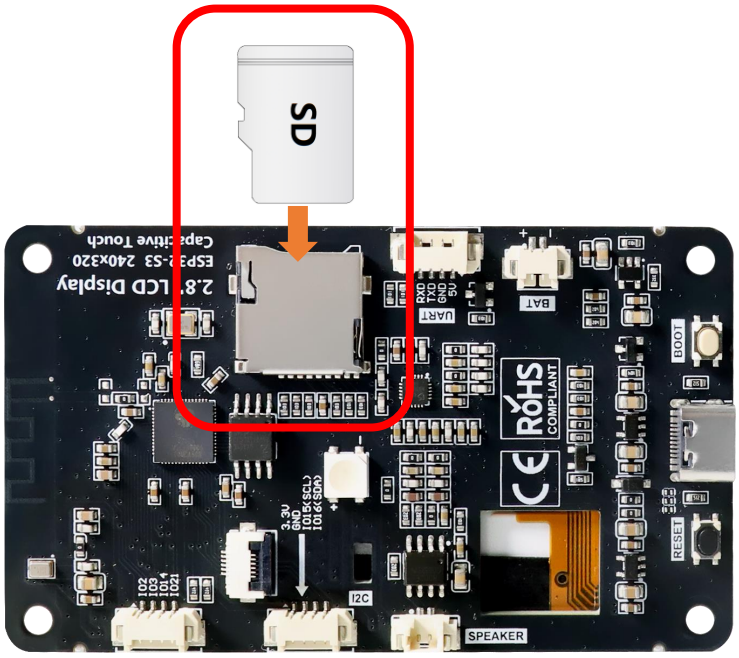
Freenove ESP32-S3 Display x 1 	USB cable x1 
Speaker x1 	

Please note that this kit does not include SD card and card reader, please buy them by yourself.

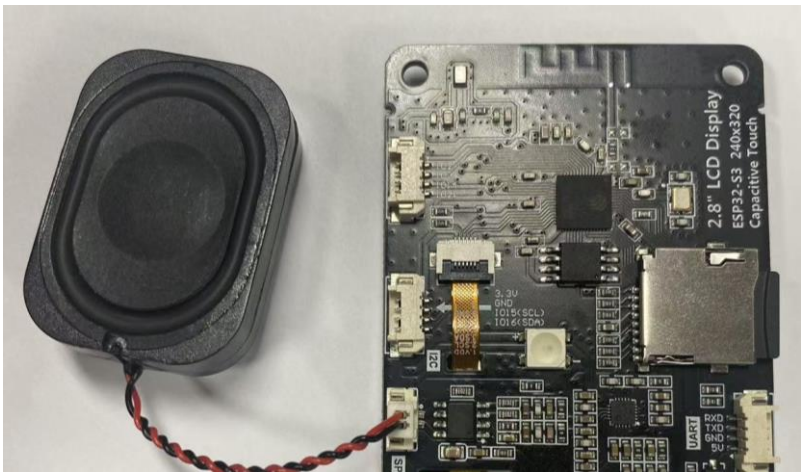
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

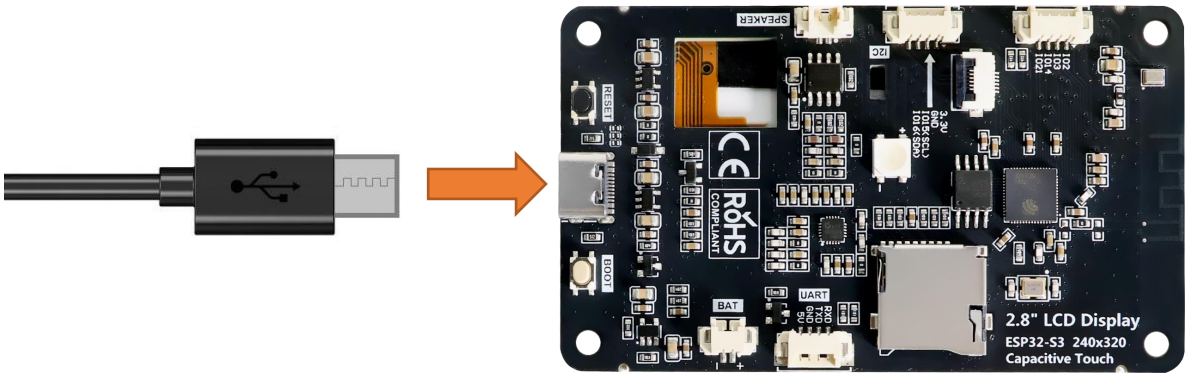
Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect speaker



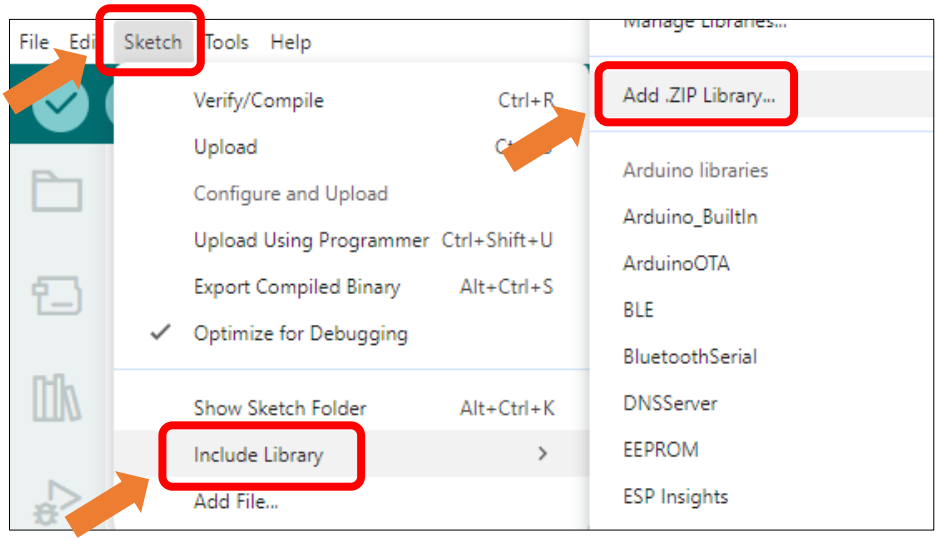
Connect Freenove ESP32-S3 to the computer using the USB cable.



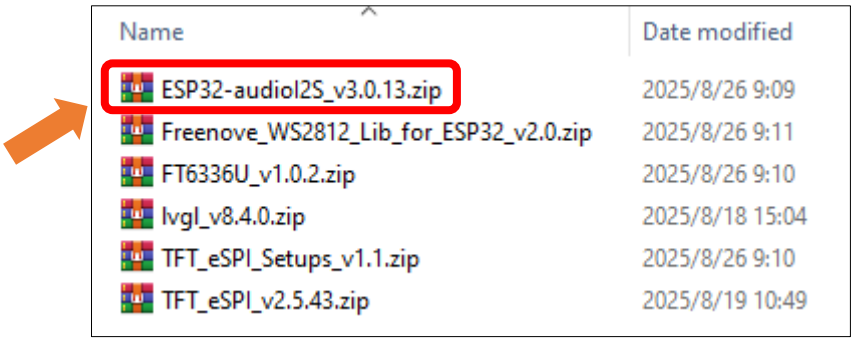
Sketch

Install the needed libraries.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **ESP32-audioI2S_v3.0.13.zip**



Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_07.1_Music**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_07.1_Music.ino**".

Sketch_07.1_Music

The following is the program code:

```
1  #include <Arduino.h>
2  #include "FS.h"
```

```
#include "SD_MMC.h"
#include "SPI.h"
#include "es8311.h"
#include "Audio.h"
#include "Wire.h"
#include "ESP_I2S.h"
#include "demo_music.h"

//ESP32-S3 IO Pin define
#define SD_SCK 38
#define SD_CMD 40
#define SD_D0 39
#define SD_D1 41
#define SD_D2 48
#define SD_D3 47

//I2S IO Pin define
#define I2S_MCK 4
#define I2S_BCK 5
#define I2S_DINT 6
#define I2S_DOUT 8
#define I2S_WS 7
#define AP_ENABLE 1
#define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
#define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
#define I2C_SPEED 400000  /*!< I2C master clock frequency */

Audio audio;
I2SClass es8311_i2s;

void driver_es8311_init(void) {
   pinMode(AP_ENABLE, OUTPUT);
   digitalWrite(AP_ENABLE, LOW);

   Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);

   es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
   if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO,
I2S_STD_SLOT_LEFT)) {
     Serial.println("Failed to initialize I2S bus!");
   }
}

void setup() {
```

```
47      Serial.begin(115200);
48
49      //SD card init
50      if (!SD_MMC.setPins(SD_SCK, SD_CMD, SD_D0, SD_D1, SD_D2, SD_D3)) {
51        Serial.println("Pin change failed!");
52        return;
53      }
54      while (!SD_MMC.begin()) {
55        Serial.println("SD card does not exist,please insert SD card.");
56        delay(100);
57      }
58
59      driver_es8311_init();
60      if (es8311_codec_init() != ESP_OK) {
61        Serial.println("ES8311 init failed!");
62        return;
63      }
64      //audio init
65      audio.setPinout(I2S_BCK, I2S_WS, I2S_DOUT, I2S_MCK);
66      audio.setVolume(10);
67      if (!demo_music()) {
68        return;
69      }
70
71      //play music
72      demo_music_play(0);
73  }
74
75  void loop() {
76      audio.loop();
77  }
```

Code Explanation

Include necessary header files.

```
1   #include <Arduino.h>
2   #include "FS.h"
3   #include "SD_MMC.h"
4   #include "SPI.h"
5   #include "es8311.h"
6   #include "Audio.h"
7   #include "Wire.h"
8   #include "ESP_I2S.h"
```

Define the pins.

```
10  //ESP32-S3 IO Pin define
11  #define SD_SCK 38
```

```
#define SD_CMD 40
#define SD_D0 39
#define SD_D1 41
#define SD_D2 48
#define SD_D3 47

//I2S IO Pin define
#define I2S_MCK 4
#define I2S_BCK 5
#define I2S_DINT 6
#define I2S_DOUT 8
#define I2S_WS 7
#define AP_ENABLE 1
#define I2C_SCL 15
#define I2C_SDA 16
#define I2C_SPEED 400000
```

Declare an I2S object

```
Audio audio;
I2SClass es8311_i2s;
```

Set the baud rate to 115200

```
Serial.begin(115200);
```

SD card init

```
if (!SD_MMC.setPins(SD_SCK, SD_CMD, SD_D0, SD_D1, SD_D2, SD_D3)) {
    Serial.println("Pin change failed!");
    return;
}
while (!SD_MMC.begin()) {
    Serial.println("SD card does not exist,please insert SD card.");
    delay(100);
}

driver_es8311_init();
if (es8311_codec_init() != ESP_OK) {
    Serial.println("ES8311 init failed!");
    return;
}
```

Read audio data and play it.

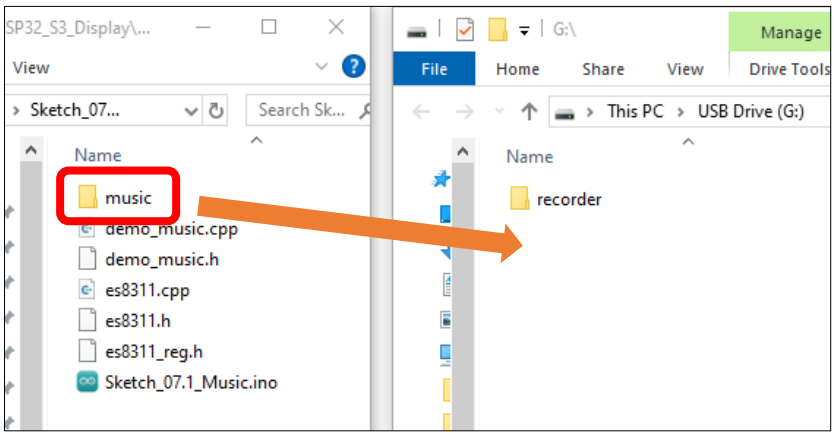
```
audio.setPinout(I2S_BCK, I2S_WS, I2S_DOUT, I2S_MCK);
audio.setVolume(10);
if (!demo_music()) {
    return;
}
```

play music

```
demo_music_play(1);
```

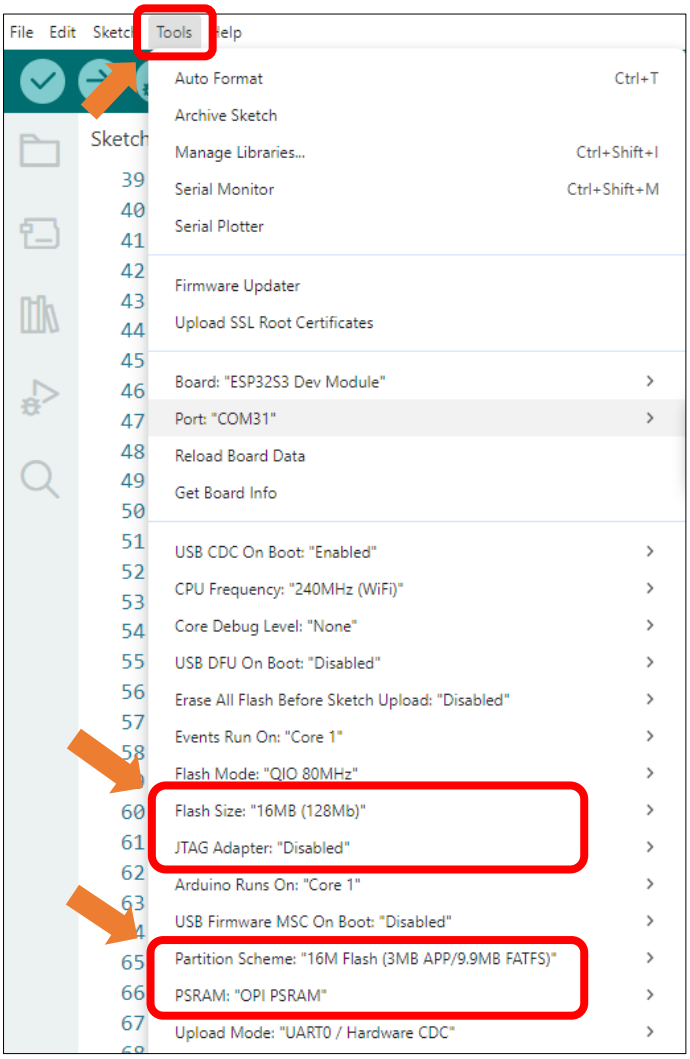
This product does not include SD card, and SD card reader, please buy them by yourself. For more information, please refer to SD card sections.

Before uploading the code, copy the music to the root directory of the SD card with the SD card reader.

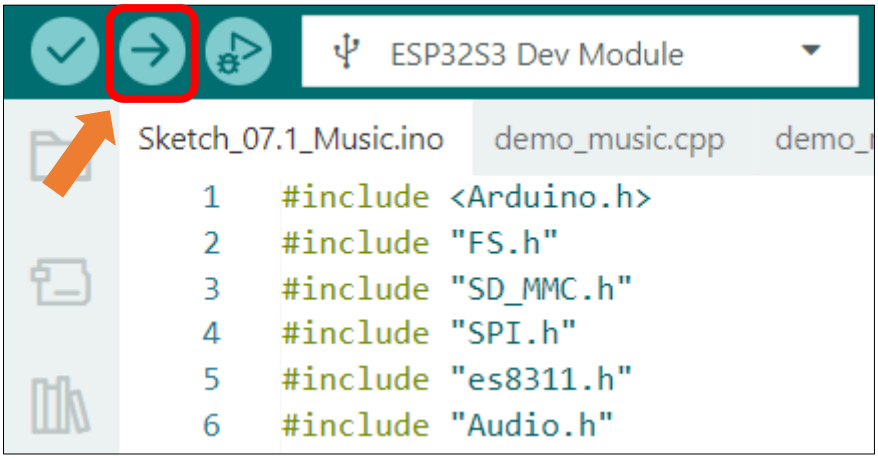


It is necessary to change the settings in Arduino IDE before clicking the Uploading button, as shown below.

Caution: Incorrect settings will result in compilation error or uploading failure. To achieve desired result, please configure exactly the same as below.



Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.



The speaker plays the music in the SD card.

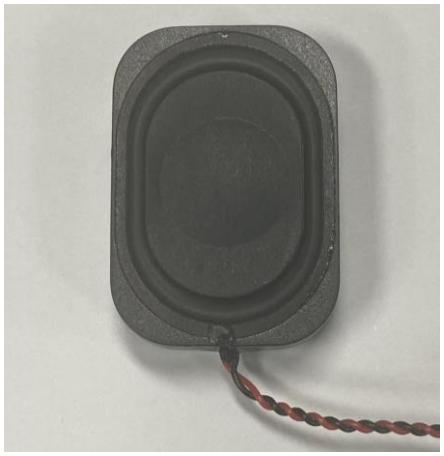
Project 7.2 Echo

Component List

Freenove ESP32-S3 Display x 1

USB cable x1

Speakerx1



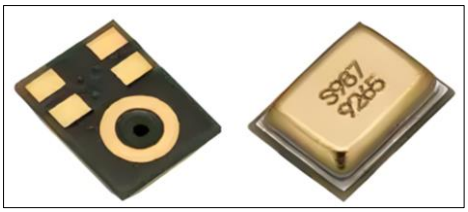
Please note that this kit does not include SD card and card reader, please buy them by yourself.

Component knowledge

MEMS-MIC

A MEMS Microphone (Micro-Electro-Mechanical Systems Microphone) is a miniature microphone manufactured using MEMS technology. It integrates mechanical sensing elements and electronic circuits on the same chip to achieve sound signal acquisition and conversion. Its working principle primarily involves a tiny vibrating diaphragm that detects sound pressure changes, then converts these mechanical vibrations into electrical signals, enabling sound capture and transmission.

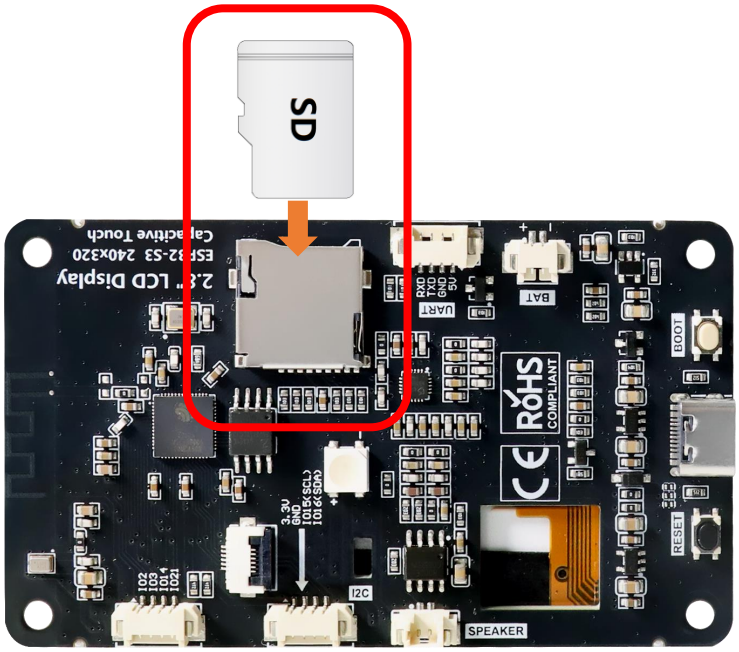
MEMS microphones are characterized by their compact size, high sensitivity, excellent stability, and ease of mass production. They are widely used in electronic devices such as smartphones, earphones, and smart speakers. Compared to traditional microphones, MEMS microphones better meet the dual demands of modern electronic products for both miniaturization and performance.



Circuit

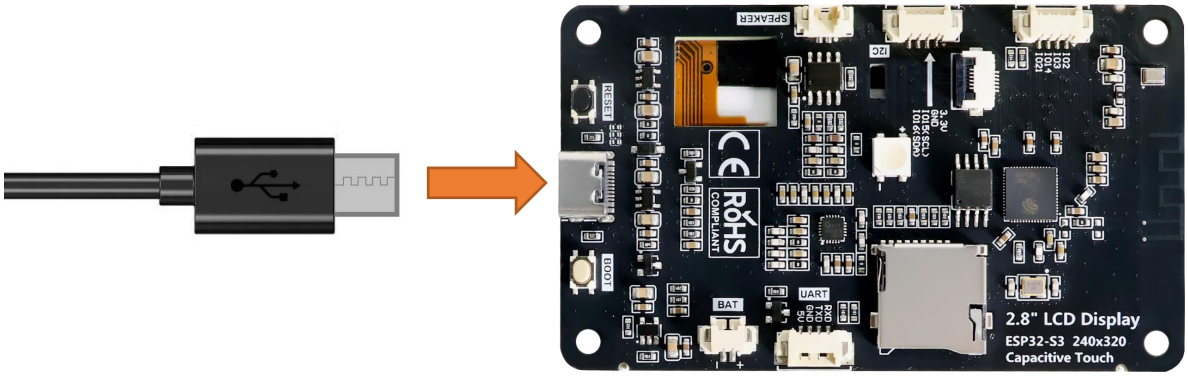
Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect speaker

Connect Freenove ESP32-S3 Display to the computer using the USB cable.



Sketch

Sketch_07.2_Echo

The following is the program code:

```
#include <Arduino.h>
#include "FS.h"
#include "SD_MMC.h"
#include "SPI.h"
#include "es8311.h"
#include "Audio.h"
#include "Wire.h"
#include "ESP_I2S.h"

//ESP32-S3 IO Pin define
#define SD_SCK 38
#define SD_CMD 40
#define SD_D0 39
```

```
#define SD_D1 41
#define SD_D2 48
#define SD_D3 47

//I2S IO Pin define
#define I2S_MCK 4
#define I2S_BCK 5
#define I2S_DINT 6
#define I2S_DOUT 8
#define I2S_WS 7
#define AP_ENABLE 1
#define I2C_SCL 15
#define I2C_SDA 16
#define I2C_SPEED 400000

I2SClass es8311_i2s;

void driver_es8311_init(void) {
   pinMode(AP_ENABLE, OUTPUT);
   digitalWrite(AP_ENABLE, LOW);

   Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);

   es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
   if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_MONO,
I2S_STD_SLOT_LEFT)) {
     Serial.println("Failed to initialize I2S bus!");
   }
}

void setup()
{
   Serial.begin(115200);
   while (!Serial) {
     delay(10);
   }

   Serial.println("Start initializing the audio device...");
   driver_es8311_init();
   if (es8311_codec_init() != ESP_OK) {
     Serial.println("ES8311 init failed!");
     return;
   }
   delay(3000);
```

```
58      Serial.println("Initialization completed.");
59  }
60
61  void loop()
62  {
63      uint8_t *wav_buffer;
64      size_t wav_size;
65      // Record 5 seconds of audio data
66      Serial.println("Start recording for 5 seconds...");
67      wav_buffer = es8311_i2s.recordWAV(5, &wav_size);
68      Serial.println("Recording completed.");
69      delay(1000);
70
71      Serial.println("Start playing the recording...");
72      es8311_i2s.playWAV(wav_buffer, wav_size);
73      Serial.println("Playback has been completed.");
74      free(wav_buffer);
75      delay(1000);
76  }
```

Code Explanation

Include necessary header files.

```
1  #include <Arduino.h>
2  #include "FS.h"
3  #include "SD_MMC.h"
4  #include "SPI.h"
5  #include "es8311.h"
6  #include "Audio.h"
7  #include "Wire.h"
8  #include "ESP_I2S.h"
```

Define the pins.

```
10  //ESP32-S3 IO Pin define
11  #define SD_SCK 38
12  #define SD_CMD 40
13  #define SD_D0 39
14  #define SD_D1 41
15  #define SD_D2 48
16  #define SD_D3 47
17
18  //I2S IO Pin define
19  #define I2S_MCK 4
20  #define I2S_BCK 5
21  #define I2S_DINT 6
22  #define I2S_DOUT 8
23  #define I2S_WS 7
```

```
24  #define AP_ENABLE 1
25  #define I2C_SCL 15
26  #define I2C_SDA 16
27  #define I2C_SPEED 400000
```

Declare an I2S object

```
19  I2SClass es8311_i2s;
```

Set the baud rate to 115200

```
36  Serial.begin(115200);
```

Initialize the audio device.

```
51  Serial.println("Start initializing the audio device...");
52  driver_es8311_init();
53  if (es8311_codec_init() != ESP_OK) {
54      Serial.println("ES8311 init failed!");
55      return;
56  }
```

Implement audio recording and playback functionality

```
61  void loop()
62  {
63      uint8_t *wav_buffer;
64      size_t wav_size;
65      // Record 5 seconds of audio data
66      Serial.println("Start recording for 5 seconds...");
67      wav_buffer = es8311_i2s.recordWAV(5, &wav_size);
68      Serial.println("Recording completed.");
69      delay(1000);
70
71      Serial.println("Start playing the recording...");
72      es8311_i2s.playWAV(wav_buffer, wav_size);
73      Serial.println("Playback has been completed.");
74      free(wav_buffer);
75      delay(1000);
76  }
```

Chapter 8 BLE

Project 8.1 BLE USART

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Component Knowledge

BLE (Bluetooth Low Energy)

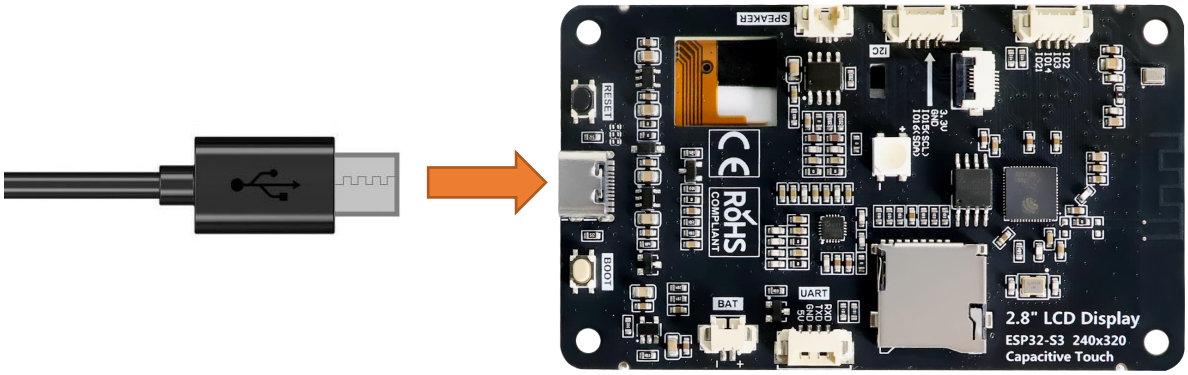
Low Energy Bluetooth (BLE) is a new feature introduced in the Bluetooth 4.0 specification, specifically designed for low-power devices and suitable for applications involving intermittent transmission of small amounts of data.

The BLE architecture follows a client-server model and consists of the following key components:
GATT (Generic Attribute Protocol): The foundational protocol for BLE communication, defining a hierarchical data structure of services and characteristics.
Service: A collection of data that performs a specific function or feature, containing one or more characteristics.
Characteristic: A specific data point within a service, consisting of a value and descriptors.
UUID: A 128-bit universally unique identifier used to distinguish services and characteristics.

BLE device can function simultaneously as a Peripheral (providing services) and a Central (connecting to peripherals). In this section, we will learn how to configure the Freenove_ESP32_S3_Display as a peripheral device to provide services.

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_08.1_BLE_USART**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_08.1_BLE_USART.ino**".

Sketch_08.1_BLE_USART

The following is the program code:

```
#include "BLEDevice.h"
#include "BLEServer.h"
#include "BLEUtils.h"
#include "BLE2902.h"
#include "String.h"

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
uint8_t txValue = 0;
long lastMsg = 0;
String rxload="Test\n";

#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
    };
    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
```

```
};

class MyCallbacks: public BLECharacteristicCallbacks {
      void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue();
        if (rxValue.length() > 0) {
          rxload="";
          for (int i = 0; i < rxValue.length(); i++){
            rxload +=(char)rxValue[i];
          }
        }
      }
};

void setupBLE(String BLEName){
    const char *ble_name=BLEName.c_str();
    BLEDevice::init(ble_name);
    BLEServer *pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());
    BLEService *pService = pServer->createService(SERVICE_UUID);
    pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX,BLECharacteristic::PROPERTY_NOTIFY);
    pCharacteristic->addDescriptor(new BLE2902());
    BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX,BLECharacteristic::PROPERTY_WRITE);
    pCharacteristic->setCallbacks(new MyCallbacks());
    pService->start();
    pServer->getAdvertising()->start();
    Serial.println("Waiting a client connection to notify...");
}

void setup() {
    Serial.begin(115200);
    setupBLE("ESP32S3_BLE");
}

void loop() {
    long now = millis();
    if (now - lastMsg > 100) {
      if (deviceConnected&&rxload.length()>0) {
          Serial.println(rxload);
          rxload="";
      }
      if(Serial.available()>0){
```

```
68            String str=Serial.readString();
69            const char *newValue=str.c_str();
70            pCharacteristic->setValue(newValue);
71            pCharacteristic->notify();
72        }
73        lastMsg = now;
74     }
75  }
```

Code Explanation

Include the necessary header libraries.

```
6  #include "BLEDevice.h"
7  #include "BLEServer.h"
8  #include "BLEUtils.h"
9  #include "BLE2902.h"
10 #include "String.h"
```

Define service UUID and characteristic UUID.

```
19 #define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
20 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
21 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```
23 class MyServerCallbacks: public BLEServerCallbacks {
24      void onConnect(BLEServer* pServer) {
25        deviceConnected = true;
26      };
27      void onDisconnect(BLEServer* pServer) {
28        deviceConnected = false;
29      }
30 };
```

The MyServerCallbacks class handles the received data and save them to the rxload string.

```
32 class MyCallbacks: public BLECharacteristicCallbacks {
33      void onWrite(BLECharacteristic *pCharacteristic) {
34        String rxValue = pCharacteristic->getValue();
35        if (rxValue.length() > 0) {
36          rxload="";
37          for (int i = 0; i < rxValue.length(); i++){
38            rxload +=(char)rxValue[i];
39          }
40        }
41      }
42 };
```

Set the baud rate to 115200.

```
62 Serial.begin(115200);
```

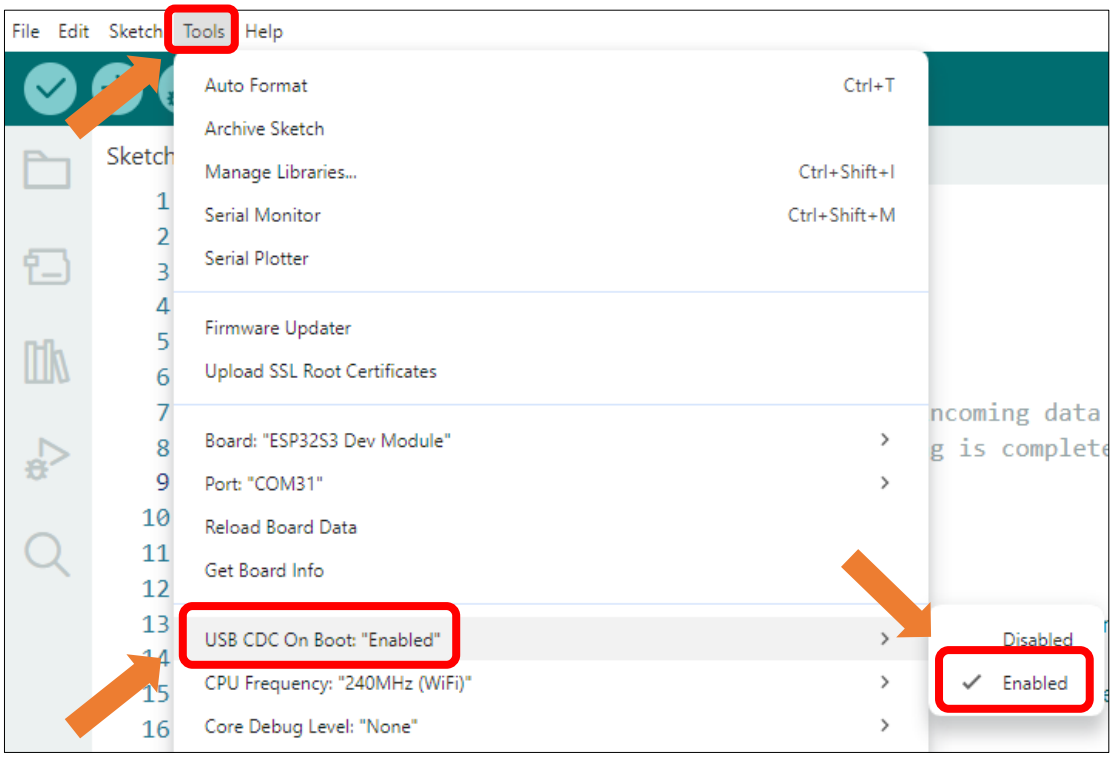
BLE Device Initialization, Service Creation, and Characteristic Setup

```
63  setupBLE("ESP32S3_BLE");
```

The Loop function check the connection status and serial data every 100 milliseconds.

```
66  void loop() {
67    long now = millis();
68    if (now - lastMsg > 100) {
69      if (deviceConnected&&rxload.length()>0) {
70          Serial.println(rxload);
71          rxload="";
72      }
73      if(Serial.available()>0){
74          String str=Serial.readString();
75          const char *newValue=str.c_str();
76          pCharacteristic->setValue(newValue);
77          pCharacteristic->notify();
78      }
79      lastMsg = now;
80    }
81  }
```

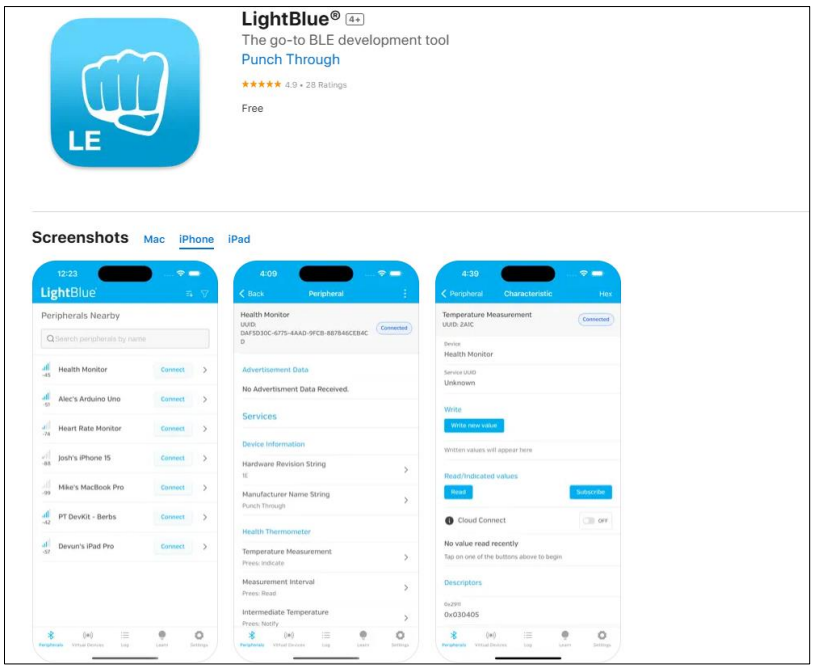
Enable the "USB CDC On Boot" feature.



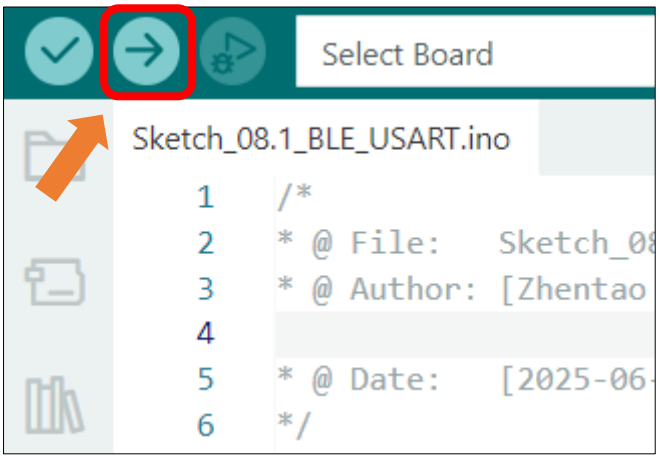
LightBlue

If you do not have this software installed on your phone, you can refer to this link:

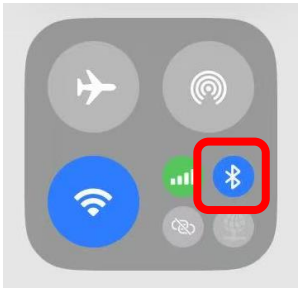
https://apps.apple.com/us/app/lightblue/id557428110



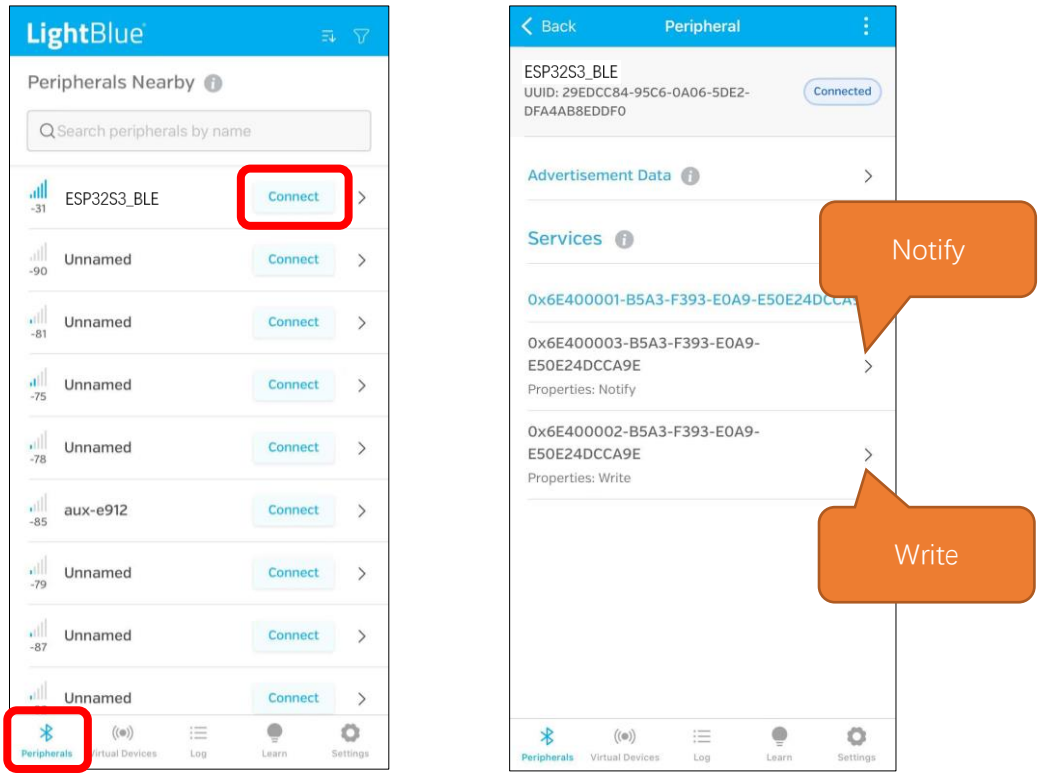
Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.



Turn ON Bluetooth on your phone, and open the LightBlue APP.

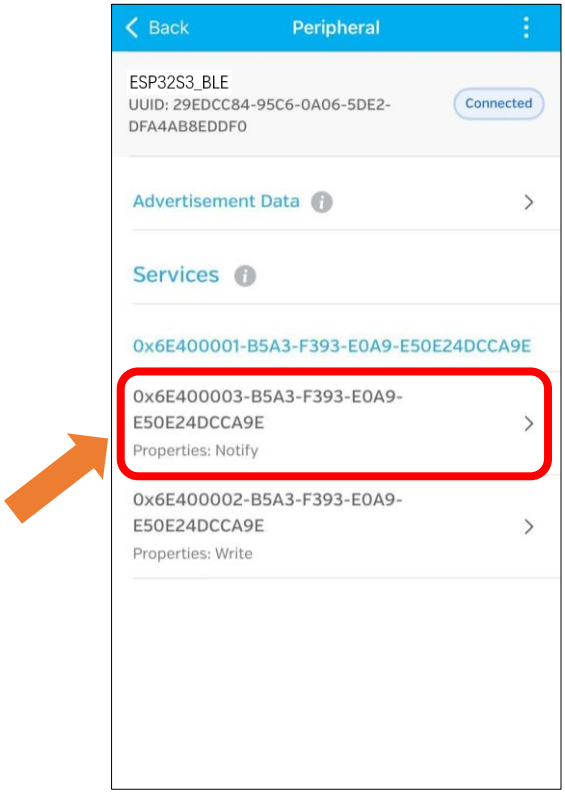


In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click the Connection button of ESP32S3_BLE

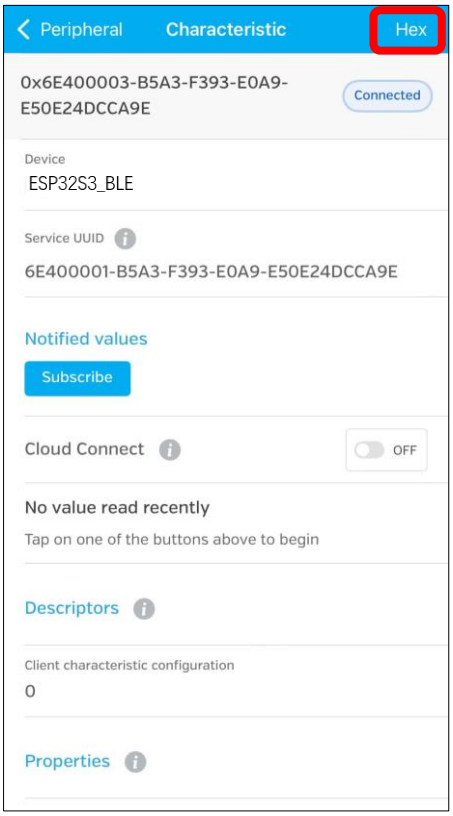


Receiving Data

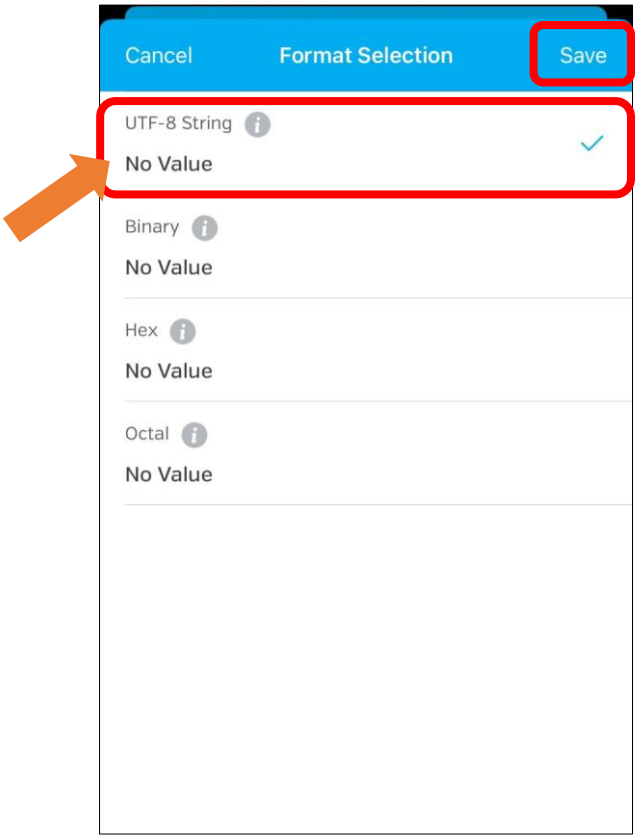
Click Notify



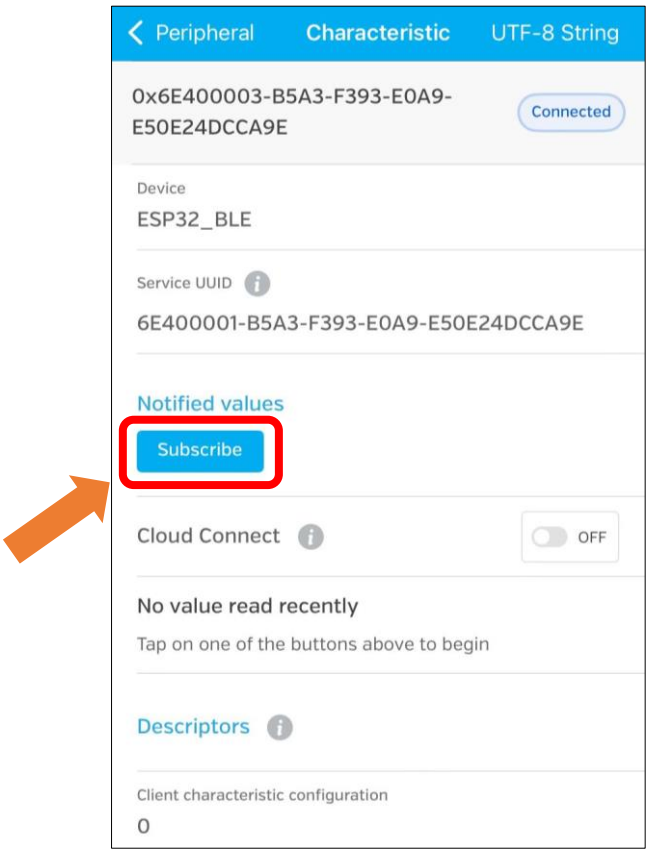
click the top right corner to change the string type.



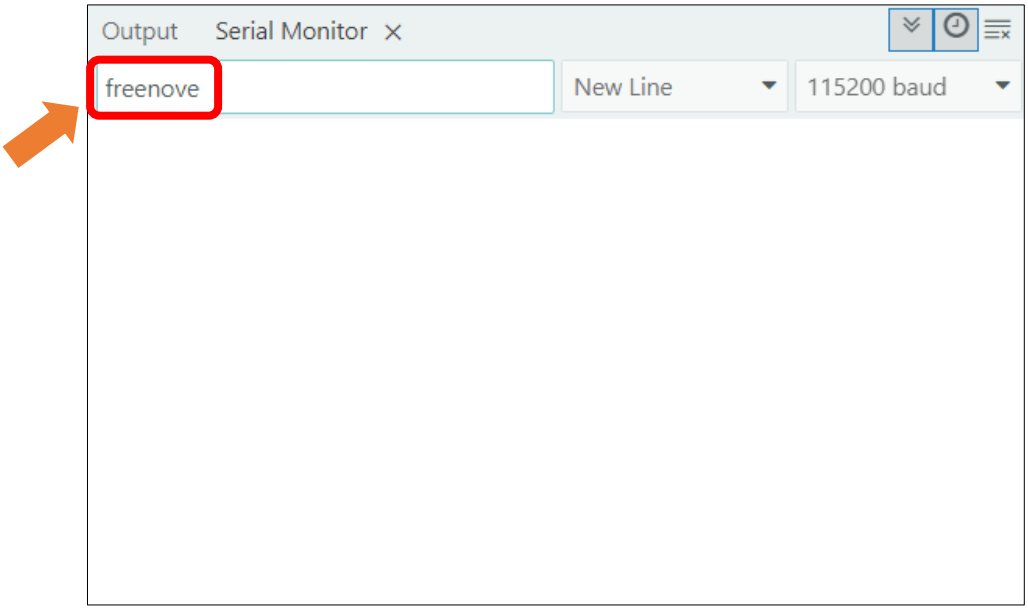
Select UTF-8 String, and click “Save”.



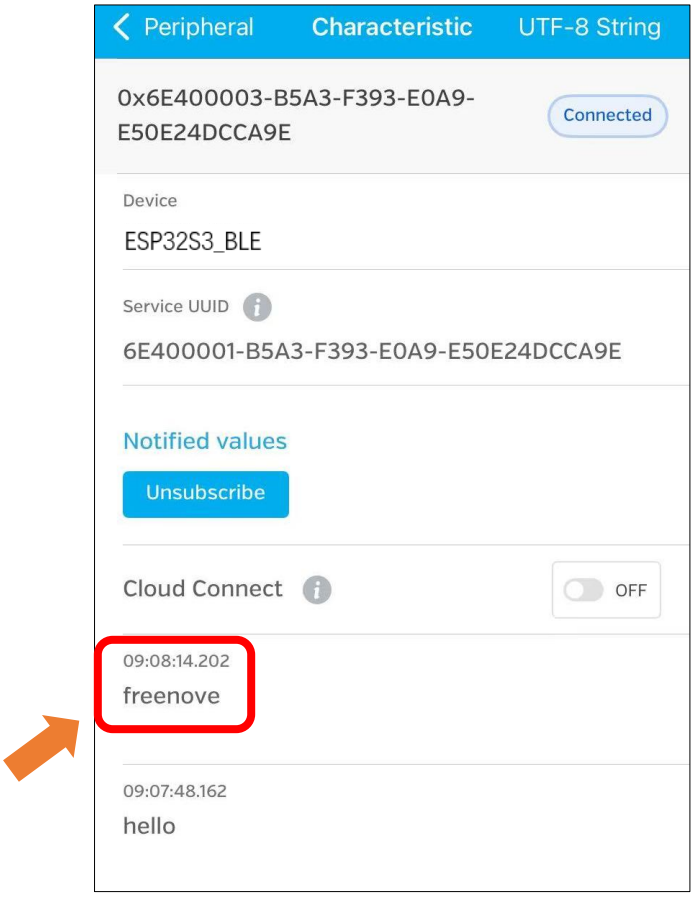
Click Subscribe



Send data on the serial monitor.

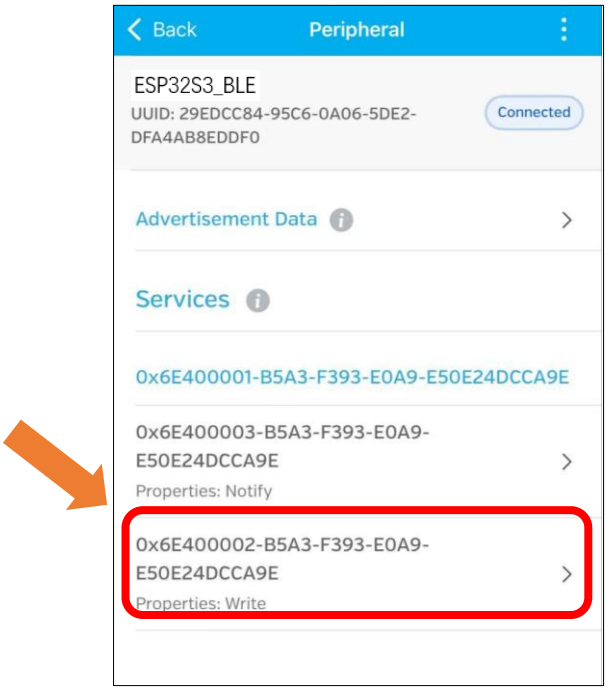


Data will be received on the LightBlue app.

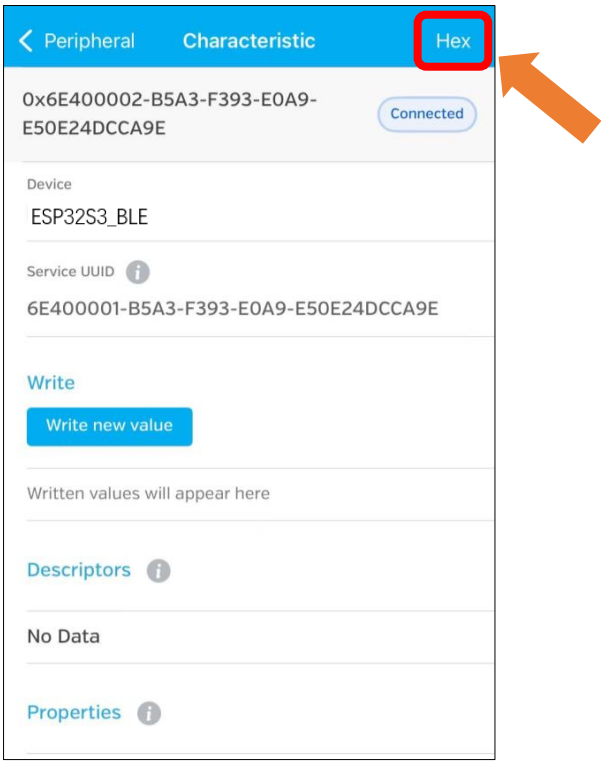


Sending Data

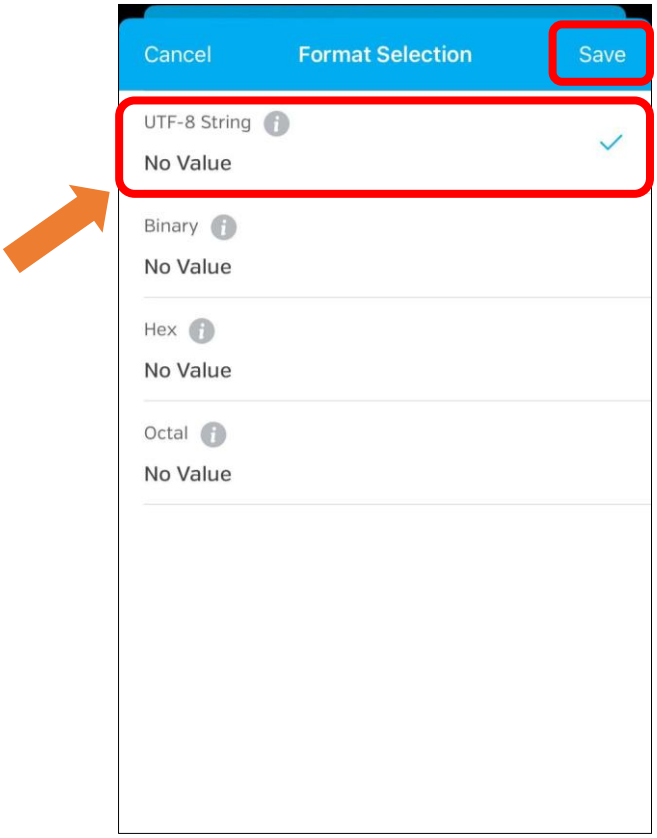
Click Write



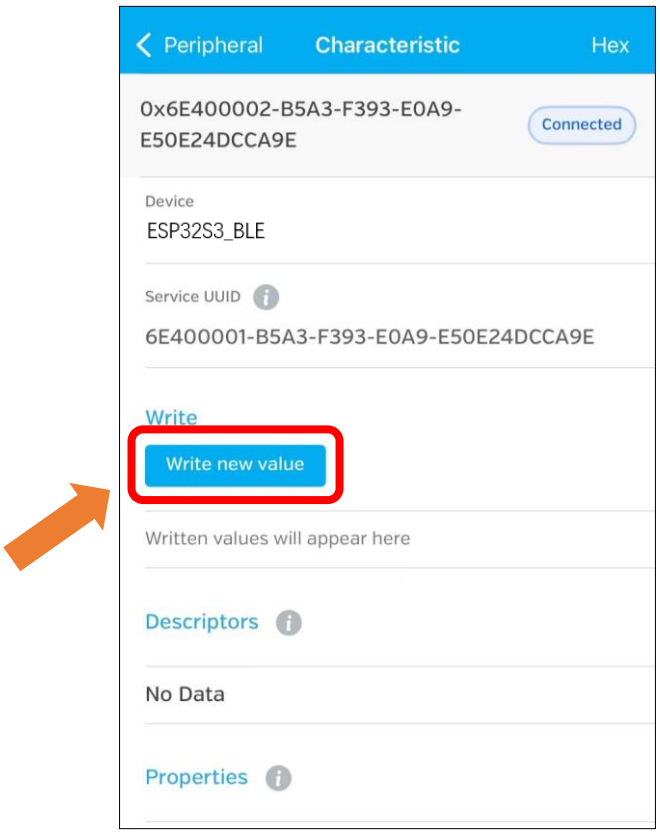
Click the top right corner to change the string type.



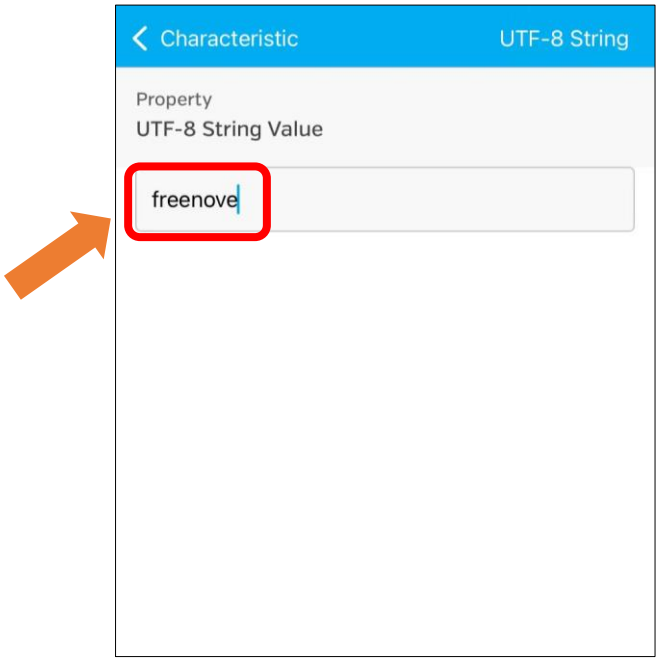
Select UTF-8 String and click “Save”.



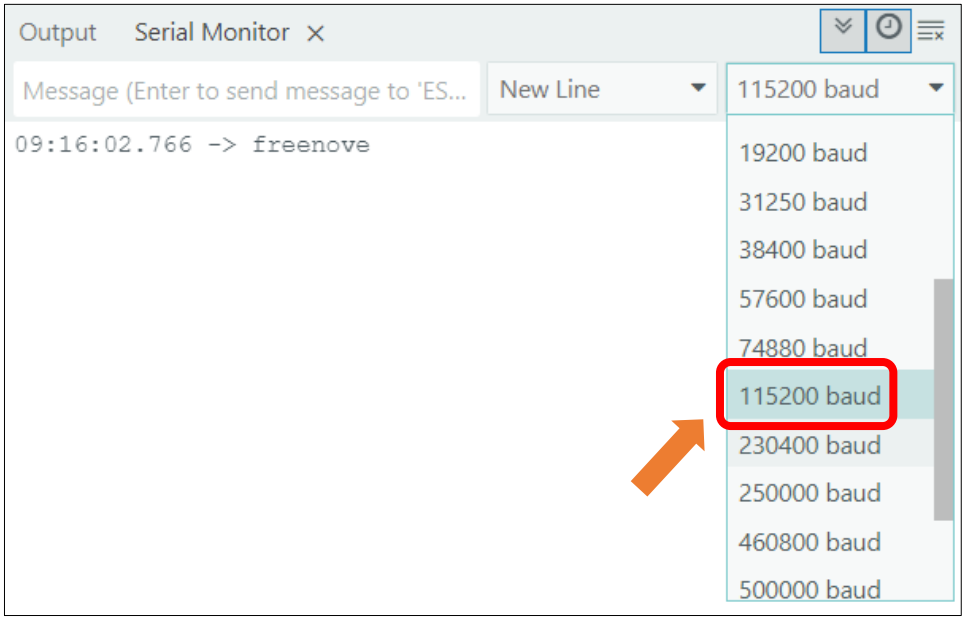
Click "Write new value"



Enter the messages to send.



Set the baud rate to 115200, and the serial monitor will print the data received.



Project 8.2 BLE RGB

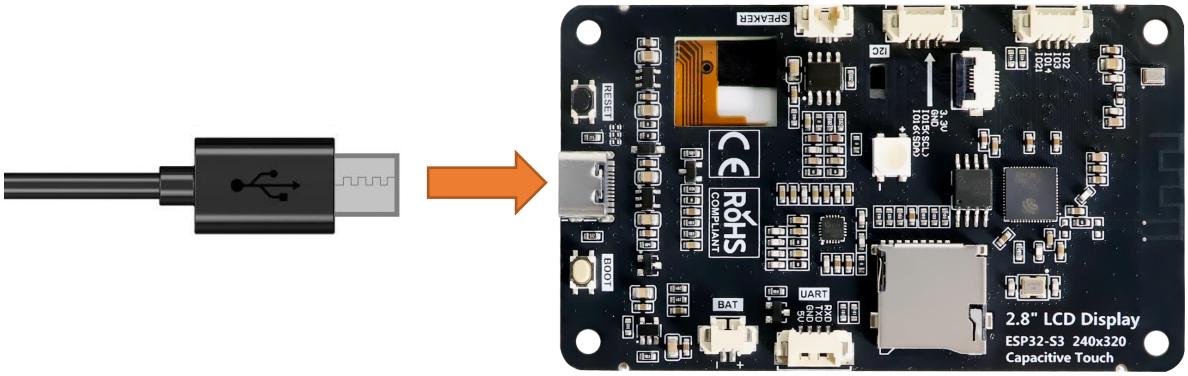
In this section we will control the RGB LED via BLE.

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test. Open"**Sketch_08.2_BLE_RGB**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_08.2_BLE_RGB.ino**".

Sketch_08.2_BLE_RGB

The following is the program code:

```
1 #include "BLEDevice.h"
2 #include "BLEServer.h"
3 #include "BLEUtils.h"
4 #include "BLE2902.h"
5 #include "String.h"
6 #include "Freenove_WS2812_Lib_for_ESP32.h"
```

```
7
8   BLECharacteristic *pCharacteristic;
9   bool deviceConnected = false;
10  uint8_t txValue = 0;
11  long lastMsg = 0;
12  String rxload = "Test\n";
13
14  #define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
15  #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
16  #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
17
18  #define KEY_PIN 0
19  #define LEDS_COUNT 1
20  #define LEDS_PIN   42
21  #define CHANNEL    0
22
23  Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);
24
25  uint8_t m_color[5][3] = { { 255, 0, 0 }, { 0, 255, 0 }, { 0, 0, 255 }, { 255, 255, 255 }, { 0,
26  0, 0 } };
27
28  void stripInit(void){
29     strip.begin();
30     strip.setBrightness(10);
31  }
32
33  void switchRGB(int value) {
34     int colorIndex = value % 4;
35     switch (colorIndex) {
36       case 1:
37         strip.setLedColorData(0, m_color[0][0], m_color[0][1], m_color[0][2]);
38         strip.show();
39         break;
40       case 2:
41         strip.setLedColorData(0, m_color[1][0], m_color[1][1], m_color[1][2]);
42         strip.show();
43         break;
44       case 3:
45         strip.setLedColorData(0, m_color[2][0], m_color[2][1], m_color[2][2]);
46         strip.show();
47         break;
48       default:
49         strip.setLedColorData(0, m_color[4][0], m_color[4][1], m_color[4][2]);
50         strip.show();
```

```
51          break;
52      }
53  }
54
55  class MyServerCallbacks : public BLEServerCallbacks {
56      void onConnect(BLEServer *pServer) {
57        deviceConnected = true;
58      };
59      void onDisconnect(BLEServer *pServer) {
60        deviceConnected = false;
61      }
62  };
63
64  class MyCallbacks : public BLECharacteristicCallbacks {
65      void onWrite(BLECharacteristic *pCharacteristic) {
66        String rxValue = pCharacteristic->getValue();
67        if (rxValue.length() > 0) {
68          rxload = "";
69          for (int i = 0; i < rxValue.length(); i++) {
70            rxload += (char)rxValue[i];
71          }
72        }
73      }
74  };
75
76  void setupBLE(String BLEName) {
77      const char *ble_name = BLEName.c_str();
78      BLEDevice::init(ble_name);
79      BLEServer *pServer = BLEDevice::createServer();
80      pServer->setCallbacks(new MyServerCallbacks());
81      BLEService *pService = pServer->createService(SERVICE_UUID);
82      pCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_TX,
83  BLECharacteristic::PROPERTY_NOTIFY);
84      pCharacteristic->addDescriptor(new BLE2902());
85      BLECharacteristic *pCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_RX,
86  BLECharacteristic::PROPERTY_WRITE);
87      pCharacteristic->setCallbacks(new MyCallbacks());
88      pService->start();
89      pServer->getAdvertising()->start();
90      Serial.println("Waiting a client connection to notify...");
91  }
92
93  void setup() {
94      Serial.begin(115200);
```

```
95      stripInit();
96      switchRGB(0);
97      setupBLE("ESP32S3_BLE");
98  }
99
100 void loop() {
101     long now = millis();
102     if (now - lastMsg > 100) {
103       if (deviceConnected && rxload.length() > 0) {
104         Serial.println(rxload);
105         if (strncmp(rxload.c_str(), "red_on", 6) == 0) {
106           switchRGB(1);
107         } else if (strncmp(rxload.c_str(), "red_off", 7) == 0) {
108           switchRGB(0);
109         } else if (strncmp(rxload.c_str(), "green_on", 8) == 0) {
110           switchRGB(2);
111         } else if (strncmp(rxload.c_str(), "green_off", 9) == 0) {
112           switchRGB(0);
113         } else if (strncmp(rxload.c_str(), "blue_on", 7) == 0) {
114           switchRGB(3);
115         } else if (strncmp(rxload.c_str(), "blue_off", 8) == 0) {
116           switchRGB(0);
117         }
118         rxload = "";
119       }
120       if (Serial.available() > 0) {
121         String str = Serial.readString();
122         const char *newValue = str.c_str();
123         pCharacteristic->setValue(newValue);
124         pCharacteristic->notify();
125       }
126       lastMsg = now;
127     }
128 }
```

Code Explanation

Include the necessary header files.

```
7   #include "BLEDevice.h"
8   #include "BLEServer.h"
9   #include "BLEUtils.h"
10  #include "BLE2902.h"
11  #include "String.h"
12  #include "Freenove_WS2812_Lib_for_ESP32.h"
```

Define Service UUID and Characteristic UUID.

```
20  #define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
```

```
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
```

Define pins for the RGB LED.

```
#define KEY_PIN 0
#define LEDS_COUNT 1
#define LEDS_PIN   42
#define CHANNEL    0
```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
    };
    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
};
```

The MyCallbacks class handles the receiving data and save them to the rxload string.

```
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String rxValue = pCharacteristic->getValue();
      if (rxValue.length() > 0) {
        rxload="";
        for (int i = 0; i < rxValue.length(); i++){
          rxload +=(char)rxValue[i];
        }
      }
    }
};
```

Set the baud rate to 115200

```
Serial.begin(115200);
```

Initialize RGB LED setting, initialize the BLE device, create services, and set up characteristics.

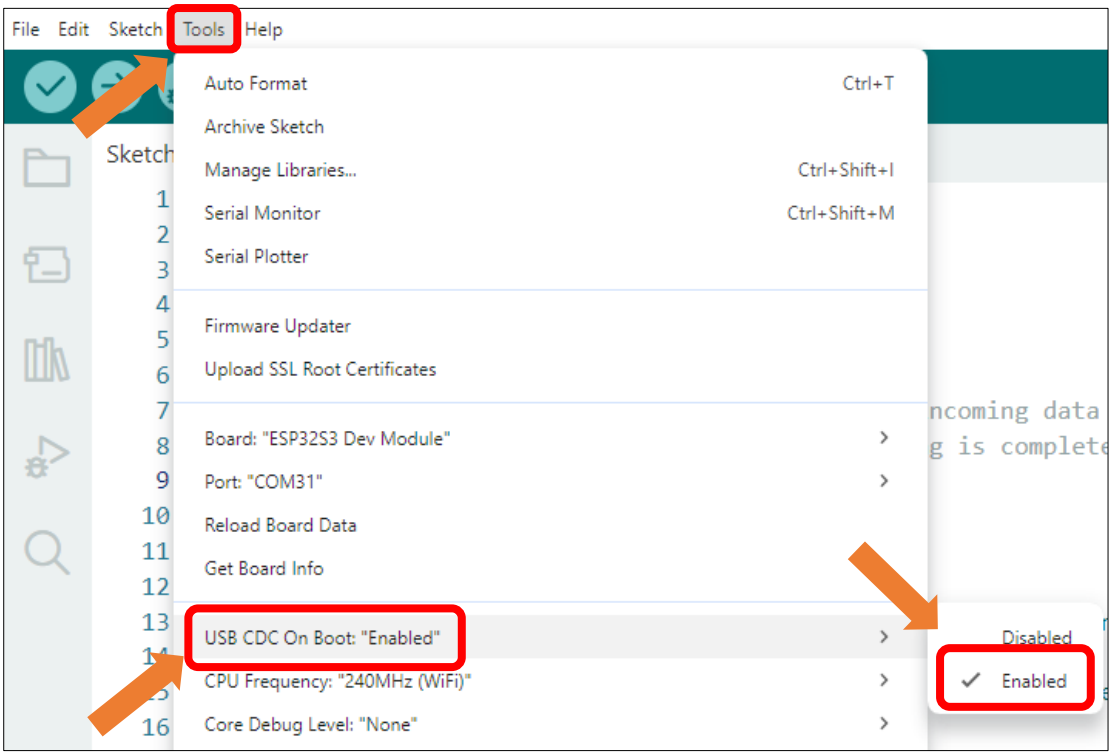
```
stripInit();
switchRGB(0);
setupBLE("ESP32S3_BLE");
```

The Loop function checks the sending command every 100 milliseconds.

```
void loop() {
  long now = millis();
  if (now - lastMsg > 100) {
    if (deviceConnected && rxload.length() > 0) {
      Serial.println(rxload);
      if (strncmp(rxload.c_str(), "red_on", 6) == 0) {
        setRGB(1, 0, 0);
      } else if (strncmp(rxload.c_str(), "red_off", 7) == 0) {
```

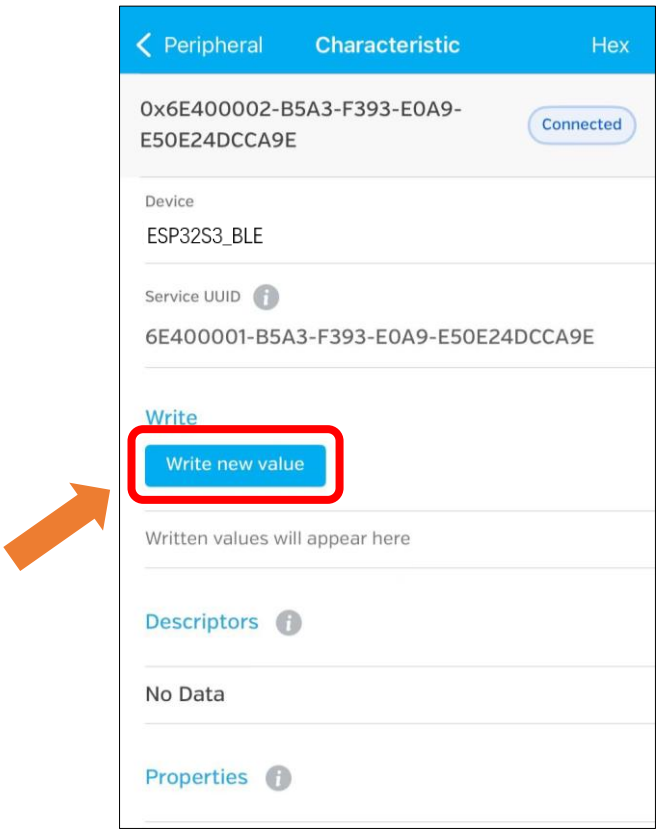
```
            setRGB(0, 0, 0);
          } else if (strncmp(rxload.c_str(), "green_on", 8) == 0) {
            setRGB(0, 1, 0);
          } else if (strncmp(rxload.c_str(), "green_off", 9) == 0) {
            setRGB(0, 0, 0);
          } else if (strncmp(rxload.c_str(), "blue_on", 7) == 0) {
            setRGB(0, 0, 1);
          } else if (strncmp(rxload.c_str(), "blue_off", 8) == 0) {
            setRGB(0, 0, 0);
          }
          rxload = "";
        }
        if (Serial.available() > 0) {
          String str = Serial.readString();
          const char *newValue = str.c_str();
          pCharacteristic->setValue(newValue);
          pCharacteristic->notify();
        }
        lastMsg = now;
      }
}
```

Enable the "USB CDC On Boot" feature.

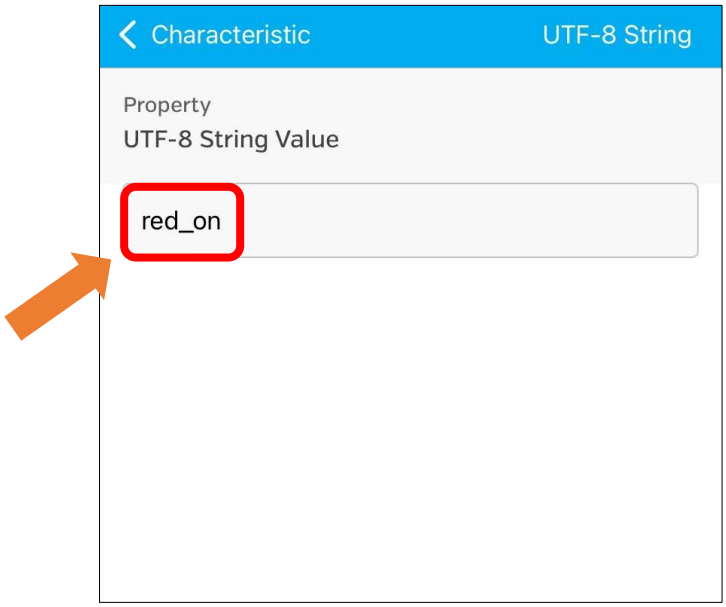


Click "**Upload**" to upload the code to Freenove ESP32-S3 Display.

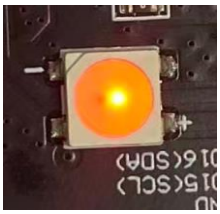
Click "Write new value"



Enter the messages to send. Here we take “red_on” as an example.



The RGB LED on the Freenove ESP32-S3 Display emits red light.



You can also use the following instructions to control the RGB LED:

red_off: red light off
green_on: green light on
green_off: green light off
blue_on: blue light on
blue_off: blue light off

Chapter 9 WIFI Web Server

Project 9.1 WIFI Web Servers LED

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Component Knowledge

Wi-Fi

Wi-Fi is a wireless LAN (WLAN) technology compliant with the IEEE 802.11 standard, enabling devices to transmit data or connect to the internet via radio waves within short-range coverage (typically up to tens of meters). In embedded systems, Wi-Fi modules provide wireless communication capabilities, allowing devices to connect to routers, access cloud services, or interact with other devices.

Its core features include:

- Wireless connectivity (eliminating the need for physical cabling)
- Moderate to high-speed data transfer (depending on protocol versions such as 802.11n/ac)
- Secure communication (ensured by encryption protocols like WPA2/WPA3)

In embedded development, Wi-Fi is one of the key technologies for enabling IoT (Internet of Things) device connectivity.

Web

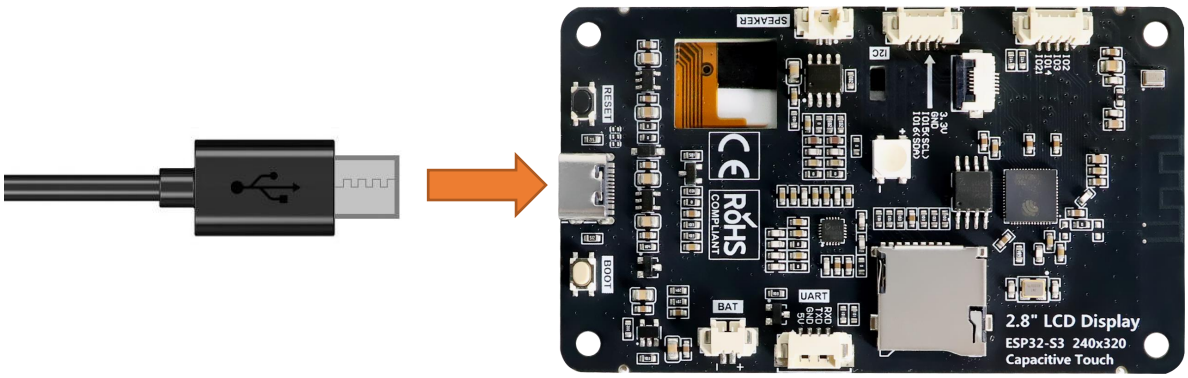
The Web (World Wide Web) is an internet-based information system that integrates global resources through hypertext links. In the embedded field, Web technology refers to devices equipped with lightweight built-in web servers, allowing users to remotely access the device interface via browsers (such as Chrome or Edge). By entering the device's IP address, users can open an interactive page to perform functions like status monitoring, parameter configuration, or firmware upgrades. Its core value lies in cross-platform compatibility (no need for dedicated software installation) and standardized protocols (HTTP/HTTPS), making it a mainstream solution for remote management of embedded devices.

HTML & CSS & JavaScript

- **HTML (HyperText Markup Language)** is the standard language for structuring web pages. It uses tags to define page elements, forming the foundational framework. In embedded web interfaces, HTML describes the layout of components such as device status displays and configuration forms.
- **CSS (Cascading Style Sheets)** controls the visual presentation of web pages, including colors, fonts, spacing, and responsive layouts. Through CSS rules, developers style HTML elements into intuitive interactive interfaces. The combination of HTML and CSS enables the creation of lightweight device control pages without complex graphics libraries, reducing resource overhead in embedded systems.
- **JavaScript (a scripting language)** adds dynamic behavior and interaction logic to web pages. In embedded web interfaces, JavaScript plays a crucial role: it responds to user actions and enables asynchronous communication with the device backend through technologies like **AJAX** or **WebSocket**. This allows the webpage to fetch real-time device status data, update displays without refreshing, and send control commands or configuration parameters—delivering a smooth and efficient remote management experience.

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_S3_Display to test. Open "**Sketch_09.1_WiFi_Web_Servers_LED**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_09.1_WiFi_Web_Servers_LED.ino**".

Important Note: Before upload the code, please ensure that the Wi-Fi SSID and Password are correctly configured in the code, and that the Wi-Fi network to connect is 2.4GHz.

```
10  const char* ssid     = "**************";
11  const char* password = "**************";
```

Sketch_09.1_WiFi_Web_Servers_LED

The following is the program code:

```
1  #include <WiFi.h>
2  #include "Freenove_WS2812_Lib_for_ESP32.h"
3
4  // WiFi credentials
```

```
5   const char* ssid     = "**************";
6   const char* password = "**************";
7
8   // Web server on port 80
9   WiFiServer server(80);
10
11  // HTTP request buffer
12  String request;
13
14  // WS2812 LED strip configuration
15  #define LEDS_COUNT 1
16  #define LEDS_PIN   42
17  #define CHANNEL    0
18
19  Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);
20
21  // LED state tracking (0=Red, 1=Green, 2=Blue)
22  bool ledStates[3] = {false, false, false}; // All OFF initially
23  String redLedState = "OFF";
24  String greenLedState = "OFF";
25  String blueLedState = "OFF";
26
27  // Timeout settings
28  unsigned long currentTime = millis();
29  unsigned long previousTime = 0;
30  const long timeoutTime = 2000;  // 2 seconds
31
32  void setupStrip(void) {
33     strip.begin();
34     strip.setBrightness(10);
35     strip.setLedColorData(0, 0, 0, 0); // Turn off all LEDs
36     strip.show();
37  }
38
39  void updateLedStrip(void) {
40     uint8_t r = ledStates[0] ? 255 : 0;
41     uint8_t g = ledStates[1] ? 255 : 0;
42     uint8_t b = ledStates[2] ? 255 : 0;
43
44     strip.setLedColorData(0, r, g, b);
45     strip.show();
46  }
47
48  void setup() {
```

```
    Serial.begin(115200);

    // Initialize LED strip
    setupStrip();

    // Connect to WiFi
    Serial.print("Connecting to ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi connected.");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());

    server.begin();
}

void loop() {
    WiFiClient client = server.available();

    if (client) {
      currentTime = millis();
      previousTime = currentTime;
      Serial.println("New Client.");
      String currentLine = "";

      while (client.connected() && currentTime - previousTime <= timeoutTime) {
        currentTime = millis();
        if (client.available()) {
          char c = client.read();
          Serial.write(c);
          request += c;

          if (c == '\n') {
            if (currentLine.length() == 0) {
              // HTTP response headers
              // Handle incoming commands first. These are AJAX requests.
              if (request.indexOf("GET /red/ON") >= 0) {
                Serial.println("Red LED ON");
```

```
93                redLedState = "ON";
94                ledStates[0] = true;
95              } else if (request.indexOf("GET /red/OFF") >= 0) {
96                Serial.println("Red LED OFF");
97                redLedState = "OFF";
98                ledStates[0] = false;
99              } else if (request.indexOf("GET /green/ON") >= 0) {
100               Serial.println("Green LED ON");
101               greenLedState = "ON";
102               ledStates[1] = true;
103             } else if (request.indexOf("GET /green/OFF") >= 0) {
104               Serial.println("Green LED OFF");
105               greenLedState = "OFF";
106               ledStates[1] = false;
107             } else if (request.indexOf("GET /blue/ON") >= 0) {
108               Serial.println("Blue LED ON");
109               blueLedState = "ON";
110               ledStates[2] = true;
111             } else if (request.indexOf("GET /blue/OFF") >= 0) {
112               Serial.println("Blue LED OFF");
113               blueLedState = "OFF";
114               ledStates[2] = false;
115             }
116
117             // Update LED strip based on states
118             updateLedStrip();
119
120             // Send HTML page for non-AJAX requests
121             if (request.indexOf("/red/") < 0 && request.indexOf("/green/") < 0 &&
122 request.indexOf("/blue/") < 0) {
123               // Send the Complete HTML page
124               client.println("HTTP/1.1 200 OK");
125               client.println("Content-type:text/html");
126               client.println("Connection: close");
127               client.println();
128
129               // Generate HTML response
130               client.println("<!DOCTYPE html><html>");
131               client.println("<head><meta charset=\"UTF-8\">");
132               client.println("<meta name=\"viewport\" content=\"width=device-width, initial-
133 scale=1.0\">");
134               client.println("<title>ESP32 Web Server LED</title>");
135               client.println("<style>");
136
```

```
137                 client.println("html{font-family:'Segoe UI',Roboto,sans-
138 serif;background:#f5f5f5;height:100vh;display:flex;justify-content:center;align-
139 items:center;}");
140                 client.println("body{margin:0;padding:20px;background:white;border-
141 radius:15px;box-shadow:0 10px 30px rgba(0,0,0,0.1);max-width:800px;width:90vw;}");
142                 client.println(".container{display:grid;grid-template-columns:repeat(3,
143 1fr);gap:20px;margin-top:20px;}");
144                 client.println(".card{border-radius:15px;padding:25px;text-align:center;min-
145 width:180px;transition:all 0.3s;cursor:pointer;border:2px
146 solid;position:relative;overflow:hidden;}");
147                 client.println(".card:hover{transform:translateY(-5px);box-shadow:0 15px 35px
148 rgba(0,0,0,0.15);}");
149                 client.println(".card:active{transform:translateY(0) scale(0.98);}");
150                 client.println("#red-card{background:linear-
151 gradient(145deg,#ffcdd2,#ef9a9a);border-color:#f44336;}");
152                 client.println("#green-card{background:linear-
153 gradient(145deg,#c8e6c9,#a5d6a7);border-color:#4caf50;}");
154                 client.println("#blue-card{background:linear-
155 gradient(145deg,#bbdefb,#90caf9);border-color:#2196f3;}");
156                 client.println(".card.off-state{filter:grayscale(70%) brightness(0.85);}");
157                 client.println("h1{color:#333;text-align:center;font-size:2.4rem;margin-
158 bottom:10px;border-bottom:2px solid #eee;padding-bottom:15px;}");
159                 client.println(".status{font-size:1.4rem;font-weight:600;margin:25px
160 0;color:#333;position:relative;z-index:2;}");
161                 client.println(".device-info{text-align:center;margin-top:25px;color:#777;font-
162 size:0.9rem;padding-top:15px;border-top:1px solid #eee;}");
163                 client.println("@media (max-width:768px){.container{grid-template-columns:1fr;}
164 body{padding:15px;} h1{font-size:2rem;}}");
165                 client.println("</style></head>");
166                 client.println("<body>");
167                 client.println("<h1>ESP32 Web Server LED</h1>");
168                 client.println("<div class=\"container\">");
169 
170                 // Red LED control
171                 client.println("<div class=\"card " + String(redLedState == "OFF" ? "off-
172 state" : "") + "\" id=\"red-card\" onclick=\"toggleLED('red', this)\">");
173                 client.println("<h2>Red LED</h2>");
174                 client.println("<p class=\"status\" id=\"status-red\">" + redLedState +
175 "</p>");
176                 client.println("</div>");
177 
178                 // Green LED control
179                 client.println("<div class=\"card " + String(greenLedState == "OFF" ? "off-
180 state" : "") + "\" id=\"green-card\" onclick=\"toggleLED('green', this)\">");
```

```
181                 client.println("<h2>Green LED</h2>");
182                 client.println("<p class=\"status\" id=\"status-green\">" + greenLedState +
183 "</p>");
184                 client.println("</div>");
185 
186                 // Blue LED control
187                 client.println("<div class=\"card " + String(blueLedState == "OFF" ? "off-
188 state" : "") + "\" id=\"blue-card\" onclick=\"toggleLED('blue', this)\">");
189                 client.println("<h2>Blue LED</h2>");
190                 client.println("<p class=\"status\" id=\"status-blue\">" + blueLedState +
191 "</p>");
192                 client.println("</div>");
193 
194                 client.println("</div>");
195                 client.println("<div class=\"device-info\">Device IP: " +
196 WiFi.localIP().toString() + " | ESP32 Web Server LED</div>");
197                 client.println("<script>");
198                 client.println("function toggleLED(color, element) {");
199                 client.println("  const statusElement = document.getElementById('status-' +
200 color);");
201                 client.println("  const currentState = statusElement.innerText;");
202                 client.println("  const newState = currentState === 'ON' ? 'OFF' : 'ON';");
203                 client.println("  fetch('/' + color + '/' + newState);");
204                 client.println("  statusElement.innerText = newState;");
205                 client.println("  if (newState === 'ON') { element.classList.remove('off-
206 state'); } else { element.classList.add('off-state'); }");
207                 client.println("}");
208                 client.println("</script>");
209                 client.println("</body></html>");
210               }
211               break;
212             } else {
213               currentLine = "";
214             }
215           } else if (c != '\r') {
216             currentLine += c;
217           }
218         }
219       }
220 
221       request = "";
222       client.stop();
223       Serial.println("Client disconnected.");
224       Serial.println("");
```

```
225     }
226 }
```

Code Explanation

Include necessary header files.

```
7   #include <WiFi.h>
```

Configure WiFi SSID and password.

```
12  const char* ssid     = "**************";
13  const char* password = "**************";
```

Define pins for the RGB LED.

```
21  // WS2812 LED strip configuration
22  #define LEDS_COUNT 1
23  #define LEDS_PIN   42
24  #define CHANNEL    0
```

Initialize the configuration related to RGB LED.

```
40  strip.begin();
41  strip.setBrightness(10);
42  strip.setLedColorData(0, 0, 0, 0); // Turn off all LEDs
43  strip.show();
```

Connect to WIFI.

```
61  // Connect to WiFi
62  Serial.print("Connecting to ");
63  Serial.println(ssid);
64  WiFi.begin(ssid, password);
65  while (WiFi.status() != WL_CONNECTED) {
66      delay(500);
67      Serial.print(".");
68  }
69
70  Serial.println("");
71  Serial.println("WiFi connected.");
72  Serial.println("IP address: ");
73  Serial.println(WiFi.localIP());
74
75  server.begin();
```

Check if there are any new clients connected to the server.

```
78  WiFiClient client = server.available();
```

Parse the URL command and control the corresponding LED on and off.

```
87  // Handle incoming commands
88  if (request.indexOf("GET /red/ON") >= 0) {
89      Serial.println("Red LED ON");
90      redLedState = "ON";
91      ledStates[0] = true;
92  } else if (request.indexOf("GET /red/OFF") >= 0) {
93      Serial.println("Red LED OFF");
```

```
    redLedState = "OFF";
    ledStates[0] = false;
  } else if (request.indexOf("GET /green/ON") >= 0) {
    Serial.println("Green LED ON");
    greenLedState = "ON";
    ledStates[1] = true;
  } else if (request.indexOf("GET /green/OFF") >= 0) {
    Serial.println("Green LED OFF");
    greenLedState = "OFF";
    ledStates[1] = false;
  } else if (request.indexOf("GET /blue/ON") >= 0) {
    Serial.println("Blue LED ON");
    blueLedState = "ON";
    ledStates[2] = true;
  } else if (request.indexOf("GET /blue/OFF") >= 0) {
    Serial.println("Blue LED OFF");
    blueLedState = "OFF";
    ledStates[2] = false;
  }
```

Use CSS styles to enhance the visual appeal of web interfaces.

```
client.println("html{font-family:'Segoe UI',Roboto,sans-
serif;background:#f5f5f5;height:100vh;display:flex;justify-content:center;align-
items:center;}");
client.println("body{margin:0;padding:20px;background:white;border-radius:15px;box-shadow:0
10px 30px rgba(0,0,0,0.1);max-width:800px;width:90vw;}");
client.println(".container{display:grid;grid-template-columns:repeat(3, 1fr);gap:20px;margin-
top:20px;}");
client.println(".card{border-radius:15px;padding:25px;text-align:center;min-
width:180px;transition:all 0.3s;cursor:pointer;border:2px
solid;position:relative;overflow:hidden;}");
client.println(".card:hover{transform:translateY(-5px);box-shadow:0 15px 35px
rgba(0,0,0,0.15);}");
client.println(".card:active{transform:translateY(0) scale(0.98);}");
client.println("#red-card{background:linear-gradient(145deg,#ffcdd2,#ef9a9a);border-
color:#f44336;}");
client.println("#green-card{background:linear-gradient(145deg,#c8e6c9,#a5d6a7);border-
color:#4caf50;}");
client.println("#blue-card{background:linear-gradient(145deg,#bbdefb,#90caf9);border-
color:#2196f3;}");
client.println(".card.off-state{filter:grayscale(70%) brightness(0.85);}");
client.println("h1{color:#333;text-align:center;font-size:2.4rem;margin-bottom:10px;border-
bottom:2px solid #eee;padding-bottom:15px;}");
client.println(".status{font-size:1.4rem;font-weight:600;margin:25px
0;color:#333;position:relative;z-index:2;}");
```

```
145  client.println(".device-info{text-align:center;margin-top:25px;color:#777;font-
146  size:0.9rem;padding-top:15px;border-top:1px solid #eee;}");
147  client.println("@media (max-width:768px){.container{grid-template-columns:1fr;}
148  body{padding:15px;} h1{font-size:2rem;}}");
```

Generate the HTML contents to control the LED.

```
154  // Red LED control
155  client.println("<div class=\"card " + String(redLedState == "OFF" ? "off-state" : "") + "\"
156  id=\"red-card\" onclick=\"toggleLED('22', this)\">");
157  client.println("<h2>Red LED</h2>");
158  client.println("<p class=\"status\" id=\"status-22\">" + redLedState + "</p>");
159  client.println("</div>");
160
161  // Green LED control
162  client.println("<div class=\"card " + String(greenLedState == "OFF" ? "off-state" : "") + "\"
163  id=\"green-card\" onclick=\"toggleLED('16', this)\">");
164  client.println("<h2>Green LED</h2>");
165  client.println("<p class=\"status\" id=\"status-16\">" + greenLedState + "</p>");
166  client.println("</div>");
167
168  // Blue LED control
169  client.println("<div class=\"card " + String(blueLedState == "OFF" ? "off-state" : "") + "\"
170  id=\"blue-card\" onclick=\"toggleLED('17', this)\">");
171  client.println("<h2>Blue LED</h2>");
172  client.println("<p class=\"status\" id=\"status-17\">" + blueLedState + "</p>");
173  client.println("</div>");
```

Disconnect from the client and print the disconnection information in the serial monitor

```
172  client.stop();
173  Serial.println("Client disconnected.");
174  Serial.println("");
```

Input correct WiFi(2.4GHz) SSID and password.

```
 9   // WiFi credentials
10   const char* ssid     = "**************";
11   const char* password = "**************";
```

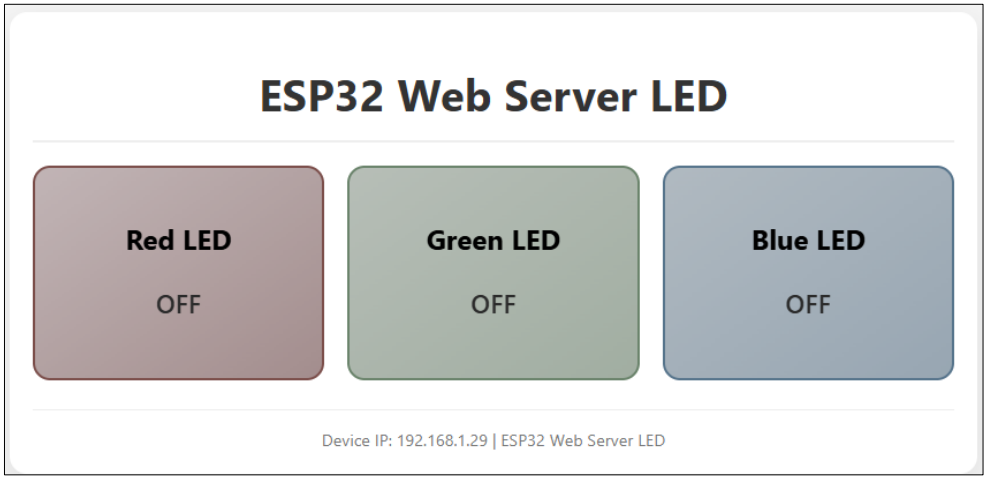
Click "**Upload**" to upload the code to Freenove ESP32 Display.

When the serial monitor prints the following information, it means the network connection is successful. Use a mobile phone or computer browser to open the IP address printed by the serial monitor.

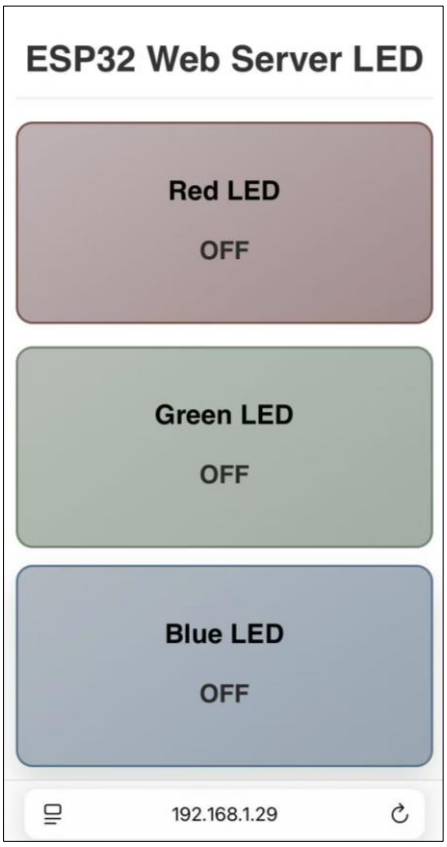
Please note: If it keeps printing dots, please confirm whether the WiFi is in the 2.4G band.

```
14:31:24.553 -> Connecting to FYI_2.4G
14:31:25.115 -> .....
14:31:27.088 -> WiFi connected.
14:31:27.088 -> IP address:
14:31:27.122 -> 192.168.1.29
```

The following screen will be displayed in the computer's browser, where you can control the color of the RGB light by clicking on the three cards.



On the phone web browser, you'll see the following contents. The color of the RGB light can be controlled by clicking on the three cards.



Chapter 10 TFT Display

Project 10.1 TFT_Rainbow

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Component Knowledge

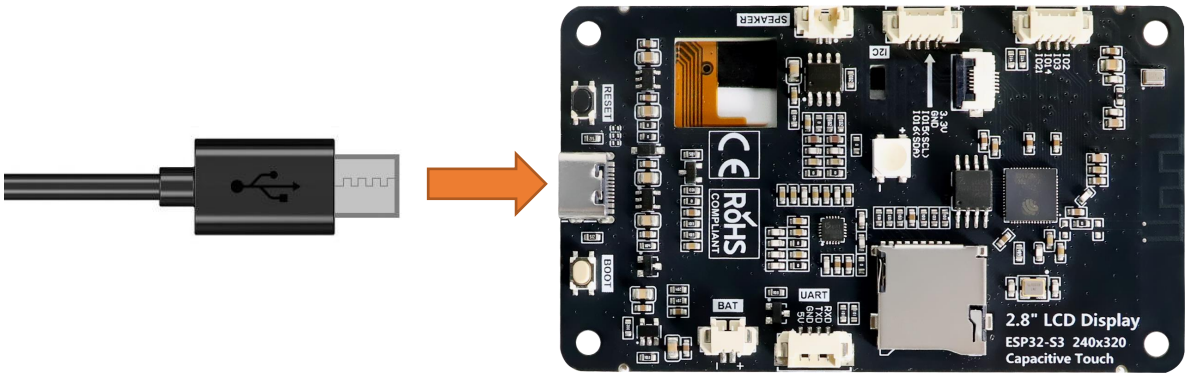
TFT Display

TFT (Thin Film Transistor) is an electronic component that serves as the foundation for TFT displays, the mainstream display technology in modern laptops and desktop computers. In these displays, each individual liquid crystal pixel is controlled by its own dedicated thin-film transistor embedded directly behind it. This architecture classifies TFT screens as a form of active-matrix LCD (AMLCD) technology.

As one of the finest LCD color displays available, TFT screens offer superior performance characteristics including rapid response times, exceptional brightness levels, and outstanding contrast ratios.

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.

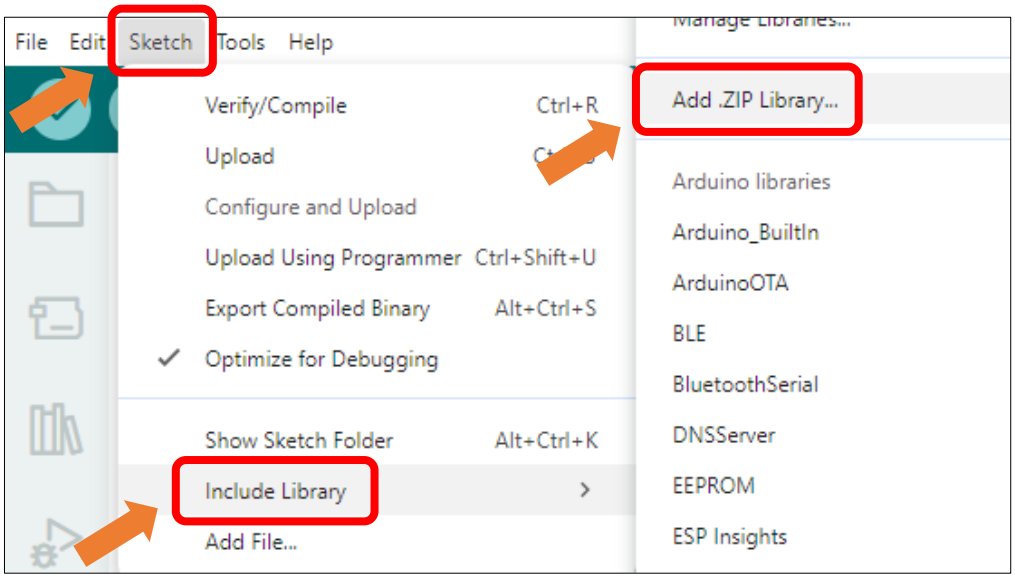


Sketch

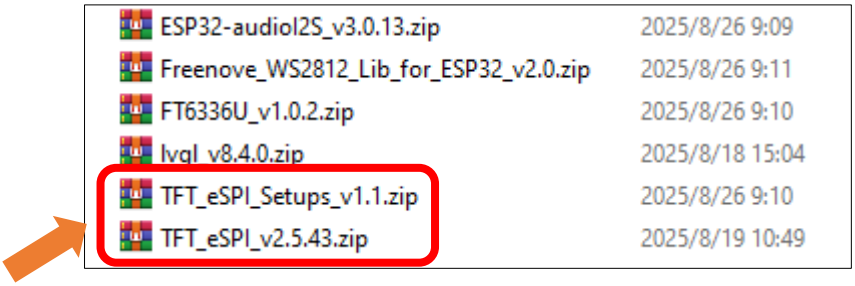
Open " **Sketch_10.1_TFT_Rainbow.ino** " folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_10.1_TFT_Rainbow.ino**".

Install Libraries

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**

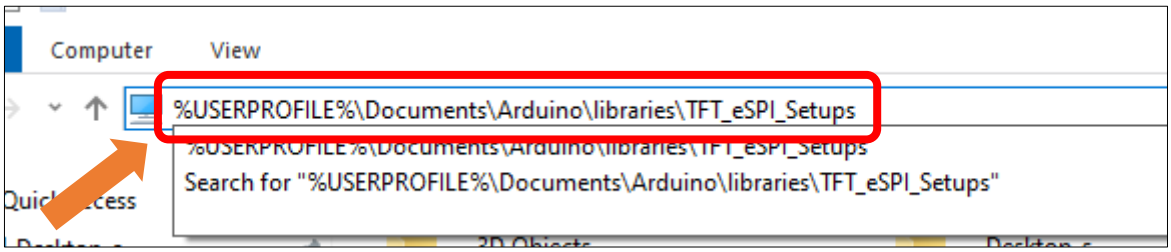


Install **TFT_eSPI_v2.5.43.zip** and **TFT_eSPI_Setups_v1.1.zip**

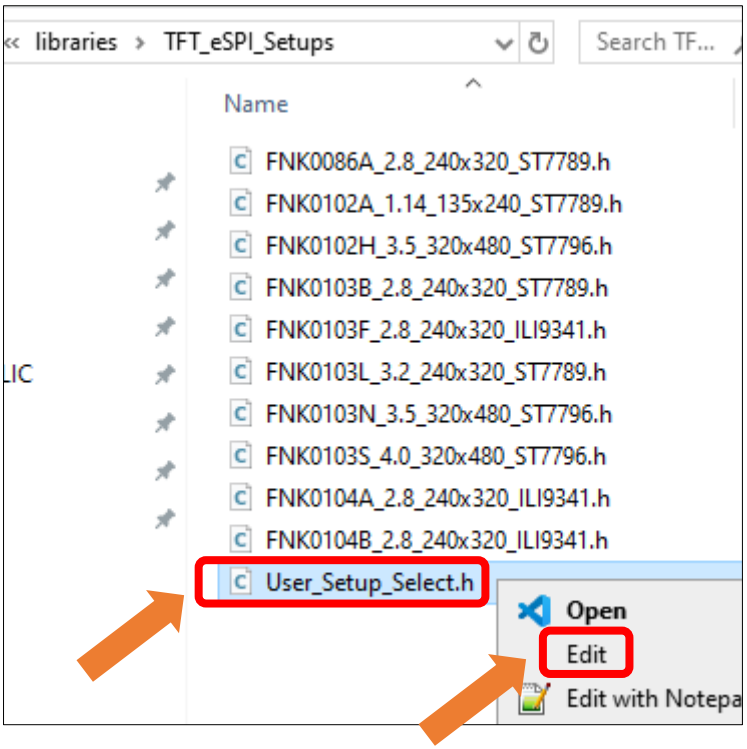


How to configure (Important)

Open This PC, input **%USERPROFILE%\Documents\Arduino\libraries\TFT_eSPI_Setups** and press the **Enter** key.



Right click **User_Setup_Select.h**, click **Edit**.



Uncomment the corresponding macro definition based on the model purchased.
If it is **1.14inch**, configure as below:

```
#define FNK0102A_1P14_135x240_ST7789
//#define FNK0102B_3P5_320x480_ST7796
//#define FNK0103B_2P8_240x320_ST7789
//#define FNK0103F_2P8_240x320_ILI9341
//#define FNK0103L_3P2_240x320_ST7789
//#define FNK0103N_3P5_320x480_ST7796
//#define FNK0103S_4P0_320x480_ST7796
```

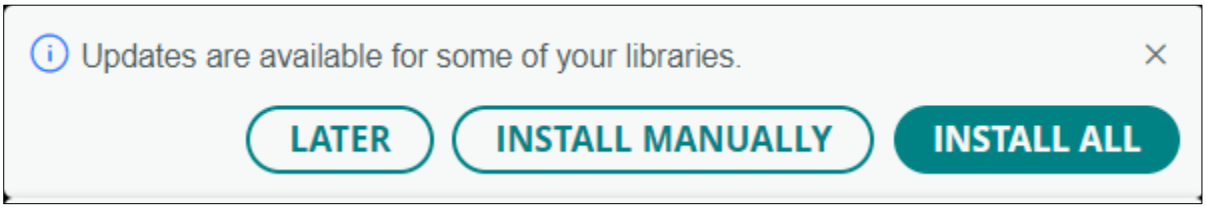
If it is **3.5inch**, configure as below:

```
//#define FNK0086A_2P8_240x320_ST7789
//#define FNK0102A_1P14_135x240_ST7789
#define FNK0102B_3P5_320x480_ST7796
//#define FNK0103B_2P8_240x320_ST7789
//#define FNK0103F_2P8_240x320_ILI9341
//#define FNK0103L_3P2_240x320_ST7789
//#define FNK0103N_3P5_320x480_ST7796
//#define FNK0103S_4P0_320x480_ST7796
```

Important Note: Only one macro definition should be uncommented.
Save the change and exit the file.

Warning:

If the following update prompt appears, click "LATER". Updating the TFT_eSPI library will reset all related configurations. If you click "INSTALL", please reinstall the TFT_eSPI library to ensure proper project operation.



Sketch_10.1_TFT_Rainbow

The following is the program code:

```
#include <TFT_eSPI.h>  // TFT display library

TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance

unsigned long targetTime = 0;      // Timing variable to control animation intervals
byte red = 31;                     // Start with full red
byte green = 0;                    // No green
byte blue = 0;                     // No blue
byte state = 0;                    // State machine variable
unsigned int colour = red << 11;  // Initial color value: Red only

void setup(void) {
   tft.init();                    // Initialize the TFT screen
   tft.setRotation(1);            // Set screen rotation (landscape mode)
   targetTime = millis() + 100;  // Set initial target time for rainbow effect
   tft.fillScreen(TFT_RED);  // Fill screen with solid red
   delay(1000);
   tft.fillScreen(TFT_GREEN);  // Fill screen with solid green
   delay(1000);
   tft.fillScreen(TFT_BLUE);  // Fill screen with solid blue
   delay(1000);
   tft.fillScreen(TFT_BLACK);  // Fill screen with solid black
   delay(1000);
   tft.fillScreen(TFT_WHITE);  // Fill screen with solid white
   delay(1000);
}

void loop() {
   if (targetTime < millis()) {      // Check if it's time to start the rainbow animation
     targetTime = millis() + 10000;  // Set next trigger after 10 seconds


```

```
    for (int i = 0; i < 320; i++) {                    // Generate horizontal rainbow using
vertical lines
      tft.drawFastVLine(i, 0, tft.height(), colour);  // Draw one vertical line
      switch (state) {
        case 0:
          green += 2;  // Transition from red to yellow (increase green)
          if (green == 64) {
            green = 63;  // Cap at max green value
            state = 1;
          }
          break;
        case 1:
          red--;  // Transition from yellow to green (decrease red)
          if (red == 255) {
            red = 0;
            state = 2;
          }
          break;
        case 2:
          blue++;  // Transition from green to cyan (increase blue)
          if (blue == 32) {
            blue = 31;  // Cap at max blue value
            state = 3;
          }
          break;
        case 3:
          green -= 2;  // Transition from cyan to blue (decrease green)
          if (green == 255) {
            green = 0;
            state = 4;
          }
          break;
        case 4:
          red++;  // Transition from blue to magenta (increase red)
          if (red == 32) {
            red = 31;  // Cap at max red value
            state = 5;
          }
          break;
        case 5:
          blue--;  // Transition from magenta to red (decrease blue)
          if (blue == 255) {
            blue = 0;
            state = 0;
```

```
77              }
78              break;
79          }
80          colour = red << 11 | green << 5 | blue;  // Combine RGB values into 16-bit color
81      }
82      tft.setTextColor(TFT_BLACK);           // Set text color to black
83      tft.setCursor(12, 5);                  // Set cursor position
84      tft.print("Original ADAfruit font!");  // Print simple string
85      tft.setTextColor(TFT_BLACK, TFT_BLACK);  // Transparent background
86      tft.drawCentreString("Font size 2", 80, 14, 2);              // Font 2
87      tft.drawCentreString("Font size 4", 80, 30, 4);              // Font 4
88      tft.drawCentreString("12.34", 80, 54, 6);                    // Font 6
89      tft.drawCentreString("12.34 is in font size 6", 80, 92, 2);  // Font 2
90      float pi = 3.14159;                                          // Value to print
91      int precision = 3;                                           // Number of decimal digits
92      int xpos = 50;                                               // X position
93      int ypos = 110;                                              // Y position
94      int font = 2;                                                // Font type
95      xpos += tft.drawFloat(pi, precision, xpos, ypos, font);  // Draw float and update x-
96  position
97      tft.drawString(" is pi", xpos, ypos, font);              // Continue drawing string from
98  updated x position
99      delay(6000);  // Wait before next cycle
100    }
101 }
```

Code Explanation

Include the necessary header file.

```
7   #include <TFT_eSPI.h>  // TFT display library
```

Define TFT display object.

```
9   TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance
```

Initialize the TFT display.

```
19  tft.init();                  // Initialize the TFT screen
20  tft.setRotation(1);          // Set screen rotation (landscape mode)
```

Change the color of the screen in the sequence of red -> green -> blue -> black ->white.

```
22  tft.fillScreen(TFT_RED);  // Fill screen with solid red
23  delay(1000);
24  tft.fillScreen(TFT_GREEN);  // Fill screen with solid green
25  delay(1000);
26  tft.fillScreen(TFT_BLUE);  // Fill screen with solid blue
27  delay(1000);
28  tft.fillScreen(TFT_BLACK);  // Fill screen with solid black
29  delay(1000);
30  tft.fillScreen(TFT_WHITE);  // Fill screen with solid white
31  delay(1000);
```

Implement the rainbow animation effect.

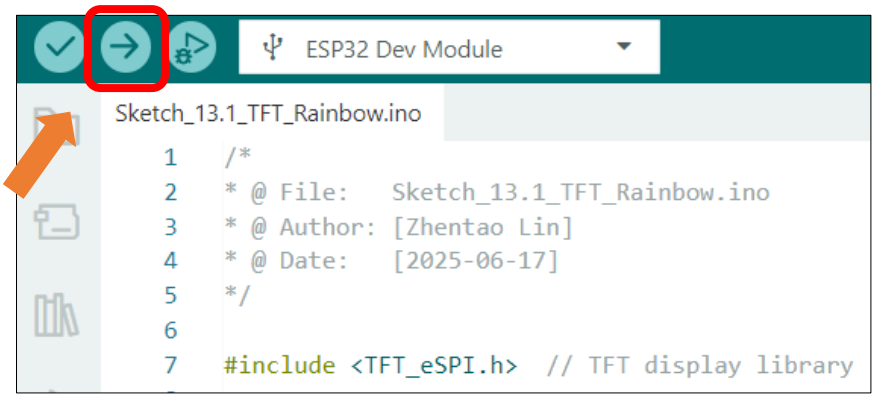
```
for (int i = 0; i < 320; i++) {           // Generate horizontal rainbow using vertical lines
   tft.drawFastVLine(i, 0, tft.height(), colour);  // Draw one vertical line
   switch (state) {
     case 0:
        green += 2;  // Transition from red to yellow (increase green)
        if (green == 64) {
          green = 63;  // Cap at max green value
          state = 1;
        }
       break;
     case 1:
       red--;  // Transition from yellow to green (decrease red)
       if (red == 255) {
         red = 0;
         state = 2;
       }
       break;
     case 2:
       blue++;  // Transition from green to cyan (increase blue)
       if (blue == 32) {
         blue = 31;  // Cap at max blue value
         state = 3;
       }
       break;
     case 3:
       green -= 2;  // Transition from cyan to blue (decrease green)
       if (green == 255) {
         green = 0;
         state = 4;
       }
       break;
     case 4:
       red++;  // Transition from blue to magenta (increase red)
       if (red == 32) {
         red = 31;  // Cap at max red value
         state = 5;
       }
       break;
     case 5:
       blue--;  // Transition from magenta to red (decrease blue)
       if (blue == 255) {
         blue = 0;
         state = 0;
```

```
81          }
82          break;
83      }
84      colour = red << 11 | green << 5 | blue;  // Combine RGB values into 16-bit color
85  }
```

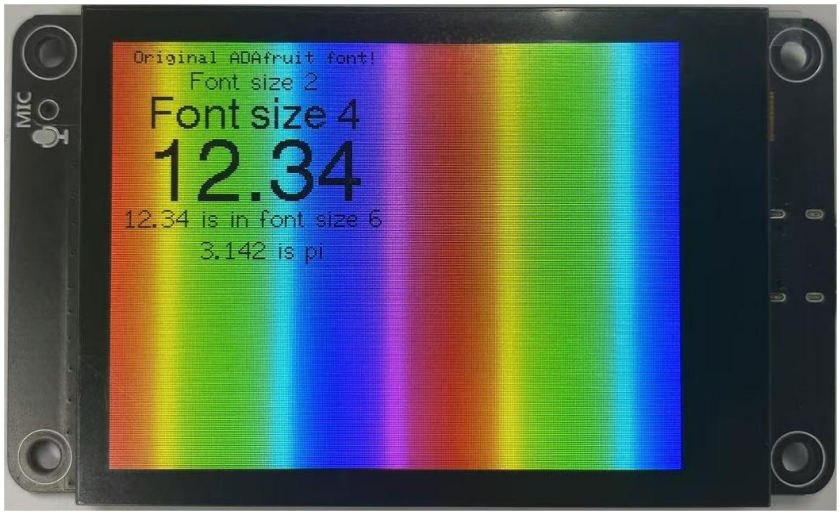
Text display.

```
87  tft.setTextColor(TFT_BLACK);           // Set text color to black
88  tft.setCursor(12, 5);                  // Set cursor position
89  tft.print("Original ADAfruit font!");  // Print simple string
90  tft.setTextColor(TFT_BLACK, TFT_BLACK);  // Transparent background
91  tft.drawCentreString("Font size 2", 80, 14, 2);              // Font 2
92  tft.drawCentreString("Font size 4", 80, 30, 4);              // Font 4
93  tft.drawCentreString("12.34", 80, 54, 6);                    // Font 6
94  tft.drawCentreString("12.34 is in font size 6", 80, 92, 2);  // Font 2
```

Click "**Upload**" to upload the code to Freenove_ESP32_S3_Display.



The TFT screen will change colors in the order of red -> green -> blue -> black -> white before displaying text and rainbow effects.



Project 10.2 Flash JPG DMA

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Component Knowledge

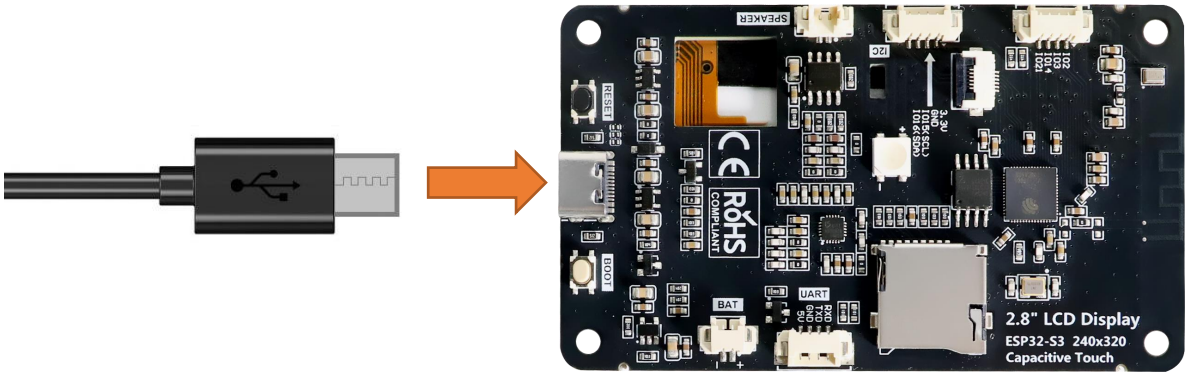
DMA

DMA, or Direct Memory Access, is a hardware feature that allows peripherals to transfer data to and from memory without needing the CPU to be directly involved, dramatically improving overall system efficiency while minimizing processor workload.

The core mechanism of DMA relies on a dedicated DMA controller taking over data transfer tasks. The CPU only needs to initialize the transfer parameters before offloading the operation, allowing computation and I/O operations to proceed in parallel.

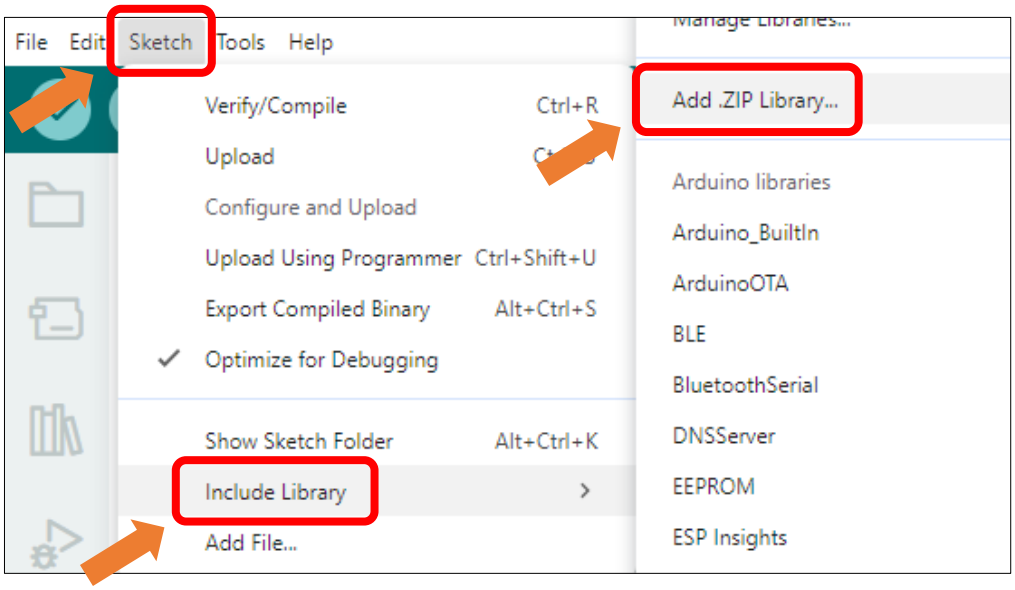
Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.

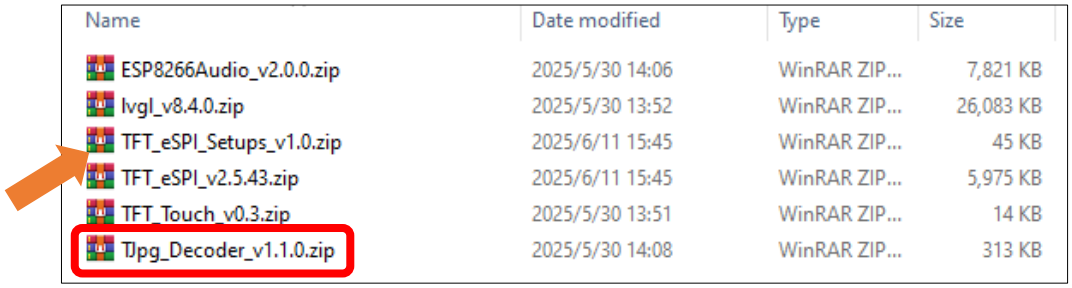


Sketch

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **TJpg_Decoder_v1.1.0.zip**



Open"**Sketch_10.2_Flash_Jpg_DMA**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_10.2_Flash_Jpg_DMA.ino**".

Sketch_10.2_Flash_Jpg_DMA

The following is the program code:

```
#include <TFT_eSPI.h>  // TFT display library
#include "panda.h"         // Include raw JPEG image data array
#include <TJpg_Decoder.h>  // Include JPEG decoder library

TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance

// Callback function to draw decoded JPEG blocks on screen
bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
   if (y >= tft.height()) return 0;  // Stop decoding if off-screen
   tft.pushImage(x, y, w, h, bitmap);  // Draw without DMA
   return 1;  // Continue decoding next block
}

void setup() {
```

```
15      Serial.begin(115200);
16      Serial.println("\n\n Testing TJpg_Decoder library");
17
18      tft.begin();  // Initialize TFT display
19      tft.setTextColor(TFT_WHITE, TFT_BLACK);
20      tft.fillScreen(TFT_BLACK);  // Clear screen with black background
21      tft.setRotation(0);
22
23      TJpgDec.setJpgScale(1);           // Set scale factor (1=full size)
24      tft.setSwapBytes(true);           // Match color byte order
25      TJpgDec.setCallback(tft_output);  // Register drawing callback
26  }
27
28  void loop() {
29      uint16_t w = 0, h = 0;
30      TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset));  // Get image dimensions
31      Serial.print("Width = ");
32      Serial.print(w);
33      Serial.print(", height = ");
34      Serial.print(h);
35
36      uint32_t dt = millis();
37      tft.startWrite();
38      TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset));  // Draw the JPEG image
39      tft.endWrite();
40
41      dt = millis() - dt;
42      Serial.print(", dt = ");
43      Serial.print(dt);
44      Serial.println(" ms");
45
46      delay(2000);  // Wait before redraw
47  }
```

Code Explanation

Include necessary header files.

```
1   #include <TFT_eSPI.h>  // TFT display library
2   #include "panda.h"         // Include raw JPEG image data array
3   #include <TJpg_Decoder.h>  // Include JPEG decoder library
```

Create TFT object instance.

```
19  TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance
```

JPEG decoding callback function

```
12  bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
13      if (y >= tft.height()) return 0;  // Stop decoding if off-screen
14      tft.pushImage(x, y, w, h, bitmap);  // Draw without DMA
```

```
15      return 1;  // Continue decoding next block
16  }
```

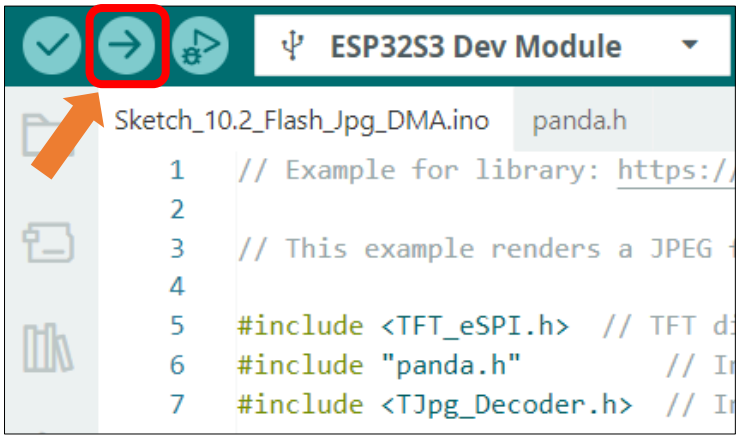
Get JPG size.

```
46  TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset));  // Get image dimensions
```

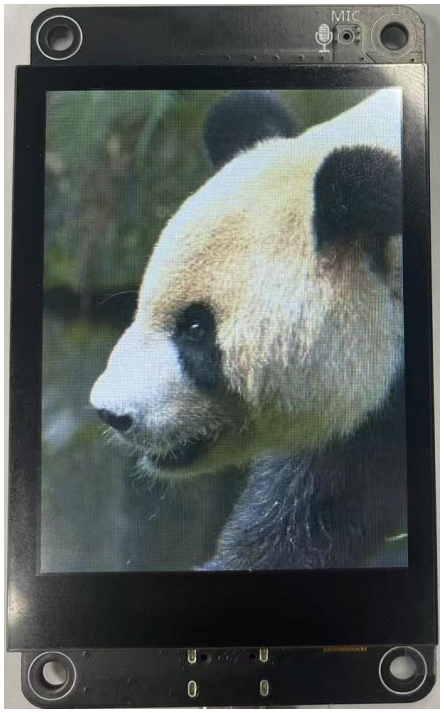
Draw images on the TFT screen.

```
57  tft.startWrite();
58  TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset));  // Draw the JPEG image
59  tft.endWrite();
```

Click "**Upload**" to upload the code to Freenove ESP32 Display



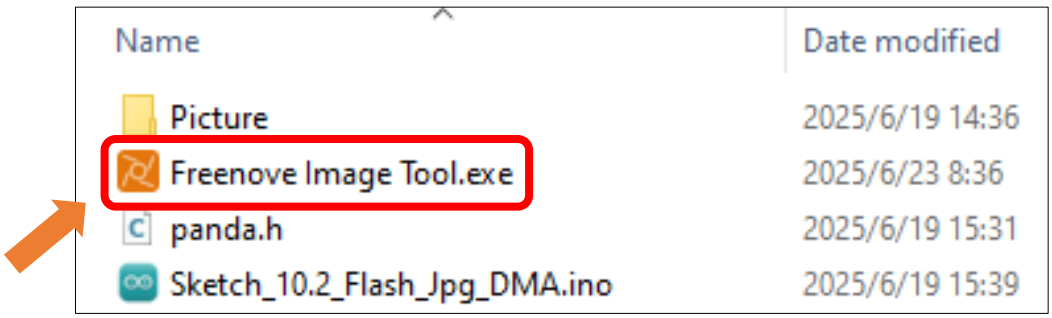
After the code is uploaded, an image will be displayed on the TFT screen. The image is from https://github.com/Bodmer/TJpg_Decoder



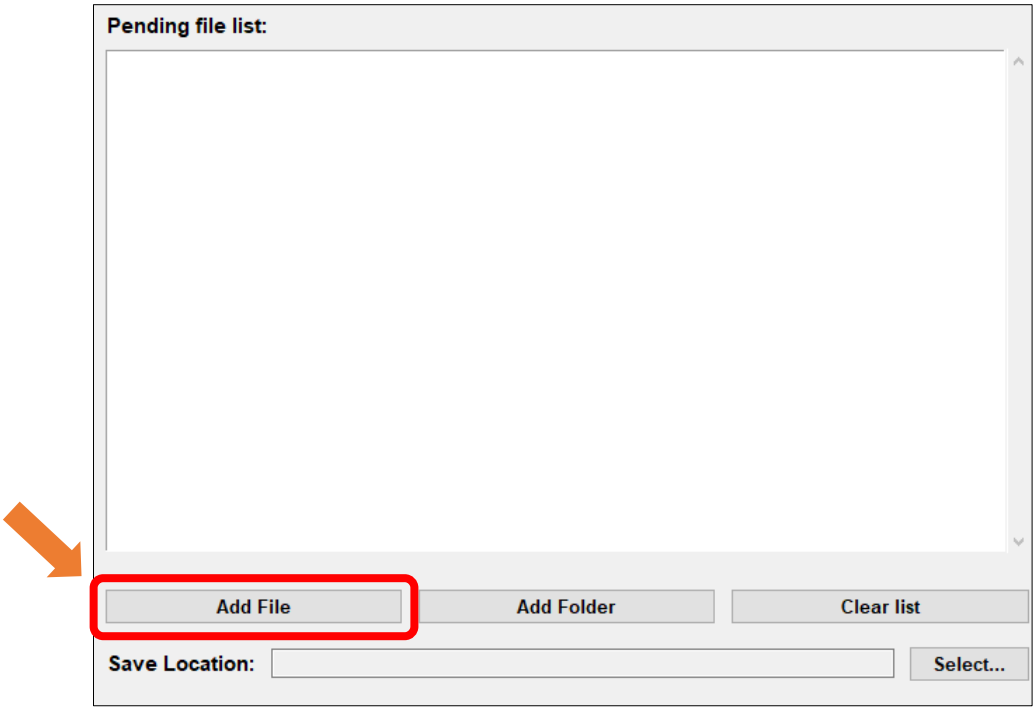
Custom image display

You can customize the image displayed on the display according to your personal preferences.

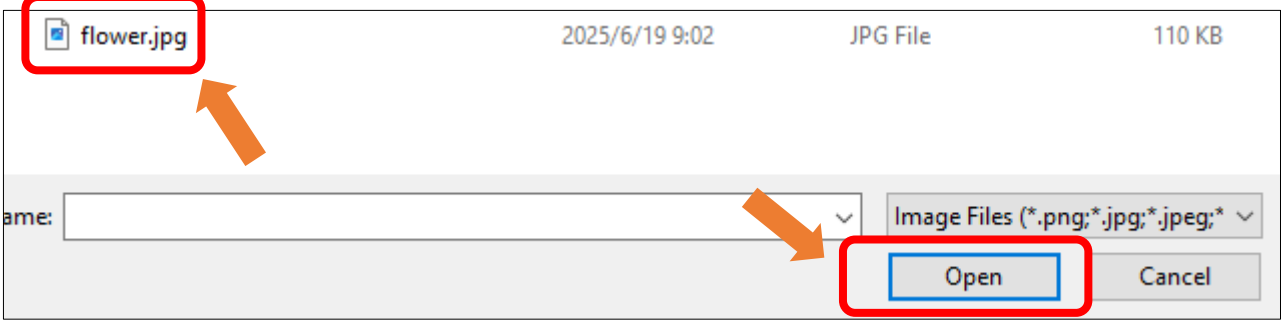
First, open **Freenove_ESP32_S3_Display\Sketches\Sketch_10.2_Flash_Jpg_DMA\Freenove Image Tool.exe**



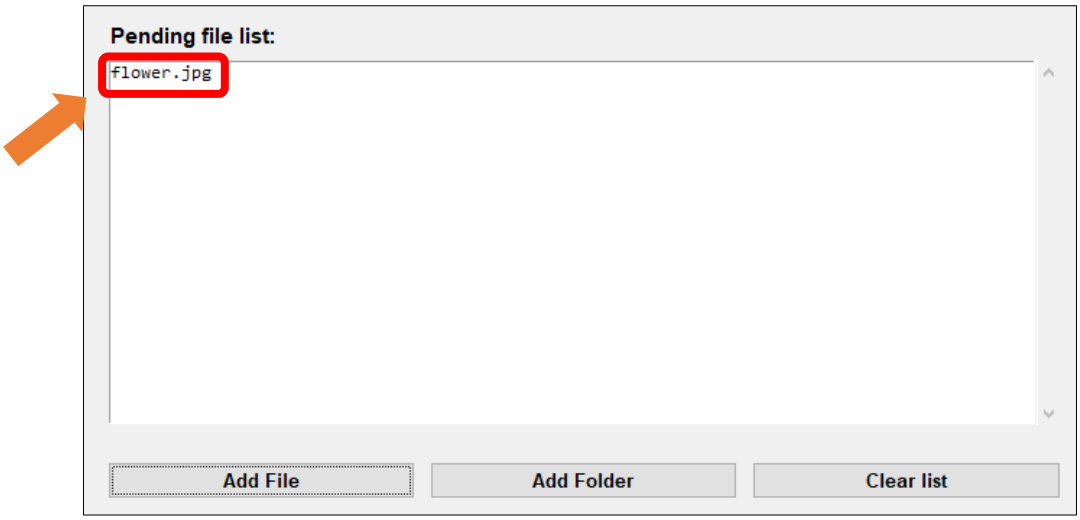
Click **"Add File"**



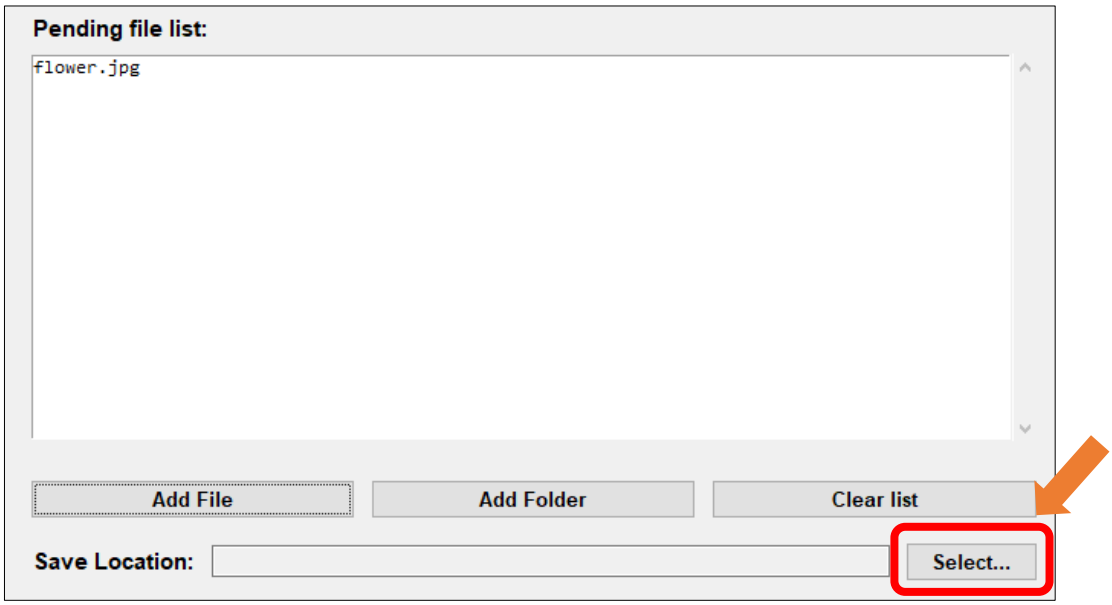
Select any image you like



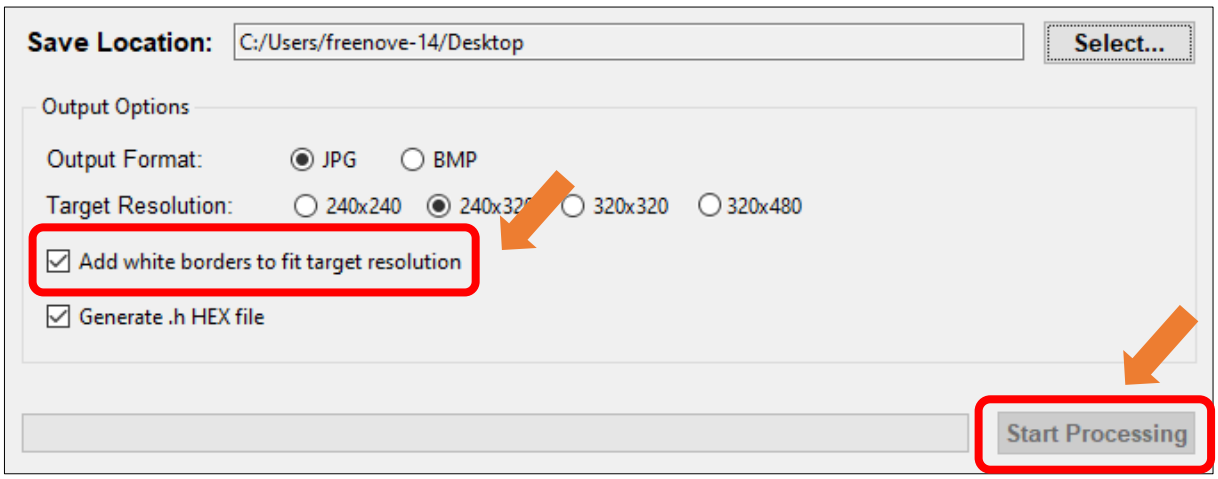
The image files from your folder will now appear in the **Pending File List**.



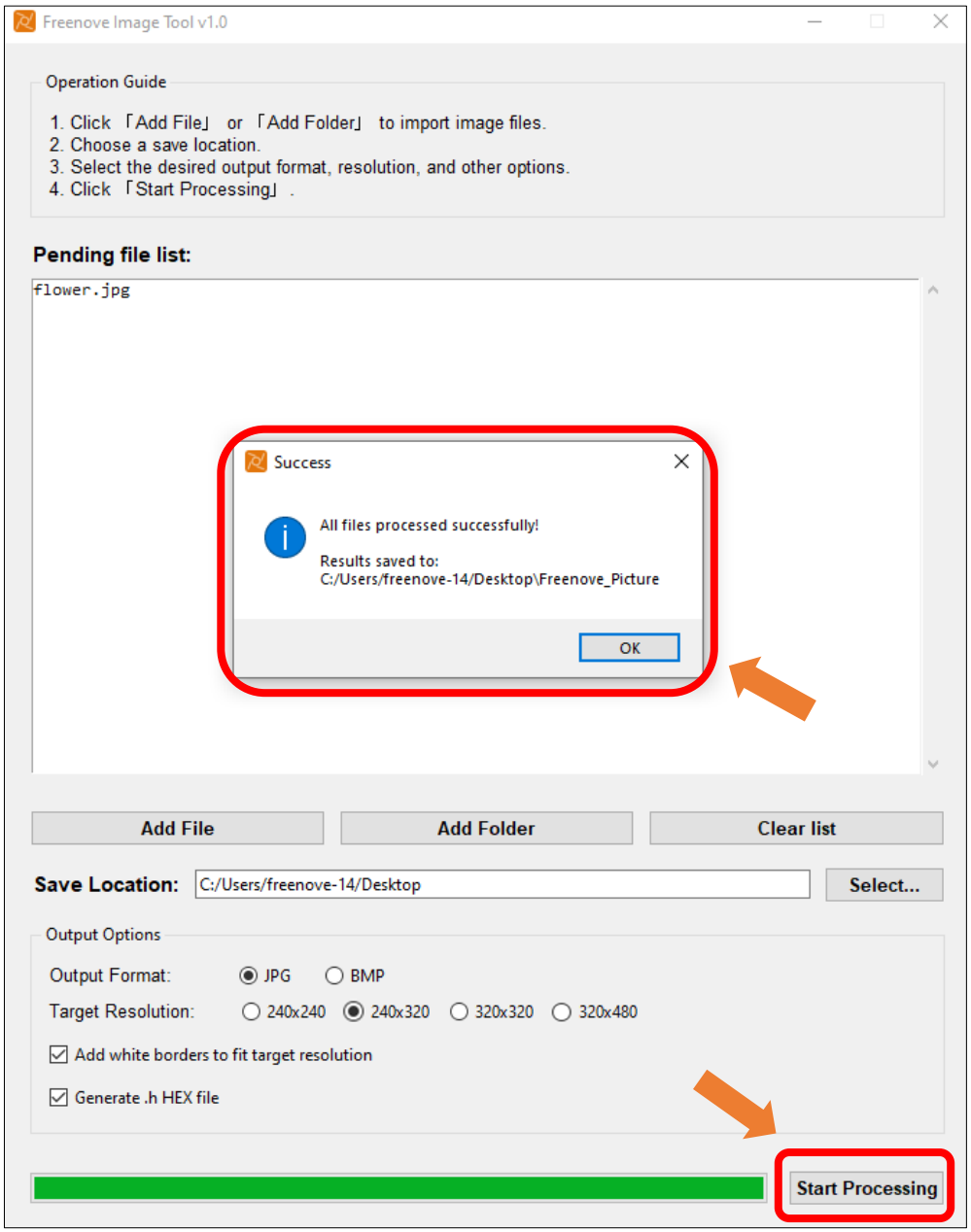
Click "Select…" to change the save location.



The resolution size is selected according to the screen resolution. Check the "**Add white borders to fit target resolution**" and "**Generate .h HEX file**" options.



Click **Start Processing**. Wait for the progress bar to complete and the target folder will be generated.



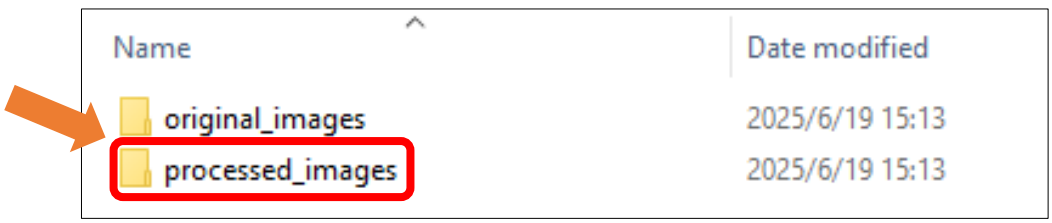
There will be three folders in the generated folder

original_images: backup of images before processing

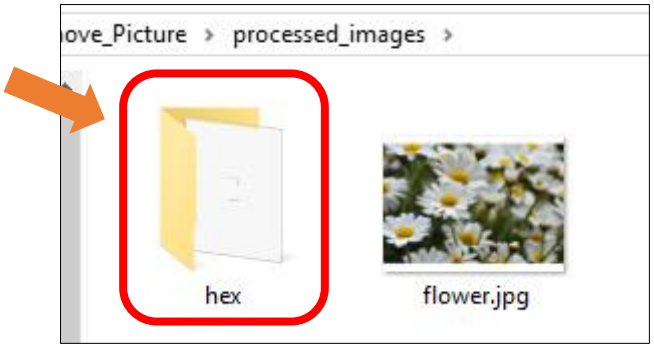
processed_images: processed images

processed_images\hex: generated .h files corresponding to the images

Double click to open **processed_images**



Double click to open **hex**



Replace the entire content of panda.h with the copied data from the .h file, then upload the sketch again.

Sketch_10.2_Flash_Jpg_DMA.ino | panda.h

```
/*
    If you need to customize the displayed image, first use the Freenove Image Tool
    to crop it to the appropriate width and height for your display.

    The tool will automatically adjust the image quality to ensure proper display.

    Paste the generated array into this file, ensuring the correct array format:

        const uint8_t name[] PROGMEM = {

    to start and and end with:

    };
*/

const unsigned char img_asset[] PROGMEM = {
    0xFF, 0xD8, 0xFF, 0xE0, 0x00, 0x10, 0x4A, 0x46, 0x49, 0x46, 0x00, 0x01,
    0x01, 0x00, 0x00, 0x01, 0x00, 0x01, 0x00, 0x00, 0xFF, 0xDB, 0x00, 0x43,
    0x00, 0x05, 0x03, 0x04, 0x04, 0x04, 0x03, 0x05, 0x04, 0x04, 0x04, 0x05,
    0x05, 0x05, 0x06, 0x07, 0x0C, 0x08, 0x07, 0x07, 0x07, 0x07, 0x0F, 0x0B,
    0x0B, 0x09, 0x0C, 0x11, 0x0F, 0x12, 0x12, 0x11, 0x0F, 0x11, 0x11, 0x13,
    0x16, 0x1C, 0x17, 0x13, 0x14, 0x1A, 0x15, 0x11, 0x11, 0x18, 0x21, 0x18,
    0x1A, 0x1D, 0x1D, 0x1F, 0x1F, 0x1F, 0x13, 0x17, 0x22, 0x24, 0x22, 0x1E,
    0x24, 0x1C, 0x1E, 0x1F, 0x1E, 0xFF, 0xDB, 0x00, 0x43, 0x01, 0x05, 0x05,
    0x05, 0x07, 0x06, 0x07, 0x0E, 0x08, 0x08, 0x0E, 0x1E, 0x14, 0x11, 0x14,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
    0x1E, 0x1E, 0xFF, 0xC0, 0x00, 0x11, 0x08, 0x01, 0x40, 0x01, 0xE0, 0x03,
    0x01, 0x22, 0x00, 0x02, 0x11, 0x01, 0x03, 0x11, 0x01, 0xFF, 0xC4, 0x00,
    0x1C, 0x00, 0x00, 0x02, 0x02, 0x03, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00,
```

The image will display on the screen.

Note:

To adjust the image display orientation, modify the following code in Sketch_10.2_Flash_Jpg_DMA.ino:

```
42  tft.setRotation(uint8_t rotation);
```

Value	Rotation Angle	Description
0	0°	Default orientation (no rotation)
1	90°	Clockwise 90-degree rotation
2	180°	Clockwise 180-degree rotation
3	270°	Clockwise 270-degree rotation

Chapter 11 TFT Touch

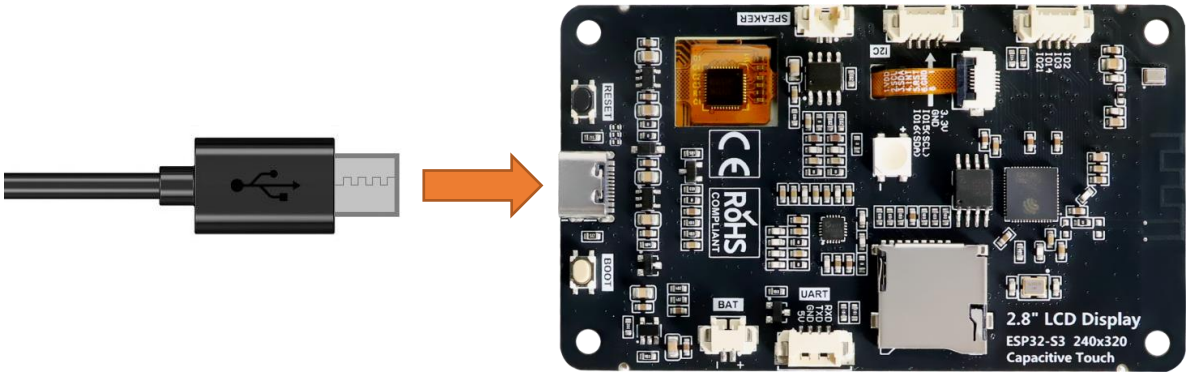
Project 11.1 TFT Touch

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.

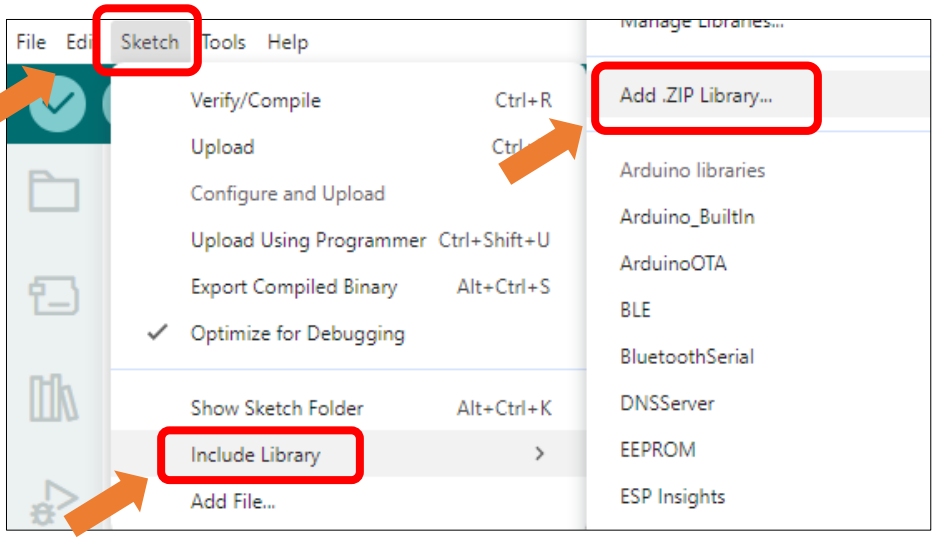


Sketch

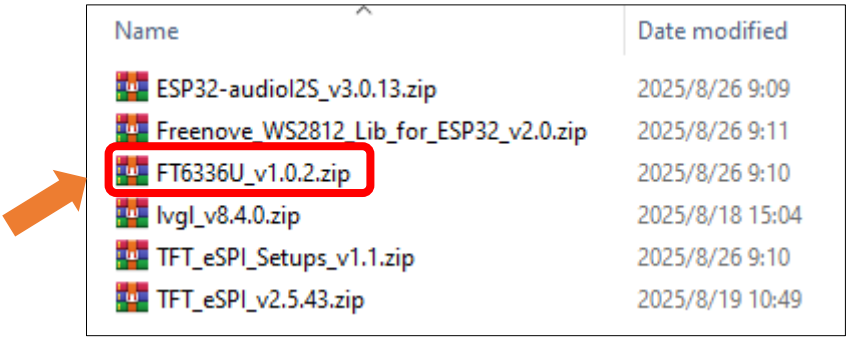
Open“**Sketch_11.1_Touch**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_11.1_Touch.ino**”.

Install the needed libraries.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **ESP32-audioI2S_v3.0.13.zip**



Next, we download the code to Freenove_ESP32_S3_Display to test. Open“**Sketch_11_Touch**” folder under **“Freenove_ESP32_S3_Display\Sketcheses”** and double-click “**Sketch_11.1_Touch.ino**”.

Sketch_11.1_Touch

The following is the program code:

```
#include "FT6336U.h"

#define I2C_SDA 16
#define I2C_SCL 15
#define RST_N_PIN 18
#define INT_N_PIN 17

FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
FT6336U_TouchPointType tp;

```

```
11  void setup() {
12      Serial.begin(115200);
13      ft6336u.begin();
14      Serial.print("FT6336U Firmware Version: ");
15      Serial.println(ft6336u.read_firmware_id());
16      Serial.print("FT6336U Device Mode: ");
17      Serial.println(ft6336u.read_device_mode());
18  }
19
20  void loop() {
21      tp = ft6336u.scan();
22      char tempString[128];
23      sprintf(tempString, "FT6336U TD Count %d / TD1 (%d, %4d, %4d) / TD2 (%d, %4d, %4d)\r\n",
24  tp.touch_count, tp.tp[0].status, tp.tp[0].x, tp.tp[0].y, tp.tp[1].status, tp.tp[1].x,
25  tp.tp[1].y);
26      Serial.print(tempString);
27      delay(100);
28  }
```

Code Explanation

Include the necessary header files.

```
1  #include "FT6336U.h"
```

Initialize the screen.

```
8  FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
9  FT6336U_TouchPointType tp;
```

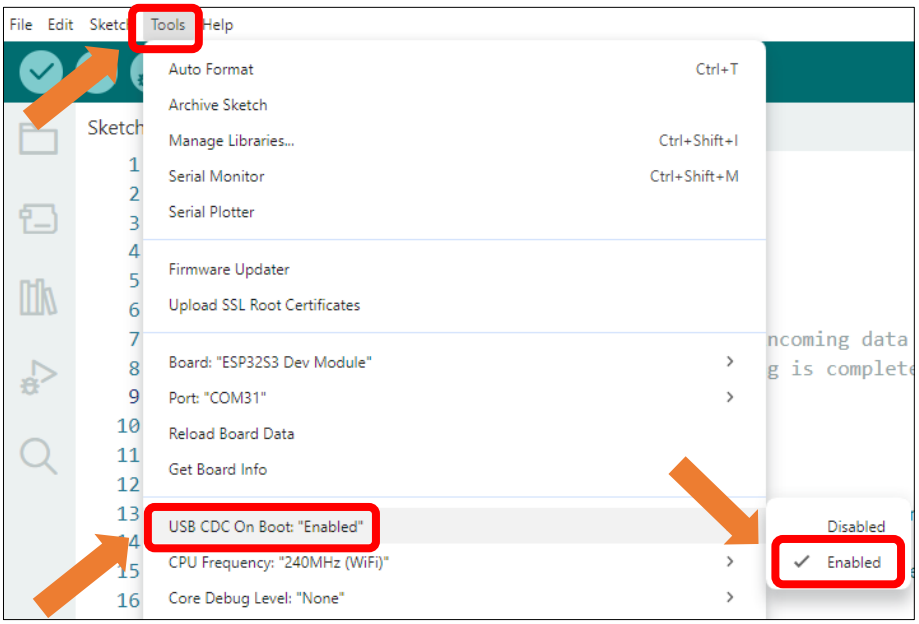
Set the baud rate to 115200.

```
19  Serial.begin(115200);
```

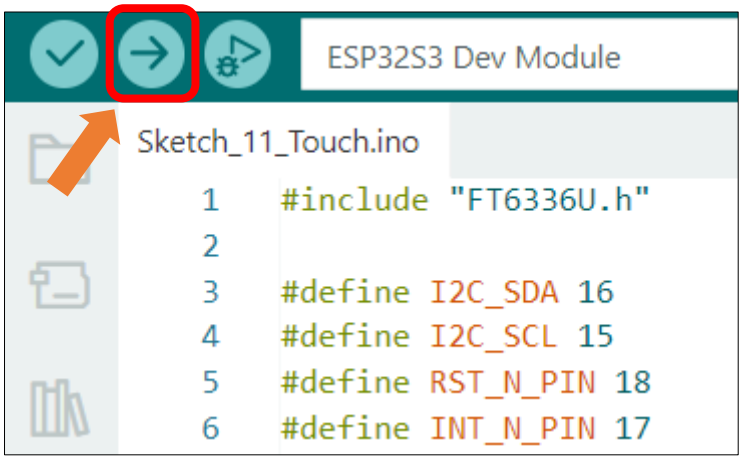
Scan touch point data and output it via the serial port.

```
21  tp = ft6336u.scan();
22  char tempString[128];
23  sprintf(tempString, "FT6336U TD Count %d / TD1 (%d, %4d, %4d) / TD2 (%d, %4d, %4d)\r\n",
24  tp.touch_count, tp.tp[0].status, tp.tp[0].x, tp.tp[0].y, tp.tp[1].status, tp.tp[1].x,
25  tp.tp[1].y);
26  Serial.print(tempString);
27  delay(100);
```

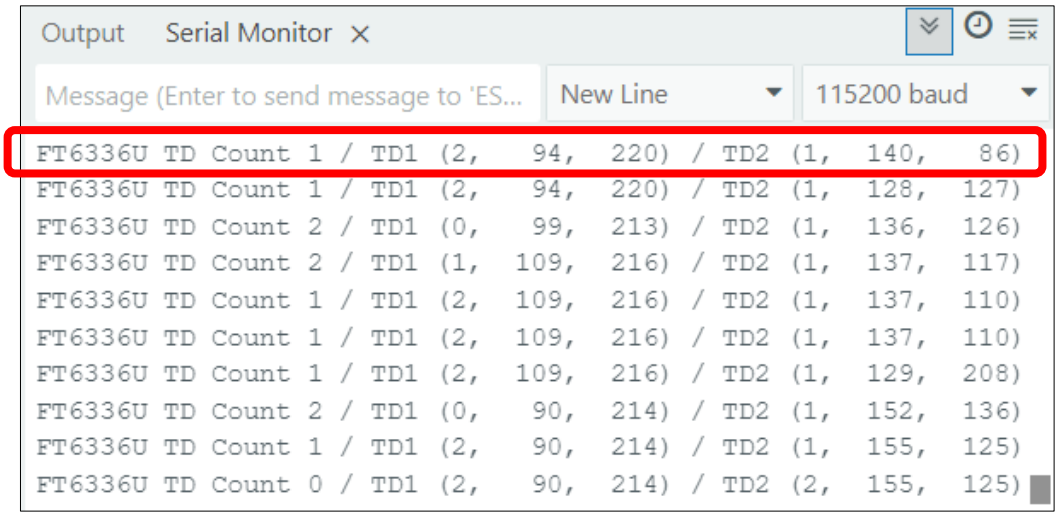
Enable the "USB CDC On Boot" feature.



Click "**Upload**" to upload the code to Freenove ESP32 Display, set the baud rate to 115200.



When the touch screen is pressed, the serial monitor will output the coordinates of the touch point(s) in real time. TD1 corresponds to the first touch point and will always be displayed. If a second touch point is detected simultaneously, its coordinates will be output via TD2. The specific format of the output can be seen in the image below.



Chapter 12 TFT Touch Drawing

After learning this chapter, you will be able to draw freely on the screen.

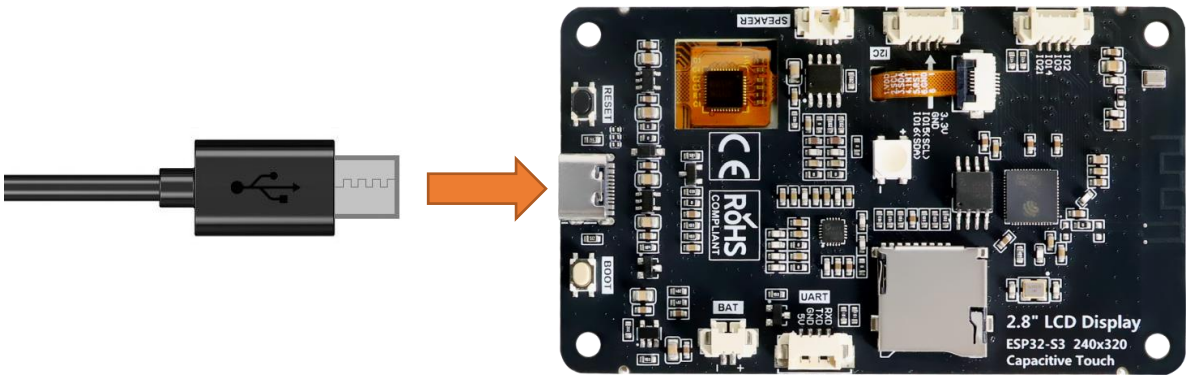
Project 12.1 TFT Touch Drawing

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Open "**Sketch_12.1_TFT_Touch_Draw**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_12.1_TFT_Touch_Draw.ino**".

Sketch_12.1_TFT_Touch_Draw

The following is the program code:

```
1  #include <TFT_eSPI.h>  // TFT display library
2  #include "FT6336U.h"
3
```

```
// Invoke custom TFT driver library
TFT_eSPI tft = TFT_eSPI();       // Invoke custom library

#define SCREEN_WIDTH 240
#define SCREEN_HEIGHT 320

#define I2C_SDA 16
#define I2C_SCL 15
#define RST_N_PIN 18
#define INT_N_PIN 17

/* Create an instance of the touch screen library */
FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
FT6336U_TouchPointType tp;

// Define coordinate variables
uint16_t x, y;
uint16_t last_x, last_y;

const uint8_t ColorPalette = 21;      // Height of the color palette area
const uint8_t BrushSize = 2;          // Brush thickness (radius)
const uint8_t LineSize = 5;           // Line thickness (radius)
const uint8_t color_num = 12;
int color = TFT_WHITE;

unsigned int colors[color_num] = { TFT_RED, TFT_PINK, TFT_GREEN, TFT_BLUE, TFT_OLIVE ,
TFT_CYAN, TFT_YELLOW, TFT_WHITE, TFT_MAGENTA, TFT_ORANGE, TFT_BLACK, TFT_SILVER};

void drawUI()
{
   // Draw color palette
   for (int i = 0; i < color_num; i++) {
     tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
   }

   // Draw clear screen button
   tft.setCursor(260 , 3, 2);
   tft.setTextColor(TFT_WHITE);
   tft.print("Clear");

   // Display current color
   tft.fillRect(300, 5, 12, 12, color);

   // Add palette border/outline
```

```
48      tft.drawRect(0, 0, SCREEN_HEIGHT, ColorPalette, TFT_WHITE);
49  }
50
51  void setup()
52  {
53      Serial.begin(115200);
54      ft6336u.begin();
55      Serial.print("FT6336U Firmware Version: ");
56      Serial.println(ft6336u.read_firmware_id());
57      Serial.print("FT6336U Device Mode: ");
58      Serial.println(ft6336u.read_device_mode());
59      tft.init();
60
61      // Set the TFT and touch screen to landscape orientation
62      tft.setRotation(1);
63
64      tft.fillScreen(TFT_BLACK);
65
66      drawUI();
67  }
68
69  /* Main program */
70  void loop()
71  {
72      // Check if the touch screen is currently pressed
73      // Raw and coordinate values are stored within library at this instant
74      // for later retrieval by GetRaw and GetCoord functions.
75      tp = ft6336u.scan();
76      if (tp.touch_count!=0) // Note this function updates coordinates stored within library
77  variables
78      {
79        // Read the current X and Y axis as co-ordinates at the last touch time
80        // The values were captured when Pressed() was called!
81        x = tp.tp[0].y;
82        y = 240-tp.tp[0].x;
83
84        Serial.print(x); Serial.print(","); Serial.println(y);
85
86        /* Color palette area logic */
87        if(y < ColorPalette + 3)
88        {
89          // Enter color selection area, stroke interrupted
90          last_x = 0;
91          last_y = 0;
```

```
92
93          /* Determine stylus click area */
94          // Clicked Clear button
95          if(x >= ColorPalette * color_num)
96          {
97            tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
98          }
99          // Clicked color palette
100         else{
101           // Calculate current color based on x-coordinate
102           color = colors[x / ColorPalette];
103
104           // Update display of current color
105           tft.fillRect(300, 5, 12, 12, color);
106         }
107         delay(5);
108         return;
109       }
110
111       /* Drawing logic */
112       // Start of a new stroke
113       if(last_x == 0 && last_y == 0)
114       {
115         // Record current position
116         last_x = x;
117         last_y = y;
118       }
119
120       // Draw a line between the previous position and current position
121       tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
122
123       // Update coordinates
124       last_x = x;
125       last_y = y;
126     }
127     else{
128       // Reset coordinates when stylus leaves the screen
129       last_x = 0;
130       last_y = 0;
131     }
132 }
```

Code Explanation

Include the necessary header files.

```
#include <TFT_eSPI.h>  // TFT display library
#include "FT6336U.h"
```

Create TFT object instance.

```
// Invoke custom TFT driver library
TFT_eSPI tft = TFT_eSPI();       // Invoke custom library
```

Define screen resolution.

```
#define SCREEN_WIDTH 240
#define SCREEN_HEIGHT 320
```

Draw color palette.

```
// Draw color palette
for (int i = 0; i < color_num; i++) {
    tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
}
```

Detect whether the screen is pressed.

```
if (tp.touch_count!=0)
```

Store the x and y coordinates of the touch point in variables.

```
x = tp.tp[0].y;
y = 240-tp.tp[0].x;
```

When the Clear button is pressed, fill the screen with the current color.

```
// Clicked Clear button
if(x >= ColorPalette * color_num)
{
  tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
}
```

Calculate the selected color based on the coordinates of the touch point

```
// Clicked color palette
else{
  // Calculate current color based on x-coordinate
  color = colors[x / ColorPalette];

  // Update display of current color
  tft.fillRect(300, 5, 12, 12, color);
}
```

Draw a line based on coordinates

```
if(last_x == 0 && last_y == 0)
{
  // Record current position
  last_x = x;
  last_y = y;
}

```

```
97  // Draw a line between the previous position and current position
98  tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
99
100 // Update coordinates
101 last_x = x;
102 last_y = y;
```

Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to115200

Tips: Aliasing & Anti-aliasing

In computer graphics, **aliasing** refers to the **jagged or stair-step appearance** of lines and curves in digital images, particularly noticeable on diagonal lines and edges. This occurs due to the discrete nature of pixel grids - screens compose images from tiny square pixels that cannot perfectly represent continuous geometry. **Anti-aliasing** mitigates these artifacts through technical means to **smooth edges**, achieving more natural-looking graphics. The core technique blends transitional colors at boundary pixels, simulating human visual perception of soft edges.

Chapter 13 LVGL

Project 13.1 LVGL

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

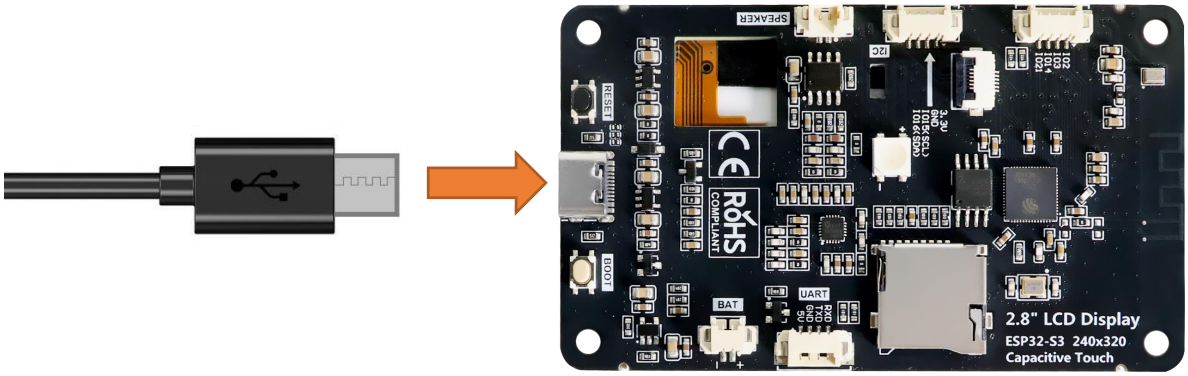
Component Knowledge

LVGL is a widely-used embedded GUI library that is implemented in pure C, making it highly portable and performant. It offers rich features and content, supporting both display and input devices such as touchscreens and keyboards.

	Features supported by LVGL
1	Powerful building blocks such as buttons, charts, lists, sliders, images, and more.
2	Advanced graphics with animation, anti-aliasing, opacity, and smooth scrolling.
3	Various input devices, such as touchpads, mice, keyboards, encoders, and more.
4	Multiple languages with UTF-8 encoding.
5	Multiple display types, including TFT and monochrome displays.
6	Fully customizable graphical elements.
7	LVGL can be used independently of any microcontroller or display hardware.
8	Highly extensible and can be configured to use very little memory (e.g. 64 kB of flash and 16 kB of RAM)
9	It can be used with or without an operating system, and supports external memory and GPUs as optional features.
10	Single-frame buffer operation, even with advanced graphics effects.
11	Written in C language to achieve maximum compatibility (compatible with C++ as well).
12	LVGL has a simulator that allows for embedded GUI design on a PC without any embedded hardware.
13	Resources to help developers quickly get started with the library, including tutorials, examples, and themes.
14	A wide range of resources.

Circuit

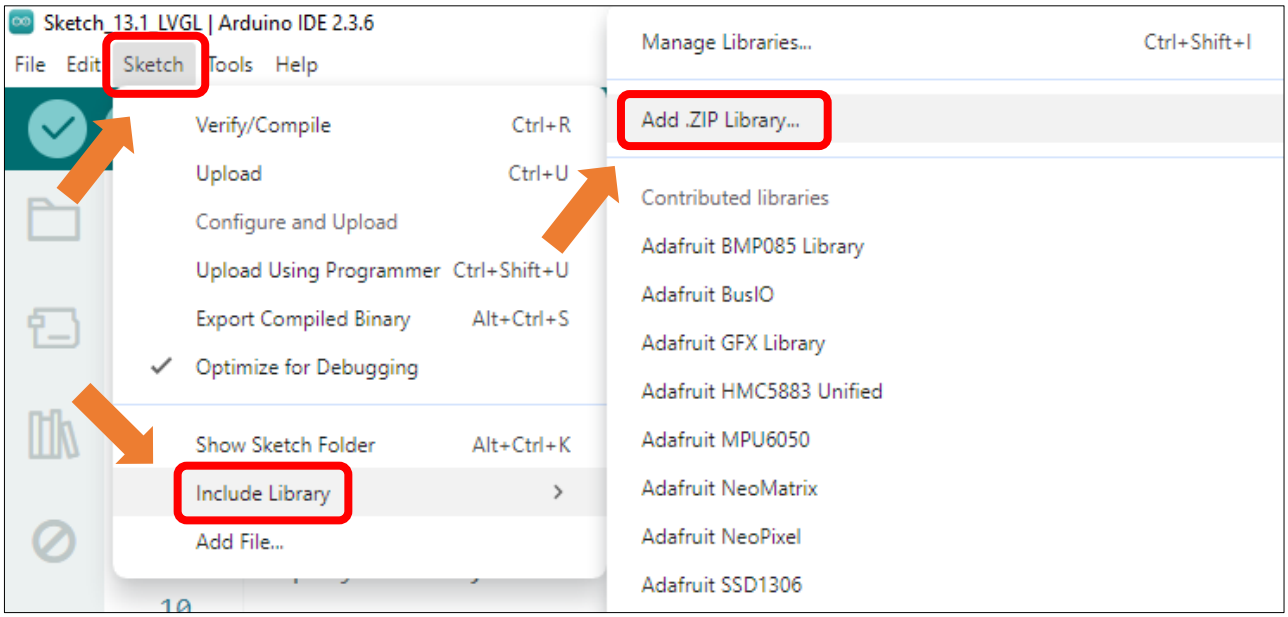
Connect Freenove ESP32 -S3 to the computer using the USB cable.



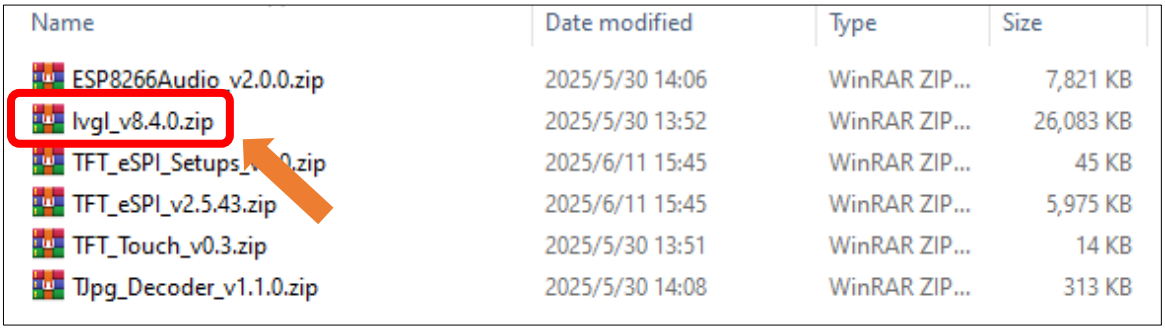
Sketch

Install Libraries

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **lvgl_v8.4.0.zip**



Open "**Sketch_13.1_LVGL**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_13.1_LVGL.ino**".

Sketch_13.1_LVGL

The following is the program code:

```
#include "display.h"

Display screen;

void setup()
{
    /* prepare for possible serial debug */
    Serial.begin( 115200 );

    /*** Init screen ***/
    screen.init();

    String LVGL_Arduino = "Hello Arduino! ";
    LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
lv_version_patch();

    Serial.println( LVGL_Arduino );
    Serial.println( "I am LVGL_Arduino" );

    /* Create simple label */
    lv_obj_t *label = lv_label_create( lv_scr_act() );
    lv_label_set_text( label, LVGL_Arduino.c_str() );
    lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );

    Serial.println( "Setup done" );
}

void loop()
{
    screen.routine(); /* let the GUI do its work */
    delay( 5 );
}
```

Code Explanation

Include the header file.

```
#include "display.h"
```

Set the baud rate to 115200

```
Serial.begin( 115200 );
```

Initialize the screen.

```
screen.init();
```

Configure the screen interface.

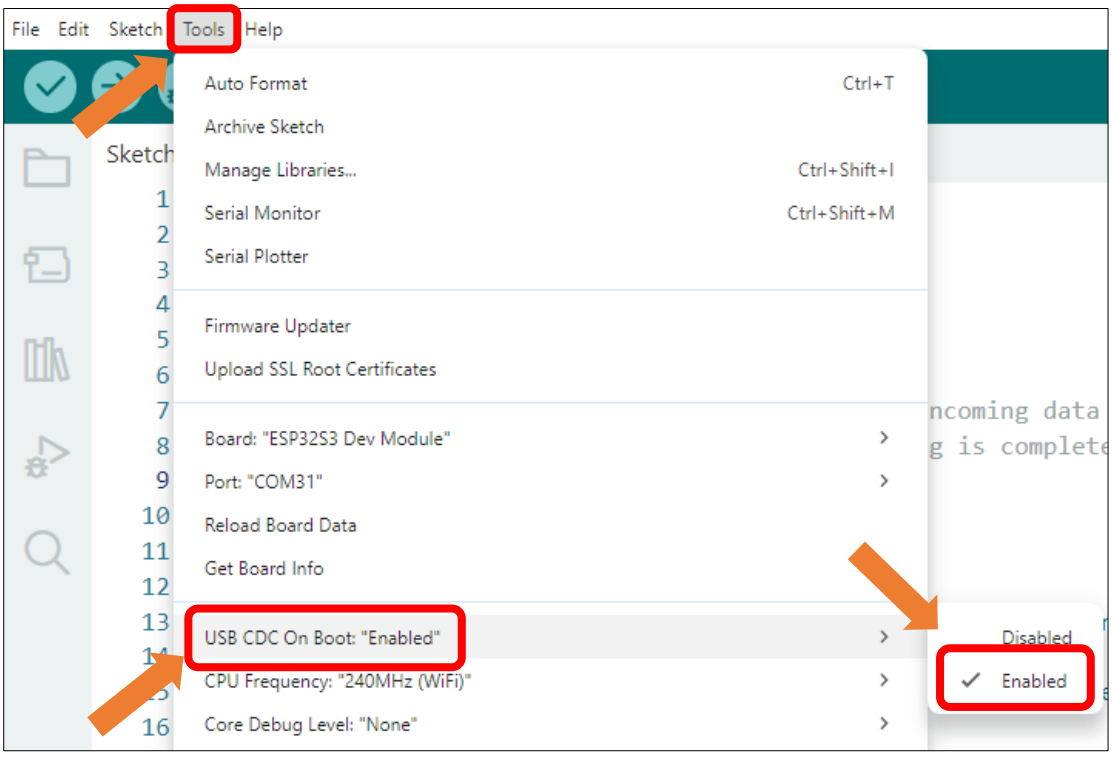
```
String LVGL_Arduino = "Hello Arduino! ";

```

```
17  LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
18  lv_version_patch();
19
20  Serial.println( LVGL_Arduino );
21  Serial.println( "I am LVGL_Arduino" );
22
23  /* Create simple label */
24  lv_obj_t *label = lv_label_create( lv_scr_act() );
25  lv_label_set_text( label, LVGL_Arduino.c_str() );
    lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );
```

Let the LVGL handle the tasks.

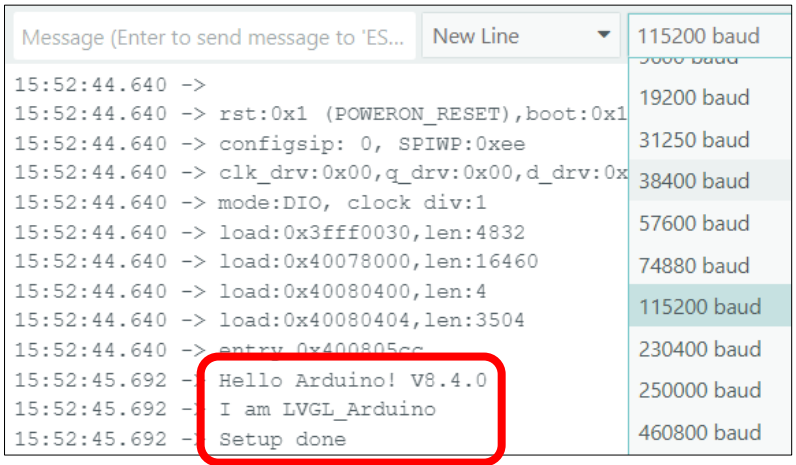
```
46  screen.routine(); /* let the GUI do its work */
```

Enable the "USB CDC On Boot" feature.

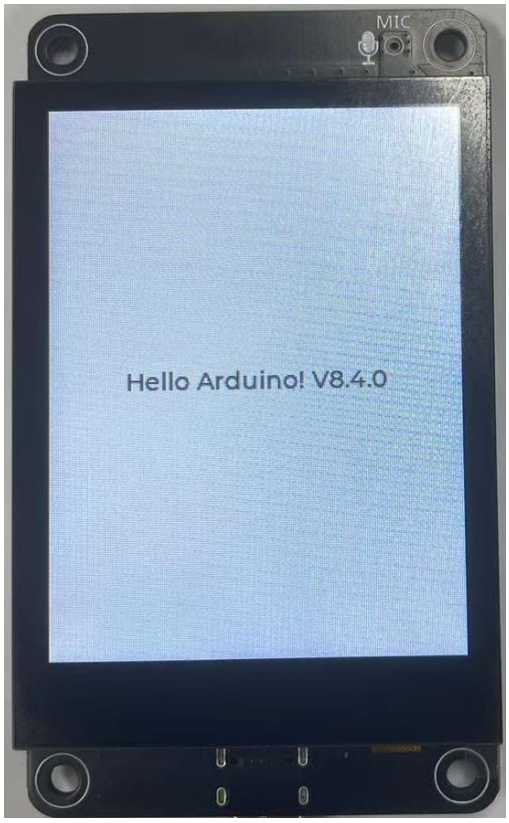


Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.

Data will be printed on the serial monitor.



The text "**Hello Arduino! V8.4.0**" will be displayed on the screen.



Chapter 14 LVGL Picture

Project 14.1 LVGL Picture

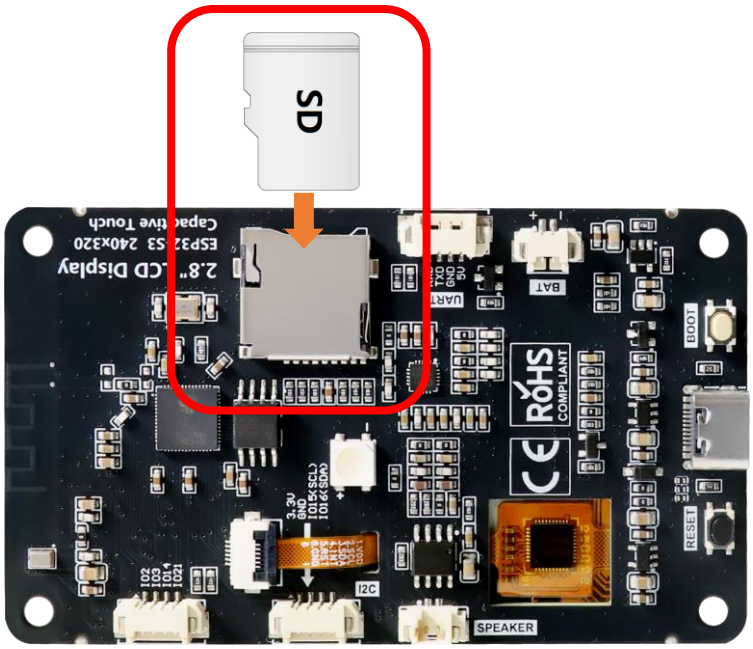
Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

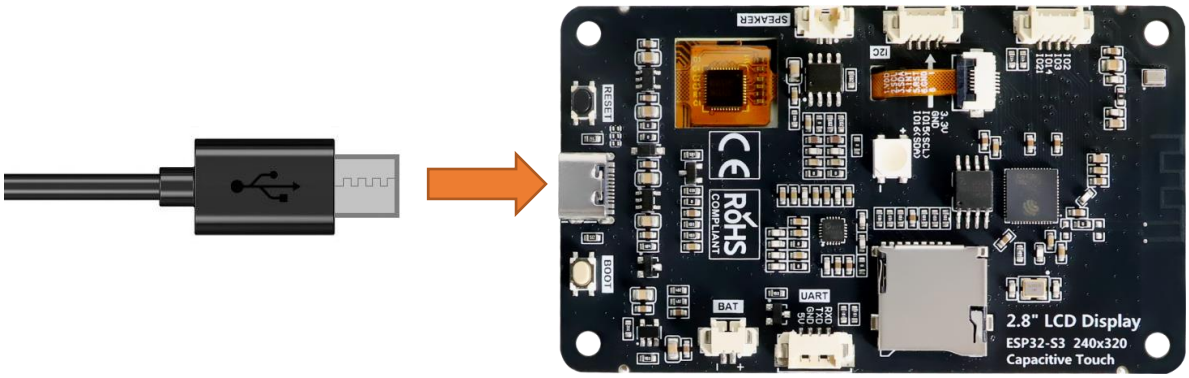
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.

Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect Freenove ESP32-S3 to the computer using the USB cable.



If you have any concerns, please feel free to contact us via support@freenove.com

Sketch

Open "**Sketch_14.1_Lvgl_Picture**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_14.1_Lvgl_Picture.ino**".

Sketch_14.1_Lvgl_Picture

The following is the program code:

```
#include "display.h"
#include "picture_ui.h"
#include "driver_sdmmc.h"

#define SD_MMC_CMD 40  // Please do not modify it.
#define SD_MMC_CLK 38  // Please do not modify it.
#define SD_MMC_D0  39  // Please do not modify it.
#define SD_MMC_D1  41  // Please do not modify it.
#define SD_MMC_D2  48  // Please do not modify it.
#define SD_MMC_D3  47  // Please do not modify it.

Display screen;  // Create an instance of the Display class

void setup() {
    /* Prepare for possible serial debug */
    Serial.begin(115200);  // Initialize serial communication at 115200 baud rate

    /* Initialize SD card */
    sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);
    create_dir(PICTURE_FOLDER);

    /*** Initialize the screen ***/
    screen.init();  // Initialize the display

    // Create a string to display LVGL version information
```

```
26      String LVGL_Arduino = "Hello Arduino! ";
27      LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
28  lv_version_patch();
29
30      // Print LVGL version information to the serial monitor
31      Serial.println(LVGL_Arduino);
32      Serial.println("I am LVGL_Arduino");
33
34      setup_scr_picture(&guider_picture_ui);
35      lv_scr_load(guider_picture_ui.picture);
36
37      // Print setup completion message to the serial monitor
38      Serial.println("Setup done");
39  }
40
41  void loop() {
42      screen.routine(); /* Let the GUI do its work */  // Handle routine display tasks
43      delay(5);                                        // Add a small delay to prevent the loop
44  from running too fast
45  }
```

Code Explanation

Include the header files.

```
13  #include "display.h"
14  #include "driver_sdspi.h"
15  #include "picture_ui.h"
```

Define the pins.

```
17  #define SD_MMC_CMD 40  // Please do not modify it.
18  #define SD_MMC_CLK 38  // Please do not modify it.
19  #define SD_MMC_D0  39  // Please do not modify it.
20  #define SD_MMC_D1  41  // Please do not modify it.
21  #define SD_MMC_D2  48  // Please do not modify it.
22  #define SD_MMC_D3  47  // Please do not modify it.
```

Set the baud rate to 115200

```
28  Serial.begin( 115200 );
```

Initialize configuration.

```
31  sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);
32  create_dir(PICTURE_FOLDER);
```

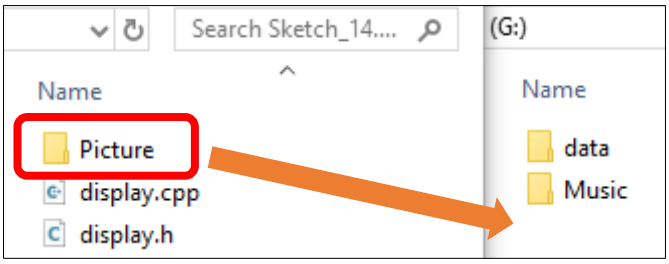
Create and load the interface.

```
46  setup_scr_picture(&guider_picture_ui);
47  lv_scr_load(guider_picture_ui.picture);
```

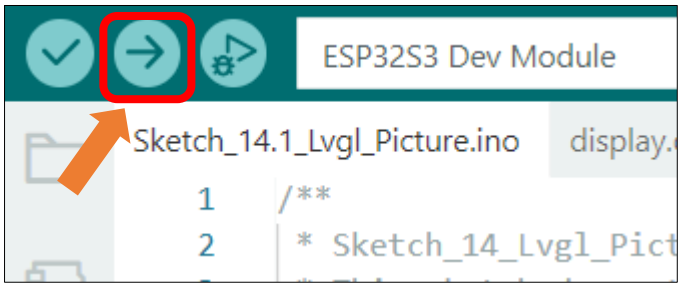
LVGL Task Processor

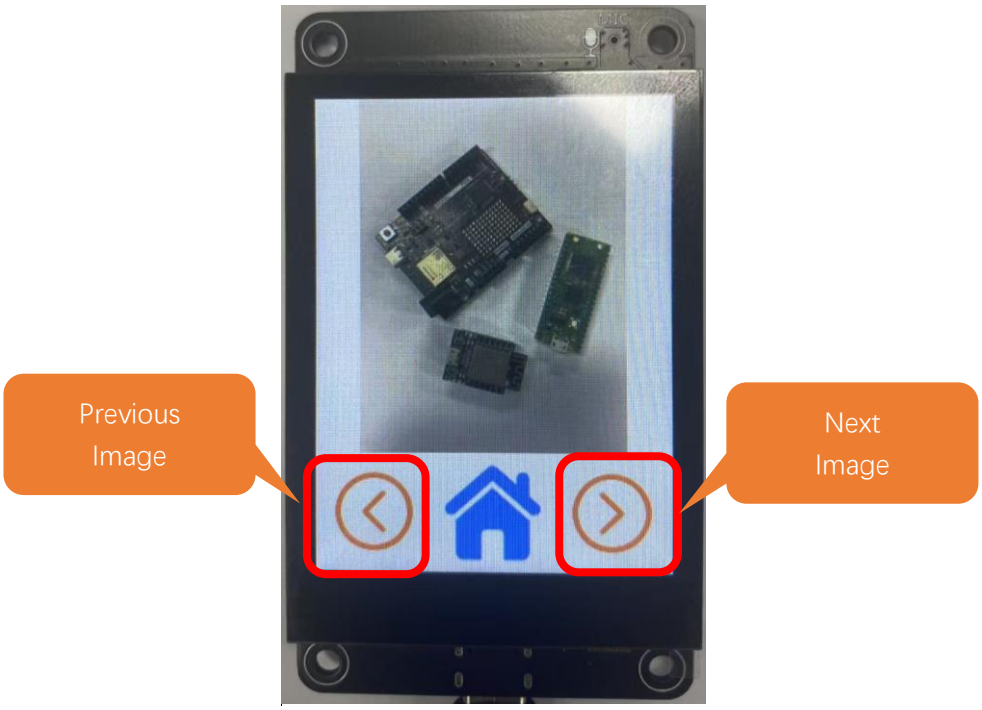
```
54  screen.routine(); /* let the GUI do its work */
```

Copy the Picture folder under the **Freenove_ESP32_S3_Display\Sketches\Sketch_14.1_Lvgl_Picture** directory to the SD card root directory.



Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200

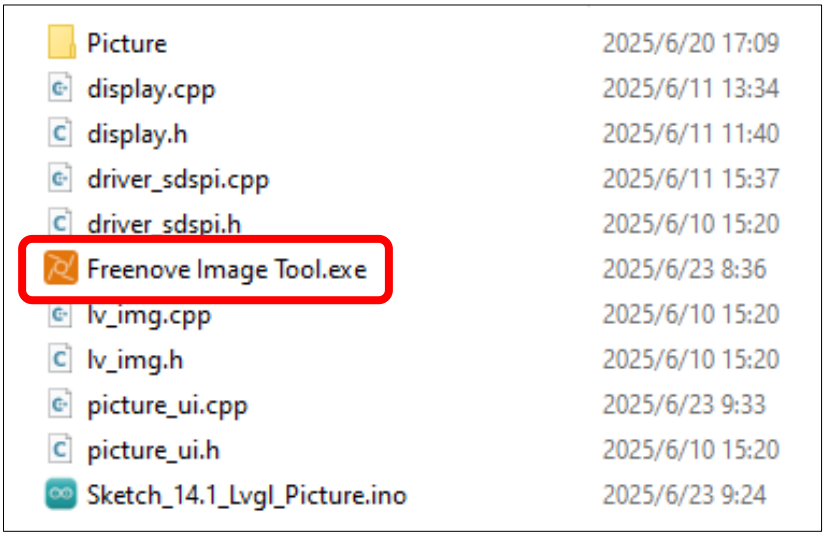




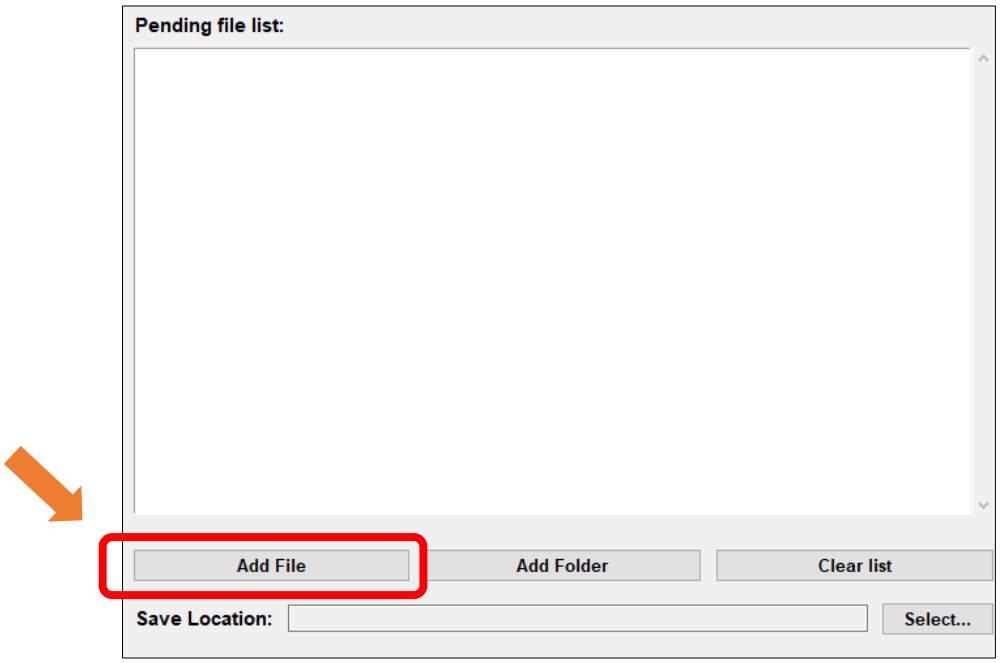
Custom image display

You can customize the image displayed on the display according to your personal preferences.

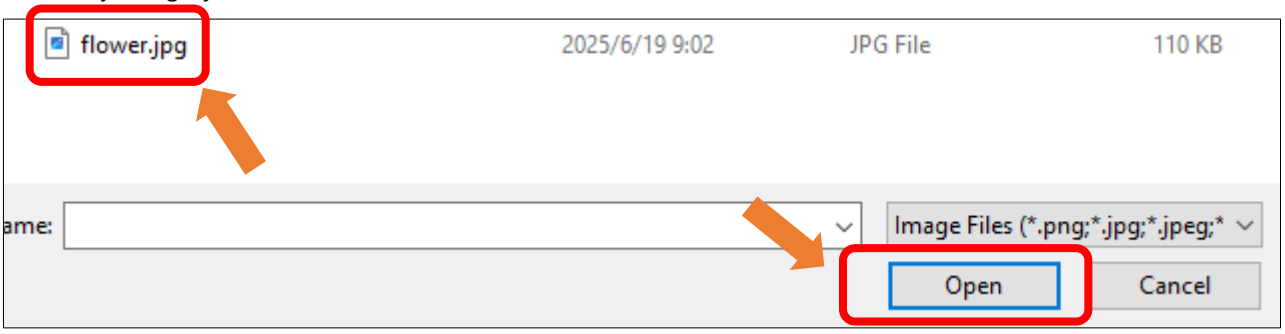
First, open **Freenove_ESP32_S3_Display\Sketches\Sketch_14.1_Lvgl_Picture\Freenove Image Tool.exe**



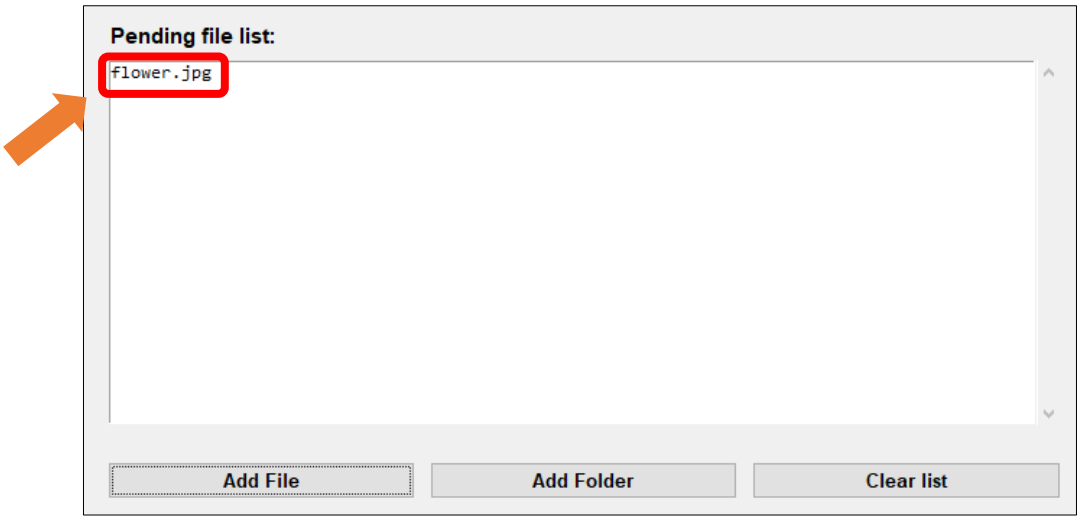
Click **"Add File"**



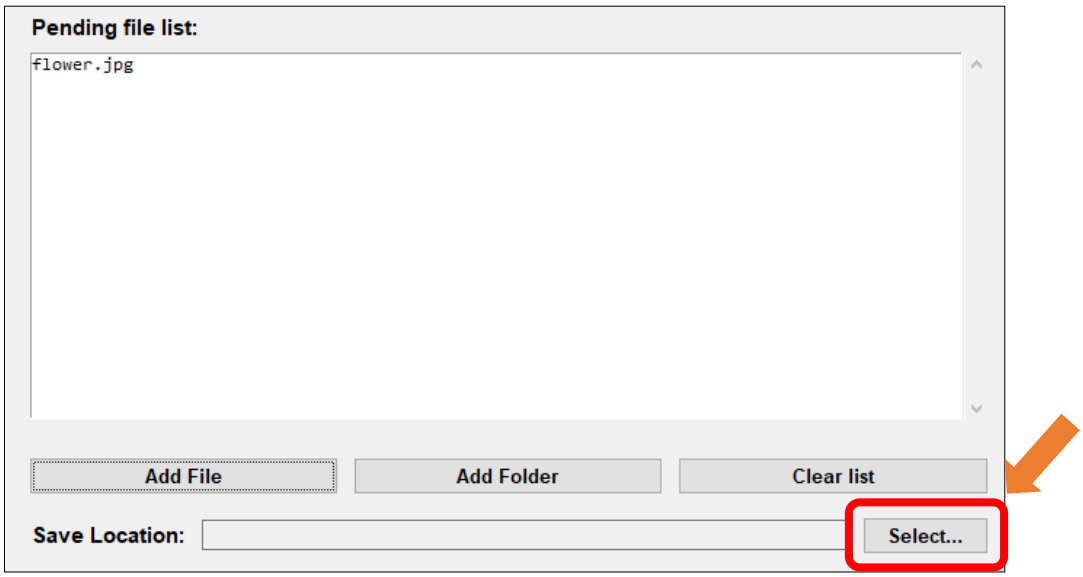
Select any image you like.



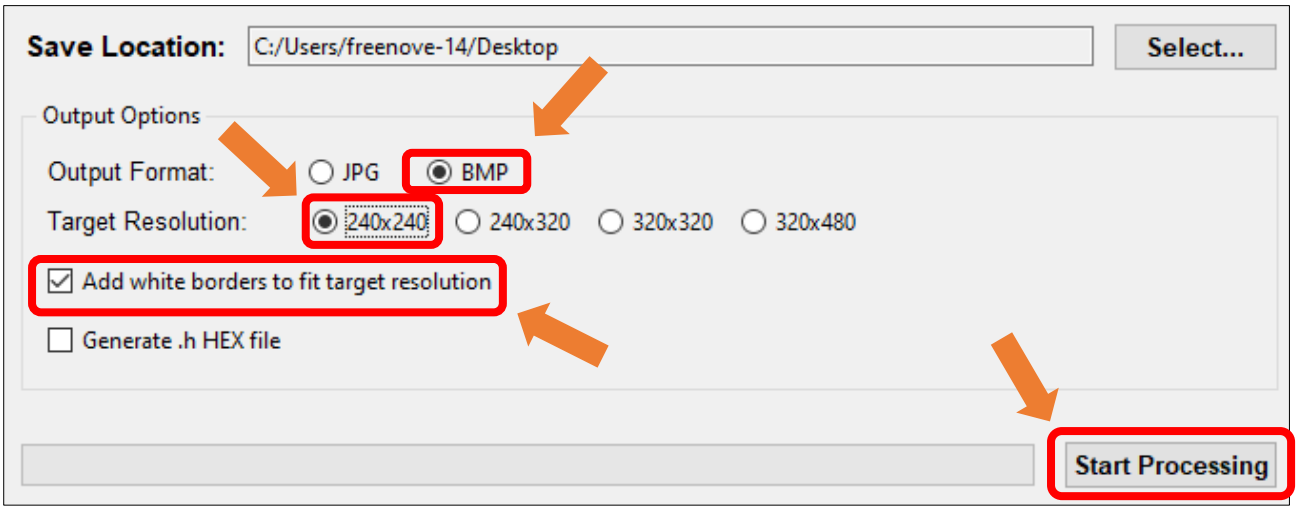
The image files from your folder will now appear in the **Pending File List**.



Click "Select…" to choose the image saving location.

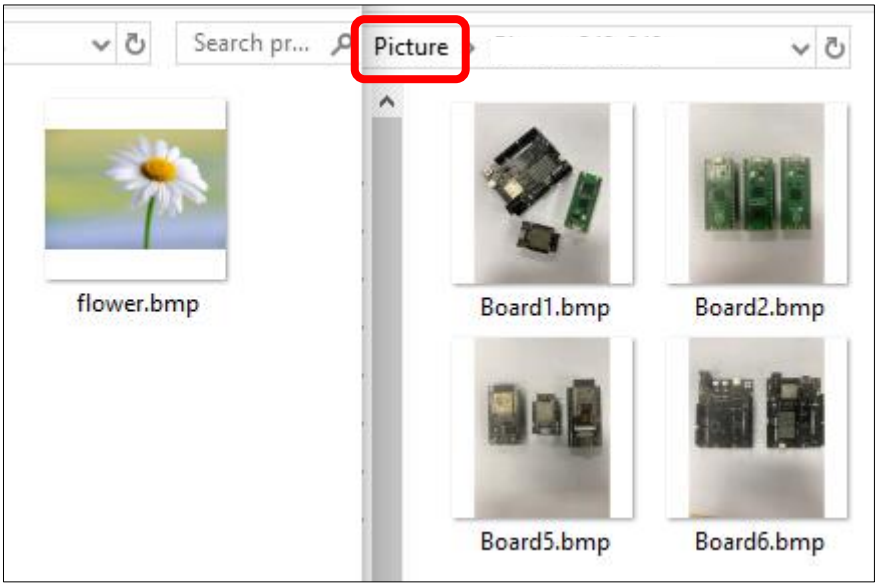


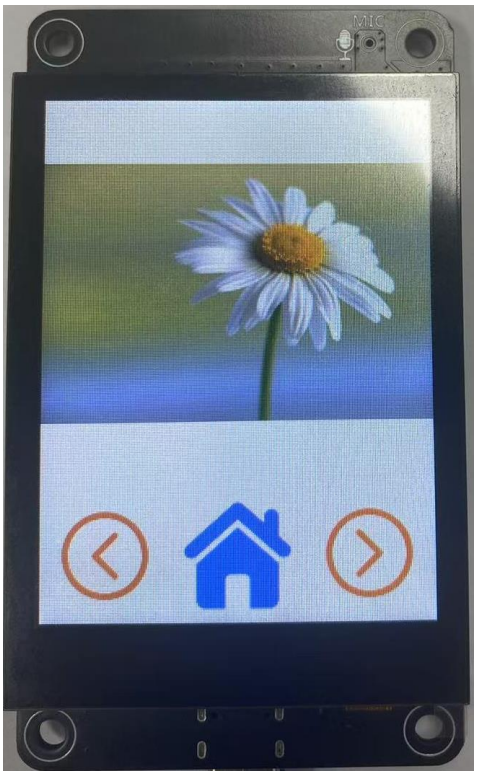
Select **BMP** for the format, set the resolution to **240x240**.



Wait for the progress bar to complete.

Copy or replace the generated image to the folder corresponding to the Picture folder in the root directory of the SD card.





Chapter 15 LVGL Timer

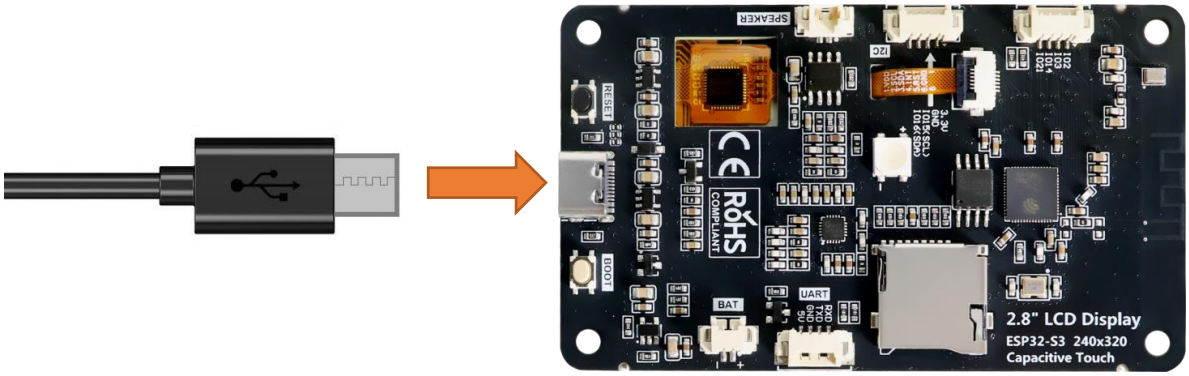
Project 15.1 LVGL Timer

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Open "**Sketch_15.1_Lvgl_Timer**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_15.1_Lvgl_Timer.ino**".

Sketch_15.1_Lvgl_Timer

The following is the program code:

```
1  #include "display.h"
2  #include "chronograph_ui.h"
3
4  Display screen;
5  void setup() {
```

```
6     Serial.begin(115200);
7
8     /*** Init screen ***/
9     screen.init();
10
11    /***  Print lvgl version ***/
12    String LVGL_Arduino = "Hello Arduino! ";
13    LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
14  lv_version_patch();
15    Serial.println(LVGL_Arduino);
16    Serial.println("I am LVGL_Arduino");
17    Serial.println("Setup done");
18
19    /*** The custom code ***/
20    setup_scr_chronograph(&guider_chronograph_ui);
21    lv_scr_load(guider_chronograph_ui.chronograph);
22  }
23
24  void loop() {
25    screen.routine();
26    delay(1);
27  }
```

Code Explanation

Include the header files.

```
7   #include "display.h"
8   #include "chronograph_ui.h"
```

Set the baud rate to 115200.

```
20  Serial.begin( 115200 );
```

Initialize configuration.

```
23  screen.init();
```

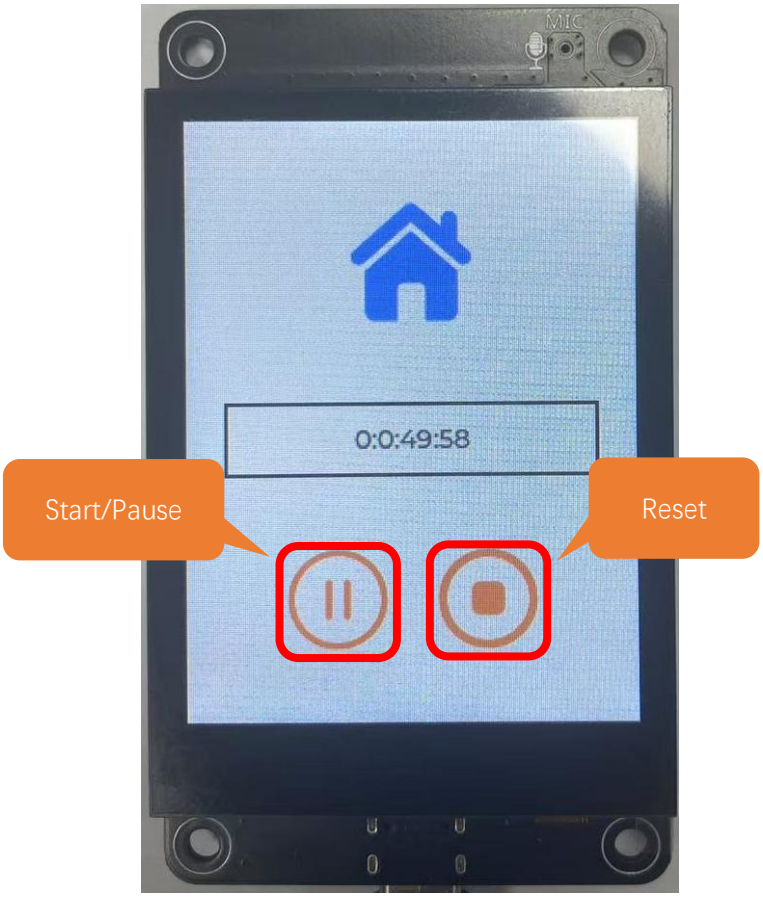
Create and load the interface.

```
32  setup_scr_chronograph(&guider_chronograph_ui);
33  lv_scr_load(guider_chronograph_ui.chronograph);
```

LVGL task processor.

```
39  screen.routine(); /* let the GUI do its work */
```

Click "**Upload**" to upload the code to Freenove ESP32-S3 Display. Set the baud rate to 115200.



Chapter 16 Lvgl RGB

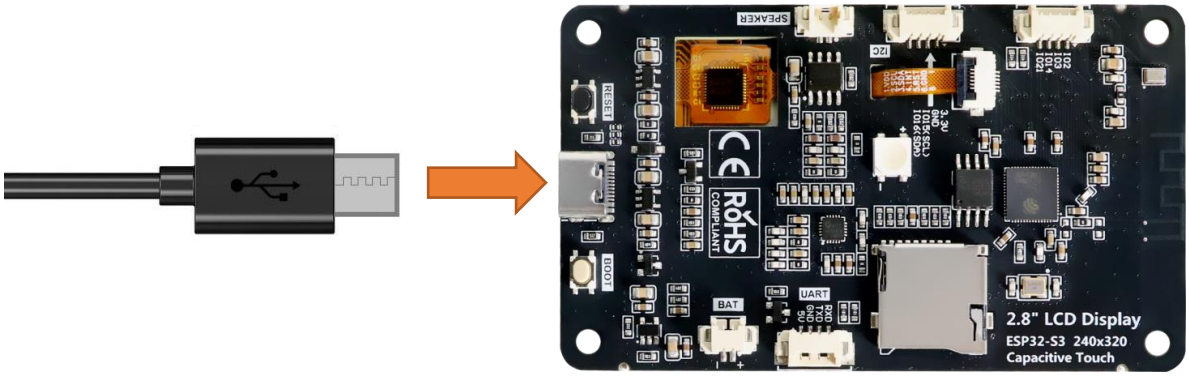
Project 16.1 LVGL RGB

Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 

Circuit

Connect Freenove ESP32 -S3 to the computer using the USB cable.



Sketch

Open "**Sketch_16.1_Lvgl_RGB**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_16.1_Lvgl_RGB.ino**".

Sketch_16.1_Lvgl_RGB

The following is the program code:

```
1  #include "display.h"
2  #include "rgb_ui.h"
3
4  #define RED_PIN 22
5  #define GREEN_PIN 16
```

```
6  #define BLUE_PIN 17
7
8  Display screen;
9  void setup() {
10     Serial.begin(115200);
11
12     /*** Init screen ***/
13     screen.init();
14
15     /***  Print lvgl version ***/
16     String LVGL_Arduino = "Hello Arduino! ";
17     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
18 lv_version_patch();
19     Serial.println(LVGL_Arduino);
20     Serial.println("I am LVGL_Arduino");
21     Serial.println("Setup done");
22
23     /*** The custom code ***/
24     rgb_init(RED_PIN, GREEN_PIN, BLUE_PIN);
25     rgb_set_color(0, 0, 0);
26     setup_scr_rgb(&guider_rgb_ui);
27     lv_scr_load(guider_rgb_ui.rgb);
28 }
29
30 void loop() {
31     screen.routine();
32     delay(5);
33 }
```

Code Explanation

Include the header files.

```
1  #include <lvgl.h>
2  #include "Arduino.h"
3  #include "display.h"
4  #include "ws2812_ui.h"
```

Set the baud rate to 115200

```
20 Serial.begin( 115200 );
```

Initialize the RGB LED.

```
23 screen.init();
```

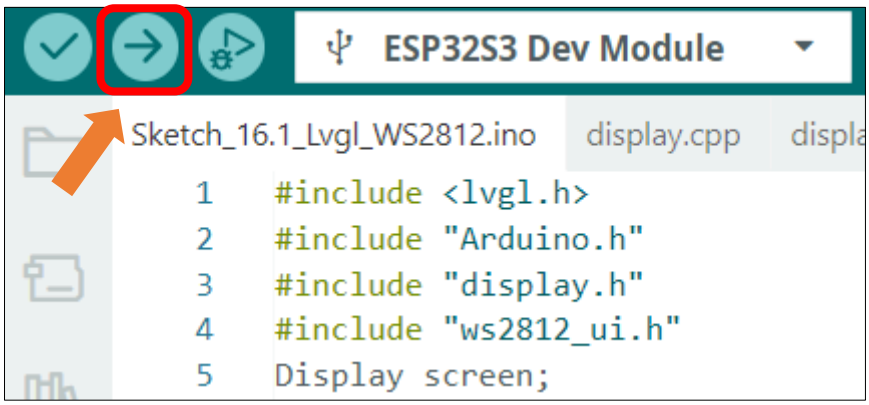
Create and load the interface.

```
32 setup_scr_rgb(&guider_rgb_ui);
33 lv_scr_load(guider_rgb_ui.rgb);
```

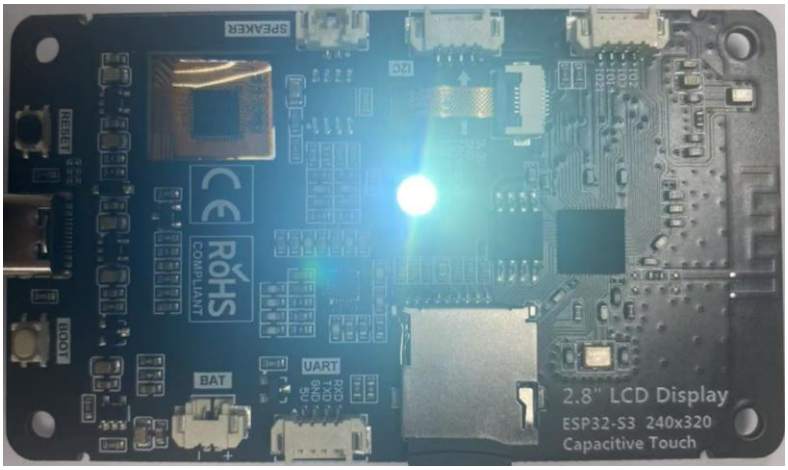
LVGL task processor.

```
39 screen.routine(); /* let the GUI do its work */
```

Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



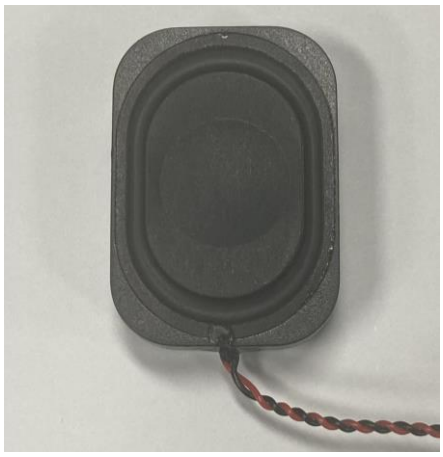
Drag to adjust the value of the red (R), green (G), and blue (B) color, and you will see the color of the LED change.



Chapter 17 LVGL Music

Project 17.1 LVGL Music

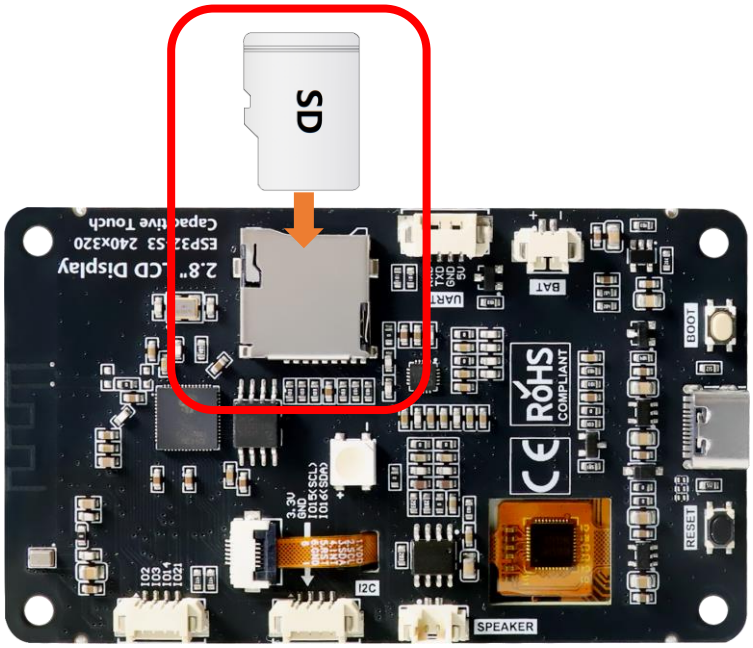
Component List

Freenove ESP32-S3 Display x 1 	USB cable x1 
Speaker x1 	

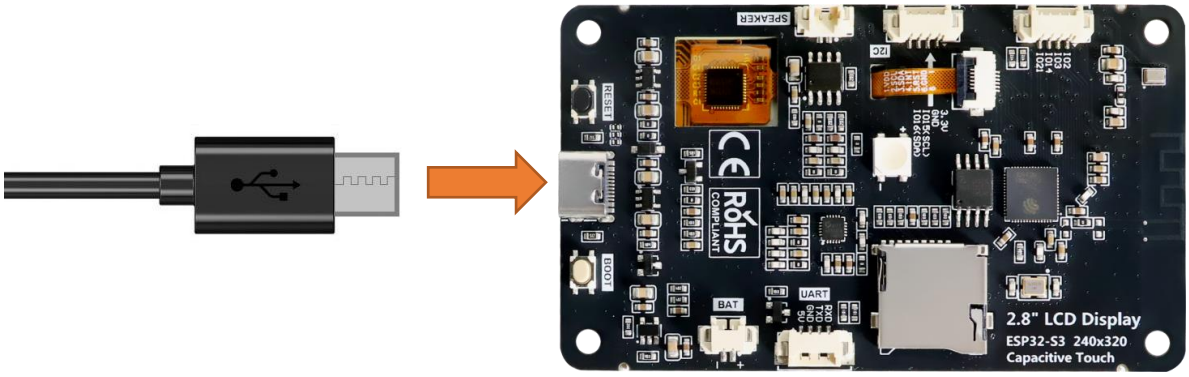
Note: This kit does not include SD card, or SD card reader. Please buy them yourself!

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Open "**Sketch_17.1_Lvgl_Music**" folder under **"Freenove_ESP32_S3_Display\Sketches"** and double-click "**Sketch_17.1_Lvgl_Music.ino**".

Sketch_17.1_Lvgl_Music

The following is the program code:

```
1  #include <Arduino.h>
2  #include "Wire.h"
3  #include "display.h"
4  #include "driver_sdmmc.h"
5  #include "music_ui.h"
6  #include "es8311.h"
7  #include "ESP_I2S.h"
```

```
8
9   #define SD_MMC_CMD 40  // Please do not modify it.
10  #define SD_MMC_CLK 38  // Please do not modify it.
11  #define SD_MMC_D0  39  // Please do not modify it.
12  #define SD_MMC_D1  41  // Please do not modify it.
13  #define SD_MMC_D2  48  // Please do not modify it.
14  #define SD_MMC_D3  47  // Please do not modify it.
15
16  //I2S IO Pin define
17  #define I2S_MCK 4
18  #define I2S_BCK 5
19  #define I2S_DINT 6
20  #define I2S_DOUT 8
21  #define I2S_WS 7
22  #define AP_ENABLE 1
23  #define I2C_SCL 15        /*!< GPIO number used for I2C master clock */
24  #define I2C_SDA 16        /*!< GPIO number used for I2C master data  */
25  #define I2C_SPEED 400000  /*!< I2C master clock frequency */
26
27  Display screen;
28  I2SClass es8311_i2s;
29
30  void driver_es8311_init(void) {
31     pinMode(AP_ENABLE, OUTPUT);
32     digitalWrite(AP_ENABLE, LOW);
33
34     Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);
35
36     es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
37     if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO,
38  I2S_STD_SLOT_LEFT)) {
39        Serial.println("Failed to initialize I2S bus!");
40     }
41  }
42
43  void setup(){
44       /* prepare for possible serial debug */
45       Serial.begin( 115200 );
46
47      /*** Init drivers ***/
48       sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2,
49  SD_MMC_D3);//Initialize the SD module
50       driver_es8311_init();
51       if (es8311_codec_init() != ESP_OK) {
```

```
52          Serial.println("ES8311 init failed!");
53          return;
54        }
55        screen.init();
56
57        String LVGL_Arduino = "Hello Arduino! ";
58        LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
59    lv_version_patch();
60        Serial.println( LVGL_Arduino );
61        Serial.println( "I am LVGL_Arduino" );
62
63        setup_scr_music(&guider_music_ui);
64        lv_scr_load(guider_music_ui.music);
65
66        Serial.println( "Setup done" );
67    }
68
69    void loop(){
70        screen.routine(); /* let the GUI do its work */
71        delay( 5 );
72    }
```

Code Explanation

Include the header files.

```
7   #include <Arduino.h>
8   #include "Wire.h"
9   #include "display.h"
10  #include "driver_sdmmc.h"
11  #include "music_ui.h"
12  #include "es8311.h"
13  #include "ESP_I2S.h"
```

Define the pins.

```
15  #define SD_MMC_CMD 40  // Please do not modify it.
16  #define SD_MMC_CLK 38  // Please do not modify it.
17  #define SD_MMC_D0  39  // Please do not modify it.
18  #define SD_MMC_D1  41  // Please do not modify it.
19  #define SD_MMC_D2  48  // Please do not modify it.
20  #define SD_MMC_D3  47  // Please do not modify it.
21
22  //I2S IO Pin define
23  #define I2S_MCK 4
24  #define I2S_BCK 5
25  #define I2S_DINT 6
26  #define I2S_DOUT 8
27  #define I2S_WS 7
```

```
#define AP_ENABLE 1
#define I2C_SCL 15          /*!< GPIO number used for I2C master clock */
#define I2C_SDA 16          /*!< GPIO number used for I2C master data  */
#define I2C_SPEED 400000    /*!< I2C master clock frequency */
```

Set the baud rate to 115200

```
Serial.begin( 115200 );
```

Initialize configuration.

```
sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2, SD_MMC_D3);//Initialize
the SD module
driver_es8311_init();
if (es8311_codec_init() != ESP_OK) {
    Serial.println("ES8311 init failed!");
    return;
}
screen.init();
```

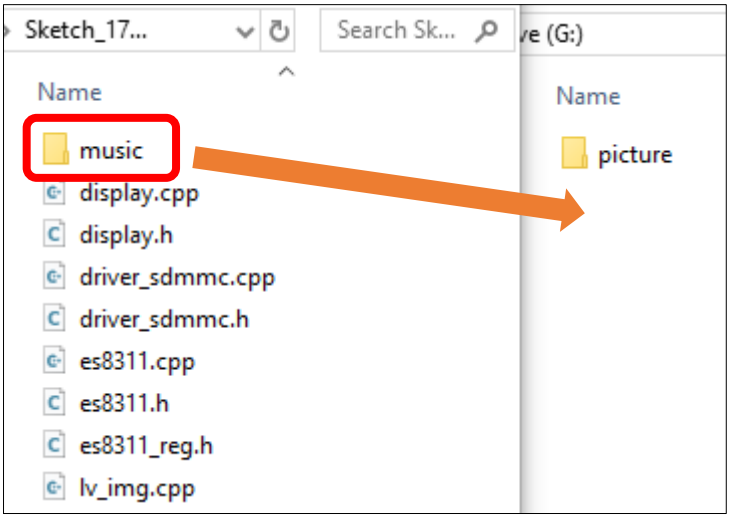
Create and load the interface.

```
setup_scr_music(&guider_music_ui);
lv_scr_load(guider_music_ui.music);
```

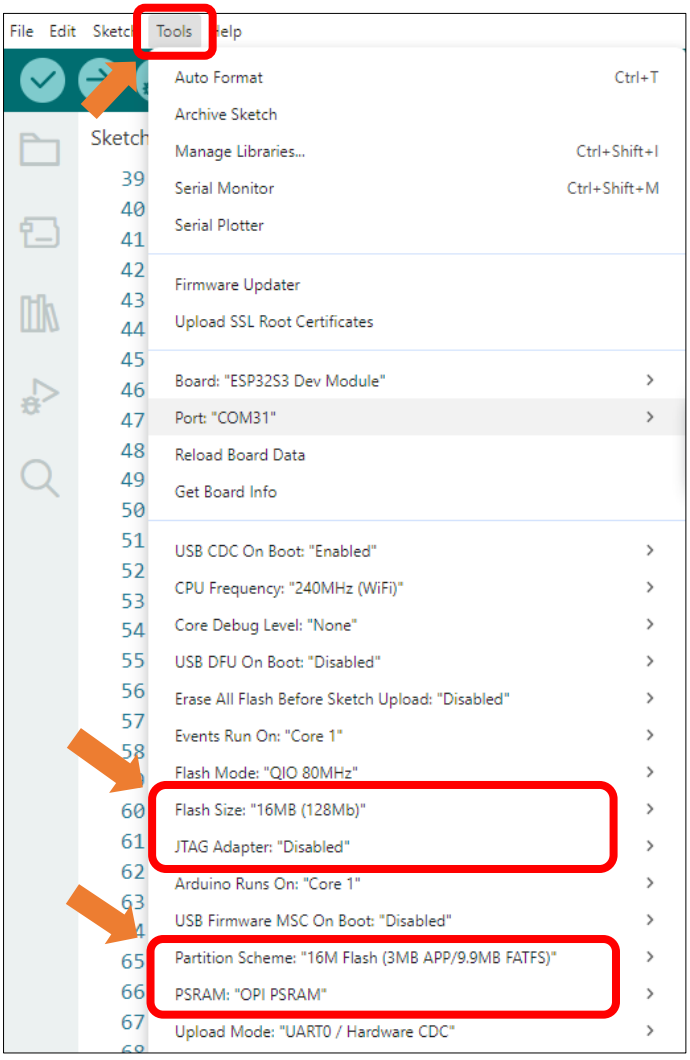
LVGL task processor.

```
screen.routine(); /* let the GUI do its work */
```

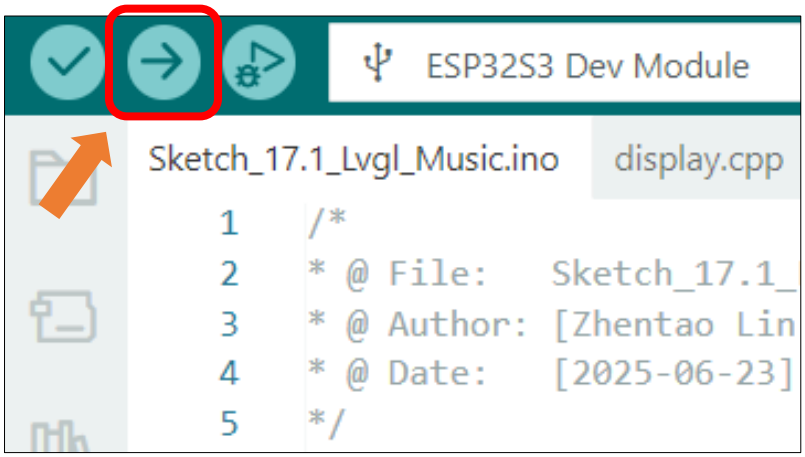
Insert the SD card to the card reader and plug them to the computer. Copy the **Music** folder under the **Freenove_ESP32_S3_Display\Sketches\Sketch_17.1_Lvgl_Music** directory to the root directory of the SD card.

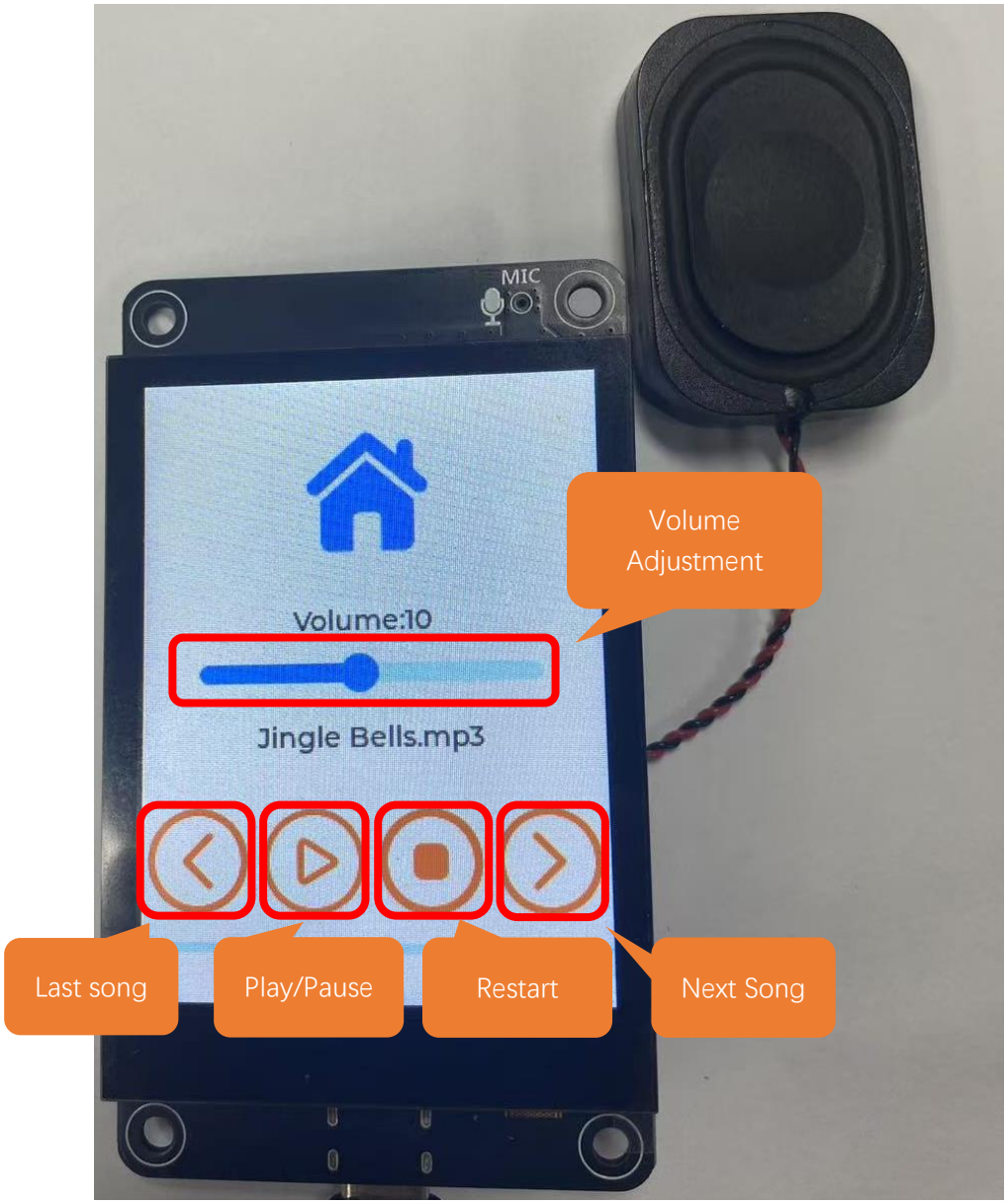


It is necessary to change the settings in Arduino IDE before clicking the Uploading button, as shown below.
Caution: Incorrect settings will result in compilation error or uploading failure. To achieve desired result, please configure exactly the same as below.



Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



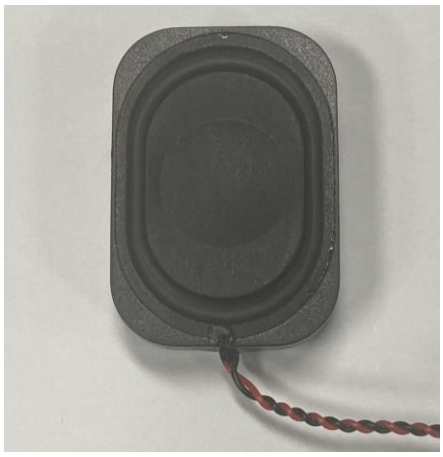


Note: If the screen flickers during playback, it may be due to insufficient power supply. You can try powering with a battery.

Chapter 18 LVGL Multifunctionality

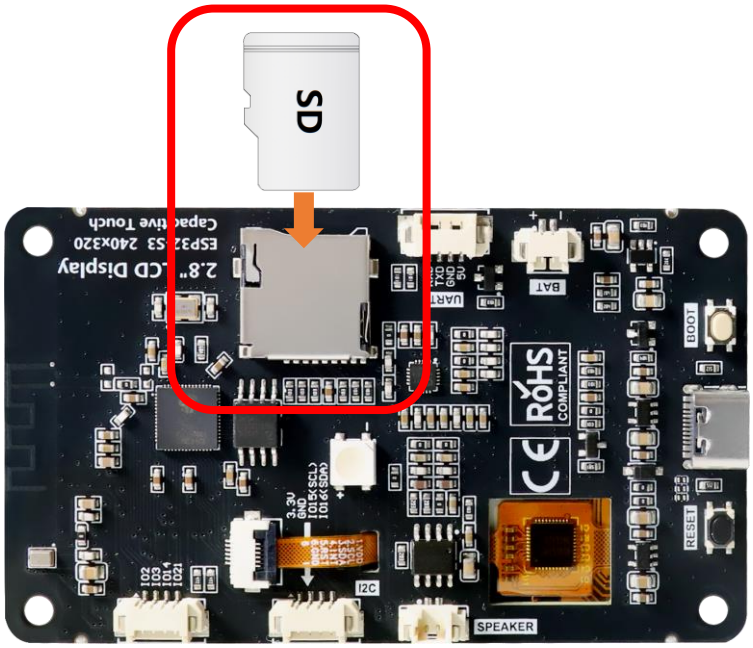
Project 18.1 LVGL Multifunctionality

Component List

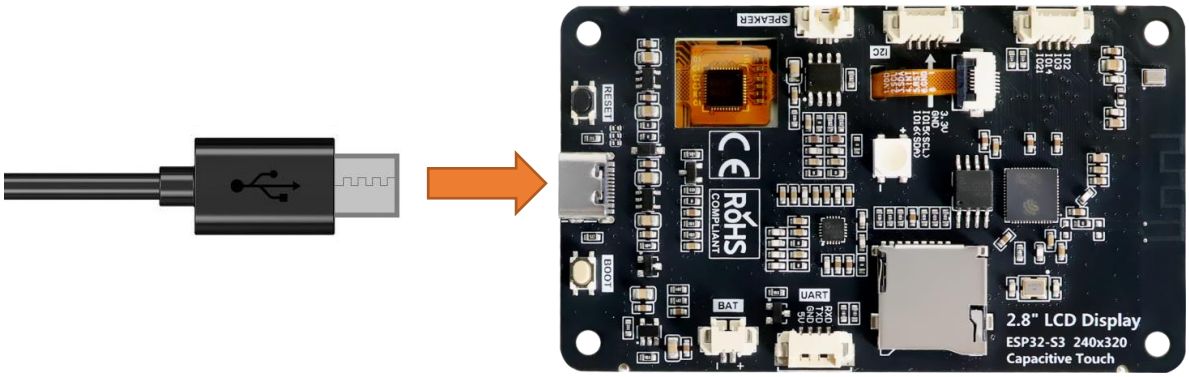
Freenove ESP32-S3 Display x 1 	USB cable x1 
Speaker x1 	

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Open“**Sketch_18.1_Lvgl_Multifunctionality**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_18.1_Lvgl_ Multifunctionality.ino**”.

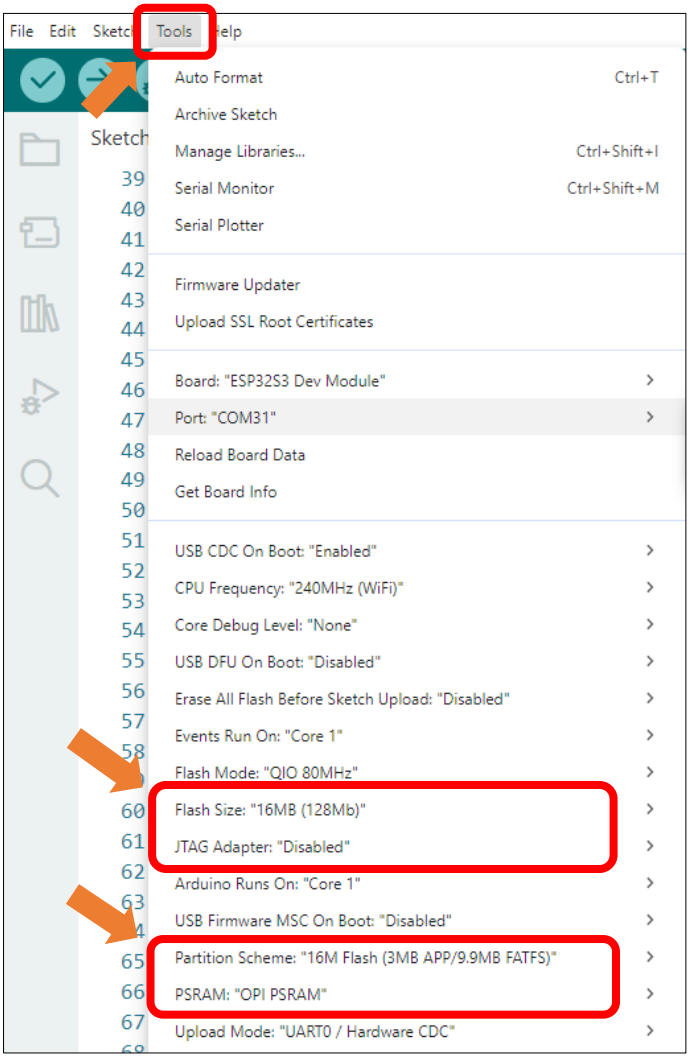
Sketch_18.1_Lvgl_Multifunctionality

The following is the program code:

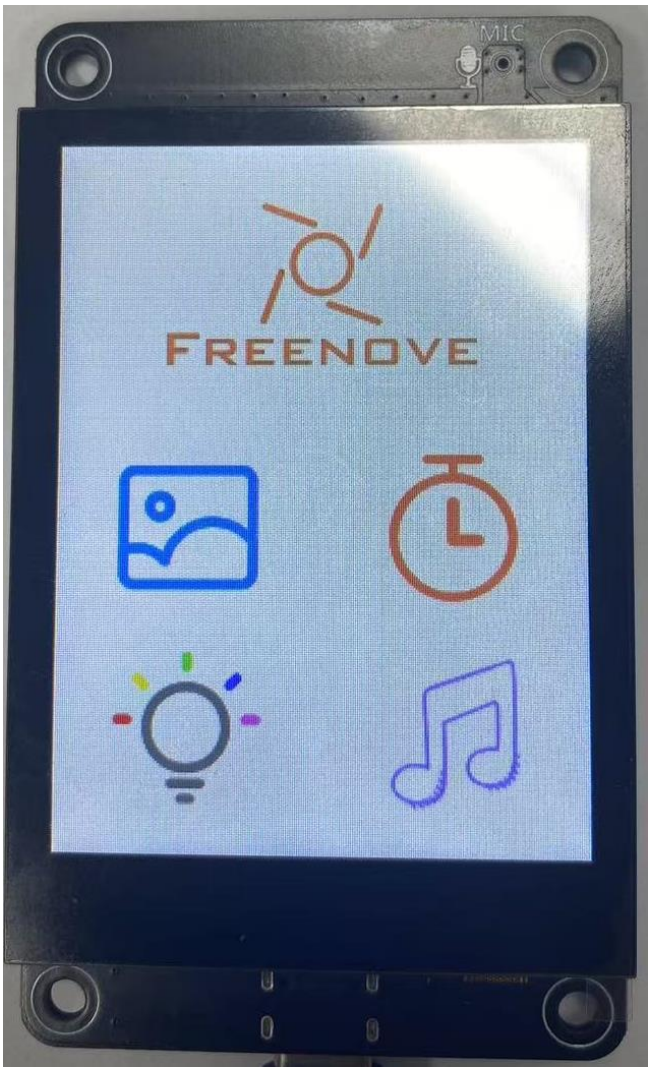
```
1  #include "public.h"
2
3  Display screen;
4  I2SClass es8311_i2s;
5
6  void driver_es8311_init(void) {
```

```
7     pinMode(AP_ENABLE, OUTPUT);
8     digitalWrite(AP_ENABLE, LOW);
9
10    Wire.begin(I2C_SDA, I2C_SCL, I2C_SPEED);
11
12    es8311_i2s.setPins(I2S_BCK, I2S_WS, I2S_DOUT, I2S_DINT, I2S_MCK);
13    if (!es8311_i2s.begin(I2S_MODE_STD, 44100, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO,
14  I2S_STD_SLOT_LEFT)) {
15      Serial.println("Failed to initialize I2S bus!");
16    }
17  }
18
19  void setup(){
20      /* prepare for possible serial debug */
21      Serial.begin( 115200 );
22
23     /*** Init drivers ***/
24      sdmmc_init(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0, SD_MMC_D1, SD_MMC_D2,
25  SD_MMC_D3);//Initialize the SD module
26      driver_es8311_init();
27      if (es8311_codec_init() != ESP_OK) {
28        Serial.println("ES8311 init failed!");
29        return;
30      }
31      screen.init();
32
33      String LVGL_Arduino = "Hello Arduino! ";
34      LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
35  lv_version_patch();
36      Serial.println( LVGL_Arduino );
37      Serial.println( "I am LVGL_Arduino" );
38
39      setup_scr_main(&guider_main_ui);
40      lv_scr_load(guider_main_ui.main);
41
42      Serial.println( "Setup done" );
43  }
44
45  void loop(){
46      screen.routine(); /* let the GUI do its work */
47      delay( 5 );
48  }
```

It is necessary to change the settings in Arduino IDE before clicking the Uploading button, as shown below.
Caution: Incorrect settings will result in compilation error or uploading failure. To achieve desired result, please configure exactly the same as below.



Click "**Upload**" to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.


MIC
FREENOVE

Chapter 19 LVGL Arduino

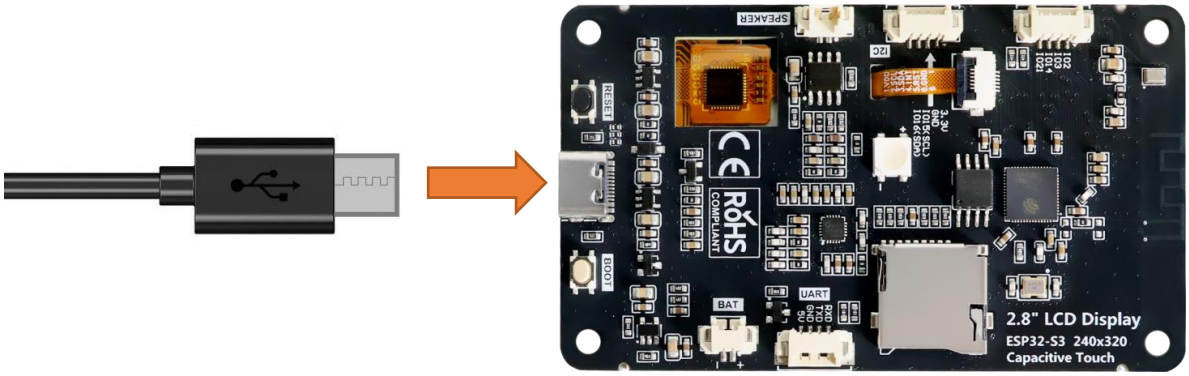
Project 19.1 LVGL Arduino

Component List

Freenove ESP32-S3 Display x 1	USB cable x1
	

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Open“**Sketch_19.1_Lvgl_Arduino**” folder under **“Freenove_ESP32_S3_Display\Sketches”** and double-click “**Sketch_19.1_Lvgl_Arduino.ino**”.

Sketch_19.1_Lvgl_Arduino

The following is the program code:

```
1  #include <lvgl.h>
2  #include "demos/lv_demos.h"
3  #include <TFT_eSPI.h>
4  #include "FT6336U.h"
```

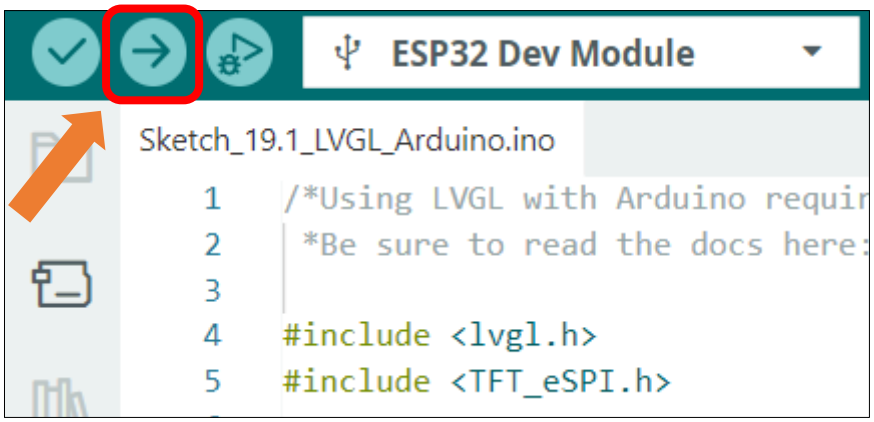
```
5
6   /*To use the built-in examples and demos of LVGL uncomment the includes below respectively.
7     *You also need to copy `lvgl/examples` to `lvgl/src/examples`. Similarly for the demos
8   `lvgl/demos` to `lvgl/src/demos`.
9     Note that the `lv_examples` library is for LVGL v7 and you shouldn't install it for this
10  version (since LVGL v8)
11    as the examples and demos are now part of the main LVGL library. */
12
13  /*Change to your screen resolution*/
14  #define TFT_DIRECTION 0   //Select TFT Direction (0 - 3)
15  #define I2C_SCL 15
16  #define I2C_SDA 16
17  #define INT_N_PIN 17
18  #define RST_N_PIN 18
19  #define TFT_SCREEN_WIDTH 240
20  #define TFT_SCREEN_HEIGHT 320
21
22  static const uint16_t screenWidth = TFT_SCREEN_WIDTH;
23  static const uint16_t screenHeight = TFT_SCREEN_HEIGHT;
24  static const uint16_t screenHeightBuf = TFT_SCREEN_HEIGHT / 10;
25  static lv_disp_draw_buf_t draw_buf;
26  static lv_color_t draw_buf1[screenWidth * screenHeightBuf];
27
28  TFT_eSPI tft = TFT_eSPI(screenWidth, screenHeight); /* TFT instance */
29  FT6336U ft6336u(I2C_SDA, I2C_SCL, RST_N_PIN, INT_N_PIN);
30  FT6336U_TouchPointType tp;
31
32  #if LV_USE_LOG != 0
33  /* Serial debugging */
34  void my_print(const char * buf)
35  {
36      Serial.printf(buf);
37      Serial.flush();
38  }
39  #endif
40
41  /* Display flushing */
42  void my_disp_flush( lv_disp_drv_t *disp_drv, const lv_area_t *area, lv_color_t *color_p )
43  {
44      uint32_t w = ( area->x2 - area->x1 + 1 );
45      uint32_t h = ( area->y2 - area->y1 + 1 );
46
47      tft.startWrite();
48      tft.setAddrWindow( area->x1, area->y1, w, h );
```

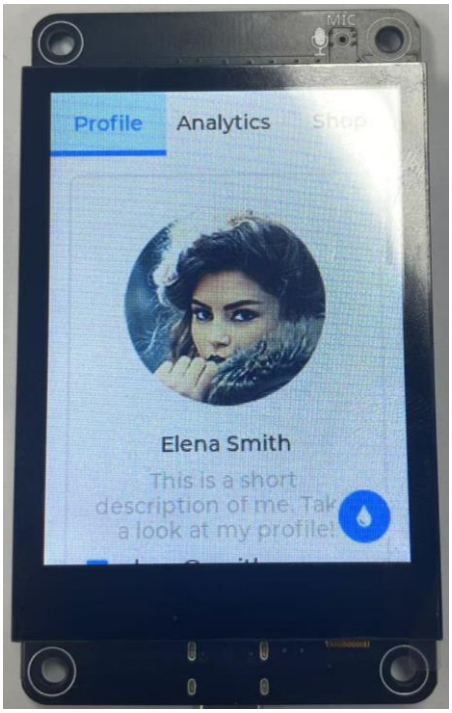
```
49      tft.pushColors( ( uint16_t * )&color_p->full, w * h, true );
50      tft.endWrite();
51
52      lv_disp_flush_ready( disp_drv );
53  }
54
55  void my_touchpad_read( lv_indev_drv_t * indev_driver, lv_indev_data_t * data )
56  {
57      uint16_t touchX, touchY;
58
59      //bool touched = tft.getTouch( &touchX, &touchY, 600 );
60      tp = ft6336u.scan();
61      int touched = tp.touch_count;
62
63      if( !touched )
64      {
65          data->state = LV_INDEV_STATE_REL;
66      }
67      else
68      {
69          int x = tp.tp[0].x;
70          int y = tp.tp[0].y;
71          if(x >= 0 && x < screenWidth && y >= 0 && y < screenHeight)
72          {
73              data->state = LV_INDEV_STATE_PR;
74              data->point.x = tp.tp[0].x;
75              data->point.y = tp.tp[0].y;
76          }
77      }
78  }
79
80  void setup()
81  {
82      Serial.begin( 115200 ); /* prepare for possible serial debug */
83
84      String LVGL_Arduino = "Hello Arduino! ";
85      LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
86  lv_version_patch();
87
88      Serial.println( LVGL_Arduino );
89      Serial.println( "I am LVGL_Arduino" );
90
91      tft.begin();          /* TFT init */
92      tft.setRotation(TFT_DIRECTION);
```

```
93      ft6336u.begin();
94
95      lv_init();
96
97  #if LV_USE_LOG != 0
98      lv_log_register_print_cb( my_print ); /* register print function for debugging */
99  #endif
100
101     lv_disp_draw_buf_init(&draw_buf, draw_buf1, NULL, screenWidth * screenHeightBuf);
102
103     /*Initialize the display*/
104     static lv_disp_drv_t disp_drv;
105     lv_disp_drv_init( &disp_drv );
106     /*Change the following line to your display resolution*/
107     disp_drv.hor_res = screenWidth;
108     disp_drv.ver_res = screenHeight;
109     disp_drv.flush_cb = my_disp_flush;
110     disp_drv.draw_buf = &draw_buf;
111     lv_disp_drv_register( &disp_drv );
112
113     /*Initialize the (dummy) input device driver*/
114     static lv_indev_drv_t indev_drv;
115     lv_indev_drv_init( &indev_drv );
116     indev_drv.type = LV_INDEV_TYPE_POINTER;
117     indev_drv.read_cb = my_touchpad_read;
118     lv_indev_drv_register( &indev_drv );
119
120     /* Create simple label */
121     // lv_obj_t *label = lv_label_create( lv_scr_act() );
122     // lv_label_set_text( label, "Hello Ardino and LVGL!");
123     // lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );
124
125     /* Try an example. See all the examples
126      * online: https://docs.lvgl.io/master/examples.html
127      * source codes:
128 https://github.com/lvgl/lvgl/tree/e7f88efa5853128bf871dde335c0ca8da9eb7731/examples */
129      //lv_example_btn_1();
130
131      /*Or try out a demo. Don't forget to enable the demos in lv_conf.h. E.g.
132 LV_USE_DEMOS_WIDGETS*/
133     lv_demo_widgets();
134     // lv_demo_benchmark();
135     // lv_demo_keypad_encoder();
136     // lv_demo_music();
```

```
137       // lv_demo_printer();
138       // lv_demo_stress();
139
140       Serial.println( "Setup done" );
141   }
142
143   void loop()
144   {
145       lv_timer_handler(); /* let the GUI do its work */
146       delay( 5 );
147   }
```

Click "**Upload**" to upload the code to Freenove ESP32 Display





Note: The examples in this section require touch interaction. If you are using a non-touchscreen version of the Freenove ESP32-S3 Display, the interface will render correctly but will not respond to touch input. Nevertheless, You may still use these examples as a reference for exploring other LVGL features.

What's next?

Thank you again for choosing Freenove products.

THANK YOU for participating in this learning experience!

We have reached the end of this Tutorial. If you find errors, omissions, or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us:
support@freenove.com

We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you want to learn more about Arduino, ESP32, Raspberry Pi Pico W, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost effective, innovative and exciting products.

Thank you again for choosing Freenove products.